FPGA course · Exam preparation · 4-hour written exam

FPGA Prototyping by VHDL Examples

Condensed from Pong P. Chu, *FPGA Prototyping by VHDL Examples*, Xilinx Spartan-3 version (Wiley, 2008), Chapters 1–17 and Appendix A, with a digital-logic foundations part (gates, chips, ALU, flip-flops, RAM/ROM, SAP-1, and SAP-1 from gates to VHDL).

Key concepts · VHDL code templates · block, state and ASMD diagrams · timing waveforms · every number worth remembering · worked calculations · compare-and-contrast tables · glossary · practice questions.

References such as **[§4.5.3]** point to the section numbers in the book, so you can jump straight to the full text and code listing.

Contents

0 How to use this summary 4
- 0.1 The book in one picture . . . 4
- 0.2 What a 4-hour written exam on this book tends to ask . . . 4
- 0.3 Box legend . . . 5

Part 0 Digital logic foundations

What the VHDL describes: gates, chips, adders and ALU, flip-flops, registers, RAM/ROM, the SAP-1 computer, and SAP-1 built from gates and in VHDL 6

A Numbers and Boolean algebra 7
- A.1 Number systems . . . 7
- A.2 Boolean algebra . . . 7
- A.3 Karnaugh maps . . . 8

B Logic gates, logic families and chips 9
- B.1 The basic gates . . . 9
- B.2 Logic families: TTL and CMOS . . . 9
- B.3 Timing of real gates . . . 10
- B.4 74xx chips worth recognising . . . 10

C Combinational building blocks and the ALU 12
- C.1 Multiplexer and demultiplexer . . . 12
- C.2 Decoder, encoder, comparator . . . 12
- C.3 Adders . . . 13
- C.4 Subtractors and the adder-subtractor . . . 13
- C.5 The ALU: in parts and as a whole . . . 13

D Latches, flip-flops, registers and counters 15
- D.1 From feedback to memory: the SR latch . . . 15
- D.2 Edge-triggered D flip-flop . . . 15
- D.3 JK and T flip-flops . . . 15
- D.4 Timing and metastability . . . 16
- D.5 Registers and shift registers . . . 16
- D.6 Counters . . . 16

E How memory works: RAM and ROM 18
- E.1 Organisation and signals . . . 18
- E.2 A small RAM built from latches (how it really works) 18
- E.3 SRAM and DRAM cells . . . 18
- E.4 ROM: memory that is really a truth table . . . 19
- E.5 Programmable logic: from ROM to FPGA . . . 19

F A simple CPU: SAP-1 21
- F.1 What SAP-1 is . . . 21
- F.2 The blocks and their control signals . . . 21
- F.3 Instruction set . . . 22
- F.4 T-states and control words . . . 22
- F.5 The controller-sequencer (hardwired control) . . . 22
- F.6 Tracing a program . . . 23
- F.7 SAP-1 in VHDL (Chu's FSMD style) . . . 23

G From logic gates to a computer 25
- G.1 The ladder: from a gate to a computer . . . 25
- G.2 The W bus: tri-state wires or an AND-OR multiplexer . . . 26
- G.3 The register: one bit cell, eight times . . . 27
- G.4 The program counter: a counter built from T flip-flops . . . 27
- G.5 The adder-subtractor: XOR gates plus a ripple of full adders . . . 28
- G.6 The memory: a decoder, a grid of cells and an OR per column . . . 29
- G.7 The control unit in gates . . . 30
- G.8 One bit of the whole data path . . . 32
- G.9 The top level: the block diagram written as VHDL 33
- G.10 One instruction through all three views . . . 35
- G.11 The same computer as one FSMD . . . 35
- G.12 Extending it: JMP and JZ in gates and in VHDL . 36

Part I Basic digital circuits

VHDL, FPGA flow, combinational & sequential design, FSM, FSMD 38

1 Gate-level combinational circuits 39
- 1.1 VHDL in one paragraph . . . 39
- 1.2 The skeleton of every design . . . 39
- 1.3 Data types and operators . . . 39
- 1.4 Structural description . . . 40
- 1.5 Testbench . . . 40

2 FPGA devices and EDA software 41
- 2.1 What an FPGA is . . . 41
- 2.2 Xilinx Spartan-3 . . . 41
- 2.3 The Digilent S3 board . . . 42
- 2.4 Development flow . . . 42
- 2.5 Tools (ISE WebPack 8.2, ModelSim XE) . . . 42

3 RT-level combinational circuits 44
- 3.1 Operators and the `numeric_std` package . . . 44
- 3.2 Routing: concurrent assignment statements . . . 45
- 3.3 Processes, `if` and `case` . . . 45
- 3.4 Constants and generics . . . 45
- 3.5 Design examples . . . 46

4 Regular sequential circuits 47
- 4.1 D flip-flop and timing . . . 47
- 4.2 HDL templates for FF and register . . . 47
- 4.3 Simple regular circuits . . . 48
- 4.4 Testbench for sequential circuits . . . 49
- 4.5 Case study 1: LED time-multiplexing . . . 49
- 4.6 Case study 2: stopwatch 00.0–99.9 s . . . 50
- 4.7 Case study 3: FIFO buffer . . . 50

5 Finite state machines (FSM) 52
- 5.1 Moore and Mealy . . . 52
- 5.2 State diagram and ASM chart . . . 52
- 5.3 FSM code template . . . 52
- 5.4 Example: rising-edge detector . . . 53
- 5.5 Example: debouncing circuit (FSM version) . . . 54

6 FSM with data path (FSMD) 55
- 6.1 RT operations and the ASMD chart . . . 55
- 6.2 Example that shows the method: FSMD debouncer . . . 55
- 6.3 Design examples . . . 56

Part II I/O modules

UART, PS2 keyboard & mouse, external SRAM, block RAM, VGA graphics & text 58

7 UART 59
- 7.1 Serial transmission . . . 59
- 7.2 Receiving subsystem . . . 59
- 7.3 Interface circuit . . . 60
- 7.4 Transmitter and complete UART . . . 60
- 7.5 Customising and error checking . . . 61

8 PS2 keyboard and mouse 62
- 8.1 PS2 port and device-to-host protocol . . . 62
- 8.2 Keyboard scan codes . . . 62
- 8.3 Mouse protocol . . . 62
- 8.4 Host-to-device transmission . . . 63

10 External SRAM and memory controller 64
- 10.1 The IS61LV25616AL SRAM . . . 64
- 10.2 Controller structure . . . 64
- 10.3 The safe design . . . 64

10.4 More aggressive designs and why they are risky 65
11 Embedded memory: block and distributed RAM 66
11.1 Storage options on the S3 board 66
11.2 Three ways to include a memory module 66
11.3 Inference templates 66
12 VGA controller I: synchronisation and graphics 68
12.1 CRT basics and the S3 VGA port 68
12.2 Synchronisation timing, 640×480 at 25 MHz . . . 68
12.3 Pixel generation schemes 70
12.4 Object-mapped graphics: the pong screen 70
12.5 Bit-mapped graphics: dot trace 71
13 VGA controller II: text and the complete pong game 72
13.1 Characters as tiles and the font ROM 72
13.2 Full-screen text display 72
13.3 The complete pong game 73

Part III PicoBlaze microcontroller

Organisation, instruction set, assembly, I/O interface, interrupts **74**

14 PicoBlaze overview and instruction set 75
14.1 From FSMD to microcontroller 75
14.2 PicoBlaze organisation 75
14.3 Development flow 76
14.4 Programming model and formats 76
14.5 Instruction set (57 instructions) 76
15 PicoBlaze assembly programming 78
15.1 Useful code segments 78
15.2 Subroutines 78
15.3 Program development 79
15.4 Tools and synthesis 79
16 PicoBlaze I/O interface 80
16.1 I/O signals and timing 80
16.2 Square circuit with switch, buttons and 7-segment display 80
16.3 Adding a combinational multiplier and a UART console 81
17 PicoBlaze interrupts 82
17.1 Interrupt handling 82
17.2 External interrupt interface 82
17.3 Software considerations 82
17.4 Design example: LED multiplexing in software . . 83

Back Revision tools

Numbers sheet, compare & contrast, template cheat sheet, glossary, practice questions **84**

18 Numbers and formulas sheet 85
19 Compare and contrast 87
20 VHDL template cheat sheet 89
21 Glossary 90
22 Mock exam (4 hours) 92

00

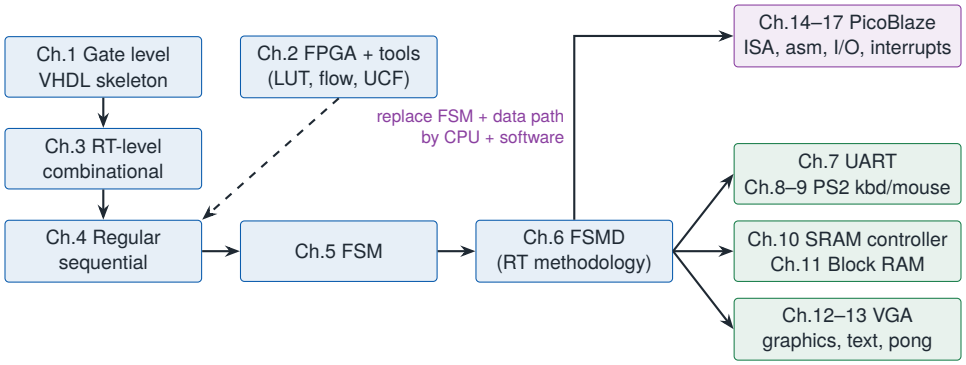
How to use this summary

The book in one page, what a 4-hour exam on this material usually looks like, answer recipes and a study plan

Preface + all chapters

0.1 The book in one picture

Chu's book teaches one method and then applies it again and again: **separate the register from the combinational logic**, describe each part with a small set of templates, and build every larger system from counters, FSMs and FSMDs that all run on the *same* 50 MHz clock. Part II reuses these blocks to build real I/O modules; Part III swaps the custom FSMD for a small programmable processor (PicoBlaze). **Part 0** (not in the book) comes first and explains the hardware being described: gates and 74xx chips, adders and the ALU, flip-flops, registers, RAM/ROM and the SAP-1 computer. Start there if the digital-logic basics are rusty.



0.2 What a 4-hour written exam on this book tends to ask

I do not have your course's past papers, so the list below is built from what the book emphasises (it repeats these ideas in almost every chapter). If you get old exams, map each question to a row of this table and to the matching chapter.

Question type	Typical wording	Where
Write VHDL	"Write the code for a mod-m counter / universal shift register / FIFO / edge detector / debouncer / UART receiver." Start from a template.	§4–§7, Back
Draw a diagram	"Draw the state diagram / ASM(D) chart / block diagram (data path + control path)."	§5, §6
Timing calculation	f_{max} from T_{cq}, T_{comb}, T_{setup}; counter width and divisor from a clock; baud divisor; VGA pixel rate; debounce time.	§4.1, §7.2, §12.2, Back
Explain / compare	Moore vs. Mealy; `when-else` vs. `with-select`; `if` vs. `case`; explicit vs. implicit data path; distributed vs. block RAM; bit- vs. tile- vs. object-mapped; polling vs. interrupt.	Compare tables
Find the bug	Latch inferred, missing sensitivity signal, multiple drivers, gated/derived clock, "delayed store" in ASMD decision box, type mismatch.	§3.5.4, §6.1.3
Trace an algorithm	Binary-to-BCD (double dabble), long division, Fibonacci, UART oversampling steps, FIFO pointers.	§6.3, §4.5.3
FPGA architecture	LUT as $2^n \times 1$ memory, logic cell / slice / CLB, macro cells, design flow steps.	§2
Digital logic basics	Boolean simplification / K-map, full adder from gates, mux/decoder design, flip-flop tables, counter by hand, RAM/ROM structure, SAP-1 T-states.	Part 0
PicoBlaze	Instruction semantics (flags!), I/O decoding with `port_id` and strobes, interrupt sequence, write a short routine.	§14–§17

Answer template that fits almost every question

1. **Define** the thing in one sentence using the book's words ("a tick is a signal asserted for exactly one clock cycle").
2. **Draw** the block / state / ASMD diagram. A diagram earns marks even when the code is incomplete.
3. **Code** it with the two-segment style: a register process, plus combinational next-state and output logic.
4. **Verify or justify**: timing numbers, one short waveform, or a testbench sketch; say what would go wrong without it.

Time plan for 4 hours

- **0–10 min**: read everything, choose questions, start with the ones you can template (counters, FSM code).
- **Budget** about 2 minutes per percentage point; leave 15 min at the end.
- Write code *after* drawing the diagram; the diagram is your plan and your partial credit.
- Always state assumptions (clock 50 MHz, active-high reset, number of bits).

How to study with this booklet

- Read one chapter, close it, and **redraw the diagrams from memory**.
- Re-type every template in the back until you can write it without looking.
- Do every ▷ worked example by hand.
- Use the glossary as flashcards; do the ⋆ practice questions under time.

0.3 Box legend

Key concept

Definitions and mechanisms you must be able to state.

▲ Pitfall

Classic mistakes: examiners love them in "find the error" questions.

▷ Worked example

Numbers worked out step by step.

Xilinx-specific

Things that only hold for Spartan-3 / ISE (the book marks these "Xilinx specific").

⋆ Exam practice

Questions to practise. The answer is in the section referred to.

Part 0
Digital logic foundations

What the VHDL describes: gates, chips, adders and ALU, flip-flops, registers, RAM/ROM, the SAP-1 computer, and SAP-1 built from gates and in VHDL

A Numbers and Boolean algebra

Binary and hexadecimal, two's complement and overflow, Boolean laws, De Morgan, SOP/POS and Karnaugh maps

Foundations (not in Chu's book)

Why this part exists

Chu's book shows how to *describe* hardware in VHDL and assumes you already know *what* the hardware is. Part 0 fills that gap: the gates, chips, adders, flip-flops, memories and a complete simple CPU (SAP-1) that the VHDL describes. Everything here is standard digital-design material (as in Floyd, Mano or Malvino's SAP-1), so exam questions about "how does it work inside" are answered here.

A.1 Number systems

- **Binary** (base 2): $1011_2 = 8+0+2+1 = 11_{10}$. Bit i has weight 2^i; n bits hold $0 \ldots 2^n - 1$.
- **Hexadecimal** (base 16): one hex digit = 4 bits. `1011 1110` = $\texttt{BE}_{16} = 190$.
- **Decimal** $\rightarrow$ **binary**: divide by 2 repeatedly, remainders read upwards; or subtract powers of two.
- **BCD**: each decimal digit in 4 bits (259 = `0010 0101 1001`); codes 1010–1111 unused.
- **Gray code**: consecutive values differ in one bit (00, 01, 11, 10); used in K-maps and position encoders.
- **ASCII**: 7-bit character code (`'A'`=`0x41`, `'0'`=`0x30`).

Signed numbers (n bits)

- **Sign-magnitude**: MSB = sign. Two zeros; awkward arithmetic.
- **One's complement**: invert all bits to negate. Two zeros.
- **Two's complement**: invert all bits and **add 1**. Range $-2^{n-1} \ldots 2^{n-1} - 1$ (8 bit: $-128 \ldots 127$). One zero; the MSB has weight -2^{n-1}. Subtraction is addition: $A - B = A + \overline{B} + 1$.
- **Sign extension**: copy the MSB to the left (`1011` $\rightarrow$ `1111 1011`, both -5).
- **Overflow** (signed): adding two numbers of the same sign gives a result with the other sign; $V = C_n \oplus C_{n-1}$ (carry into MSB XOR carry out of MSB). **Carry-out** is the unsigned overflow.

▷ Two's complement arithmetic, 8 bits

-5: 5 = `0000 0101` $\rightarrow$ invert `1111 1010` $\rightarrow$ +1 `1111 1011`. $7 + (-5)$: `0000 0111` + `1111 1011` = `1 0000 0010`: drop the carry $\Rightarrow$ `0000 0010` = 2 (no overflow: signs differ). $100 + 50$: `0110 0100` + `0011 0010` = `1001 0110` = -106 $\Rightarrow$ **overflow** (two positives gave a negative; $C_7 = 1$, $C_8 = 0$).

A.2 Boolean algebra

Variables are 0 or 1; operators **AND** ($A \cdot B$), **OR** ($A + B$), **NOT** ($\overline{A}$ or A'), **XOR** ($A \oplus B = A\overline{B} + \overline{A}B$).

Law	Form
Identity	$A + 0 = A$, $A \cdot 1 = A$
Null	$A + 1 = 1$, $A \cdot 0 = 0$
Idempotent	$A + A = A$, $A \cdot A = A$
Complement	$A + \overline{A} = 1$, $A \cdot \overline{A} = 0$
Involution	$\overline{\overline{A}} = A$
Commutative	$A + B = B + A$, $AB = BA$
Associative	$(A + B) + C = A + (B + C)$
Distributive	$A(B + C) = AB + AC$, $A + BC = (A + B)(A + C)$
Absorption	$A + AB = A$, $A(A + B) = A$
Simplification	$A + \overline{A}B = A + B$
De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$, $\overline{A + B} = \overline{A} \cdot \overline{B}$

Canonical forms

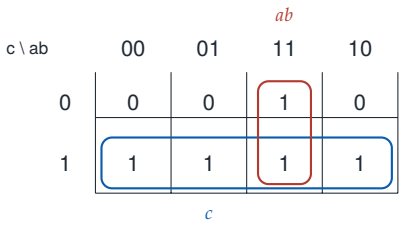
- **Minterm** m_i: AND term that is 1 for exactly one input row. **SOP** (sum of products) = OR of the minterms where $f = 1$: $f = \sum m(\ldots)$. This is what a **LUT** or a **ROM** stores (Part 0.E).
- **Maxterm** M_i: OR term that is 0 for one row. **POS** = AND of maxterms where $f = 0$: $f = \prod M(\ldots)$.
- Any function can be built from AND/OR/NOT, or from **NAND alone** or **NOR alone** (universal gates).

▷ From truth table to gates: 1-bit equality (Chu Listing 1.1)

$eq = 1$ for rows 00 and 11: $eq = \overline{i_0}\,\overline{i_1} + i_0 i_1 = \overline{i_0 \oplus i_1}$ (XNOR).

A.3 Karnaugh maps

A K-map arranges the truth table in **Gray-code order** so neighbouring cells differ in one variable. Group adjacent 1s in rectangles of size 1, 2, 4, 8... (edges wrap around); each group removes the variables that change inside it. Use the largest possible groups, as few as possible. "Don't care" cells (X) may be included if they help.



▷ $f(a,b,c) = \sum m(1,3,5,6,7)$

Group 1: the whole $c = 1$ row (m1, m3, m7, m5) $\Rightarrow c$. Group 2: m6, m7 ($a = 1, b = 1$) $\Rightarrow ab$. So $f = c + ab$ (instead of five 3-input minterms).

⋆ Exam practice

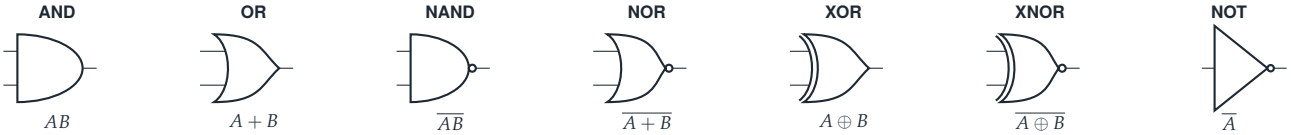
1. Convert -37 to 8-bit two's complement and back. (`1101 1011`.)
2. Prove $A + \overline{A}B = A + B$ with the laws, and simplify $\overline{\overline{A} + B} \cdot C$ with De Morgan. ($A\overline{B}C$.)
3. Simplify $f(a,b,c,d) = \sum m(0,2,8,10,5,7,13,15)$ with a K-map. ($\overline{b}\,\overline{d} + bd$.)
4. When does adding two 8-bit two's-complement numbers overflow? How can a circuit detect it?

B Logic gates, logic families and chips

Gate symbols and truth tables, universal gates, TTL vs. CMOS electrical levels, output types, propagation delay, and the 74xx chips you should recognise

Foundations (not in Chu's book)

B.1 The basic gates



A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT A
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

- **AND**: 1 only if all inputs are 1. **OR**: 1 if any input is 1. **XOR**: 1 if an *odd* number of inputs are 1 (parity, adders, comparators, controlled inverter: $B \oplus 1 = \overline{B}$).
- The bubble means inversion. NAND = AND + NOT.
- **Universal gates**: NOT = NAND with tied inputs; AND = NAND + NOT; OR = $\overline{\overline{A}\cdot\overline{B}}$ (De Morgan) = NAND of inverted inputs. Same for NOR. So a whole computer can be built from NAND chips only.

Two-level logic

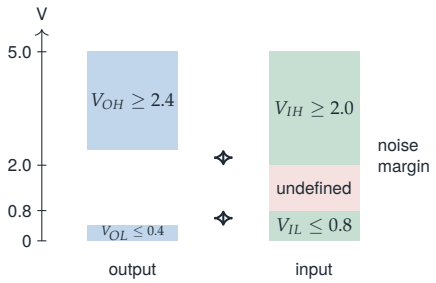
Any SOP expression = **AND-OR** network = **NAND-NAND** network (replace both levels by NAND, by De Morgan). Any POS = OR-AND = NOR-NOR. This is why CMOS and TTL designs are mostly NAND/NOR.

B.2 Logic families: TTL and CMOS

A logic value is really a **voltage range**. The chip guarantees output levels (V_{OH}, V_{OL}) and accepts input levels (V_{IH}, V_{IL}); the gap between them is the **noise margin**.

Parameter	TTL (74, 74LS)	CMOS (74HC, 5 V)
Supply	5 V ±5%	2–6 V
V_{IH} min	2.0 V	3.5 V
V_{IL} max	0.8 V	1.5 V
V_{OH} min	2.4 V (LS 2.7)	4.9 V
V_{OL} max	0.4 V (LS 0.5)	0.1 V
Noise margin	0.4 V	≈1.4 V
Static power	mW per gate	≈0 (only when switching)
Input	draws current (bipolar)	almost no current (MOS gate)
Fan-out	10 (standard TTL)	large (limited by capacitance)

TTL = transistor-transistor logic: bipolar transistors, multi-emitter input, **totem-pole** output. An unconnected TTL input floats **high**. **CMOS** = complementary MOS: a pMOS pull-up network and an nMOS pull-down network, so one path is always off ⇒ power only while switching ($P \approx CV^2f$). **FPGAs are CMOS** (Spartan-3 core at 1.2 V, I/O at 3.3 V etc.). Unused CMOS inputs must never float.



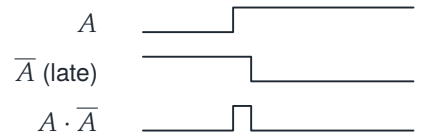
TTL levels and noise margins

Output types

- **Totem-pole / push-pull**: drives both 1 and 0. Two such outputs must **never** be tied together (short circuit = "fighting", Chu §3.3).
- **Open-collector / open-drain**: can only pull low; needs an external **pull-up resistor**; several can share one wire (**wired-AND**). PS2 lines work this way (Chu Ch. 9).
- **Tri-state**: 0, 1 or **high-Z** (disconnected) controlled by an enable; lets many devices share a **bus** (SAP-1 W-bus, SRAM data bus, Chu §3.2.3).

B.3 Timing of real gates

- **Propagation delay** t_{PLH}, t_{PHL}: input change to output change (≈10 ns standard TTL, ≈9 ns 74LS, a few ns 74HC, <1 ns inside an FPGA).
- Delays add along a path; the **longest (critical) path** limits the speed (Chu's T_{comb}).
- Unequal path delays cause **glitches** (hazards): e.g. $A \cdot \overline{A}$ briefly becomes 1 when A rises because $\overline{A}$ is late. Synchronous design hides glitches by sampling only at clock edges.
- **Rise/fall time**, **fan-in** (number of inputs), **fan-out** (loads one output can drive), **power-delay product**.



A static hazard: the "impossible" 1 caused by delay

B.4 74xx chips worth recognising

Chip numbers are the same across families: 74**LS**00 (low-power Schottky TTL), 74**HC**00 (CMOS), 74**HCT**00 (CMOS with TTL input levels). 14-pin gate chips: pin 14 = Vcc, pin 7 = GND.

Chip	Function	Chip	Function
`7400`	quad 2-input NAND	`74138`	3-to-8 decoder (active-low outputs, 3 enables)
`7402`	quad 2-input NOR	`74148`	8-to-3 priority encoder
`7404`	hex inverter	`74151 / 74153 / 74157`	8:1 mux / dual 4:1 mux / quad 2:1 mux
`7408 / 7432`	quad AND / quad OR	`7483 / 74283`	4-bit binary full adder (carry look-ahead)
`7486`	quad XOR	`74181`	4-bit ALU (16 logic + 16 arithmetic functions)
`7474`	dual D flip-flop (edge-triggered, preset/clear)	`7485`	4-bit magnitude comparator
`7476 / 74107`	dual JK flip-flop	`74161 / 74193`	sync 4-bit counter / up-down counter
`74173`	4-bit register with tri-state outputs	`74194`	4-bit universal shift register
`74373 / 74374`	octal latch / octal D-FF register (tri-state)	`74244 / 74245`	octal tri-state buffer / bus transceiver
`74189`	64-bit (16 × 4) RAM (used in SAP-1)	`7447`	BCD to 7-segment decoder

From chips to FPGA

A breadboard computer (SAP-1) is wired from these chips. An FPGA replaces them all: each small function becomes LUT contents, each flip-flop an LC flip-flop, and the wiring becomes programmable routing. Your VHDL is the "wiring diagram".

⋆ Exam practice

1. Build NOT, AND and OR from 2-input NAND gates only. How many 7400 chips does $f = AB + CD$ need? (*NAND-NAND: 3 gates, one chip.*)
2. Compute the high and low noise margins of TTL from the table.
3. Why can two totem-pole outputs not be connected, and what are the two alternatives?
4. Explain why CMOS uses almost no power when not switching.

C Combinational building blocks and the ALU

Multiplexer, demultiplexer, decoder, encoder, comparator; half and full adders and subtractors, ripple and look-ahead carry, adder-subtractor, overflow, and the ALU in parts and as a whole

Foundations (links to Chu Ch. 3)

A **combinational** circuit has no memory: outputs depend only on the present inputs (after the propagation delay). These blocks are exactly the "RT-level components" that Chu's VHDL infers from `when-else`, `with-select`, `+`, `-`, `>` (Chu Ch. 3).

C.1 Multiplexer and demultiplexer

A **multiplexer (mux)** routes one of 2^n data inputs to the output, chosen by n select lines. "A digitally controlled switch."

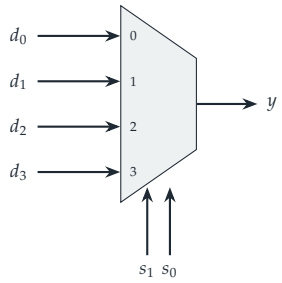
$$\text{2:1 mux:}\quad y = \bar{s}\,d_0 + s\,d_1$$

$$\text{4:1 mux:}\quad y = \overline{s_1}\,\overline{s_0}d_0 + \overline{s_1}s_0d_1 + s_1\overline{s_0}d_2 + s_1s_0d_3$$

- Wider muxes: tree of smaller ones (4:1 = three 2:1).
- A 2^n:1 mux implements **any n-input function**: put the truth-table column on the data inputs and the variables on the selects (this is how a LUT works).
- Uses: data selectors, bus sharing, register input selection in an FSMD (Chu Fig. 6.2), the ALU result selector.

A **demultiplexer** does the opposite: one input sent to one of 2^n outputs. A decoder with an enable is a demux.

4:1 mux



s_1	s_0	y
0	0	d_0
0	1	d_1
1	0	d_2
1	1	d_3

▷ XOR from a 4:1 mux

Selects $s_1s_0 = AB$; data inputs = truth table $0,1,1,0 \Rightarrow y = A \oplus B$.

C.2 Decoder, encoder, comparator

Decoder (n-to-2^n): exactly one output is active, the one whose number equals the binary input. Each output is one **minterm**.

en	a_1	a_0	y_3	y_2	y_1	y_0
0	–	–	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$y_0 = en \cdot \overline{a_1}\,\overline{a_0}$, $y_3 = en \cdot a_1a_0$, etc.

- **Uses**: memory **address decoding** (select the word line / chip), instruction decoding in a CPU, 7-segment and 1-of-8 LED drivers, output-port enables (Chu §16.2).
- Decoder + OR gate = any function (OR the minterm outputs).
- Larger decoders from smaller ones with the enables (two 74138 = 4-to-16).

Encoder (2^n-to-n): the reverse; outputs the binary code of the active input. Plain encoders assume one active input; a **priority encoder** outputs the *highest*-priority active input and a "valid" flag (Chu Table 3.4, 74148). Used for interrupt controllers and keyboards.

r_3	r_2	r_1	r_0	c_1	c_0	valid
1	–	–	–	1	1	1
0	1	–	–	1	0	1
0	0	1	–	0	1	1
0	0	0	1	0	0	1
0	0	0	0	–	–	0

Magnitude comparator: $A = B$ when all bits equal: $\prod_i \overline{a_i \oplus b_i}$ (XNORs + AND). $A > B$: find the most significant differing bit where $a_i = 1, b_i = 0$ (7485). In practice often done with a subtractor: $A - B$ and look at borrow and zero.

C.3 Adders

Half adder (2 inputs, no carry in):

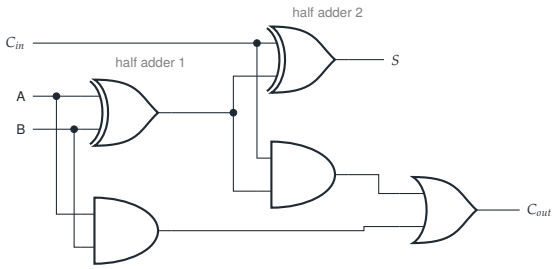
$$S = A \oplus B \qquad C = A \cdot B$$

Full adder (adds A, B and carry-in):

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B) \;=\; AB + AC_{in} + BC_{in}$$

Built from **two half adders and an OR** gate.

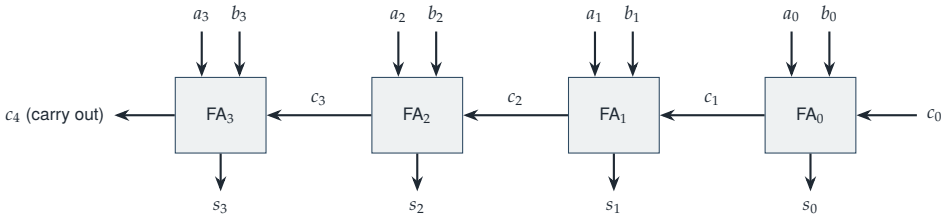


Full adder = two half adders + OR

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Ripple-carry adder

Chain n full adders: C_{out} of bit $i \rightarrow C_{in}$ of bit $i+1$. Simple, but the carry must **ripple** through all stages: delay $\propto n$. That long carry chain is often the critical path (T_{comb}).



4-bit ripple-carry adder (7483 does the same with look-ahead)

Carry look-ahead: define per bit **generate** $G_i = a_i b_i$ (this bit creates a carry) and **propagate** $P_i = a_i \oplus b_i$ (this bit passes a carry on). Then

$$c_{i+1} = G_i + P_i c_i$$

and expanding, e.g. $c_2 = G_1 + P_1 G_0 + P_1 P_0 c_0$, gives every carry in **two gate levels** at the cost of more gates. FPGAs instead have a dedicated fast **carry chain** in each logic cell (Chu §2.2.2). **Multiplication** (binary, unsigned): AND each bit of the multiplier with the multiplicand (partial products), shift, and add: an array of AND gates and adders, or the shift-and-add algorithm over several clocks (PicoBlaze `mult_soft`, Chu §15.3). Spartan-3 has hard 18×18 multipliers.

C.4 Subtractors and the adder-subtractor

Half subtractor $(A - B)$: $D = A \oplus B$, $B_{out} = \overline{A}B$.
Full subtractor $(A - B - B_{in})$:

$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = \overline{A}B + \overline{A}B_{in} + BB_{in}$$

In practice nobody builds separate subtractors: with two's complement,

$$A - B = A + \overline{B} + 1.$$

Adder-subtractor (used in SAP-1)

Put an **XOR on each B input** controlled by SUB: $b_i \oplus SUB$ passes b_i when $SUB = 0$ and inverts it when $SUB = 1$. Connect SUB also to the **carry-in** c_0 (the "+1"). One circuit then computes $A + B$ ($SUB = 0$) or $A - B$ ($SUB = 1$). Signed overflow $V = c_n \oplus c_{n-1}$; for unsigned subtraction the carry-out is 1 when *no* borrow occurred.

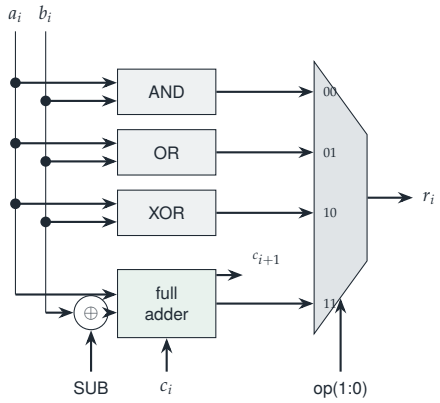
C.5 The ALU: in parts and as a whole

An **arithmetic logic unit** computes one of several functions of two operands, chosen by an **operation code**, and reports **status flags**. It is the heart of every CPU data path (PicoBlaze Ch. 14, SAP-1 Part 0.F).

Parts

- **Logic unit**: bitwise AND, OR, XOR, NOT, one gate per bit, no carries.
- **Arithmetic unit**: adder-subtractor (add, subtract, increment $A+1$ via c_0, decrement, pass A).
- **Shifter**: shift/rotate by one (wiring + mux) or barrel shifter (Chu §3.7.3).
- **Result multiplexer**: selects which unit's output leaves the ALU.
- **Flags**: **Z** (result = 0, a NOR of all bits), **C** (carry out), **N**/**S** (sign = MSB), **V** (signed overflow). Conditional jumps test these (PicoBlaze `jump z`).

Bit-slice view: an n-bit ALU is n identical 1-bit slices (each with its logic gates, a full adder and a small mux) chained through the carry. The 74181 is a 4-bit slice: 4-bit operation select `S3..S0` + mode `M` (logic/arithmetic) = 32 functions, with carry look-ahead outputs G, P to cascade.



One 1-bit ALU slice: logic gates + (adder with XOR for subtract) + result mux

op	SUB	Result
00	–	A AND B
01	–	A OR B
10	–	A XOR B
11	0	A + B
11	1	A − B

The same ALU in Chu's style (numeric_std)

```
process(op, sub, a, b)
begin
  case op is
    when "00"   => r <= a and b;
    when "01"   => r <= a or b;
    when "10"   => r <= a xor b;
    when others =>
      if sub='0' then r <= std_logic_vector(unsigned(a)+
     unsigned(b));
      else            r <= std_logic_vector(unsigned(a)-
     unsigned(b));
      end if;
  end case;
end process;
zero <= '1' when r = (r'range => '0') else '0';
```

Where you meet these blocks later

Mux → `with-select/case` and every register's next-state selection. Priority network → `when-else/if`. Decoder → register-file write enable, port decoding. Adder/comparator → `+`, `-`, `>`. ALU → PicoBlaze, SAP-1. Synthesis maps them into LUTs and carry chains.

⋆ Exam practice

1. Derive S and C_{out} of a full adder from its truth table and draw it from two half adders.
2. Draw a 4-bit adder-subtractor and compute $0101 - 0111$ with it; is there signed overflow? ($0101 + 1000 + 1 = 1110 = -2$, *no overflow.*)
3. Implement $f(a,b,c) = \sum m(1,2,6,7)$ with (a) an 8:1 mux, (b) a 3-to-8 decoder and an OR gate.
4. Design a 4-to-2 priority encoder with a valid output (equations).
5. What limits the speed of a ripple-carry adder and how does carry look-ahead help? Give c_2 in terms of G, P.
6. List the parts of an ALU and the four common status flags; how is the zero flag computed?

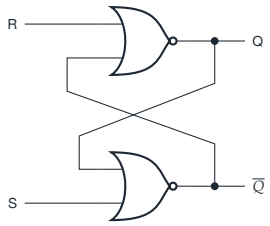
D Latches, flip-flops, registers and counters

How a circuit remembers: SR and D latches, edge-triggered D, JK and T flip-flops, characteristic and excitation tables, timing, registers, shift registers and counters built by hand

Foundations (links to Chu Ch. 4)

D.1 From feedback to memory: the SR latch

A **sequential** circuit has **memory**: its output depends on inputs *and* on what happened before. The simplest memory is two cross-coupled NOR gates: each output feeds the other's input, so the circuit has two stable states.



NOR SR latch (active-high S and R)

S	R	Q^+	action
0	0	Q	hold (memory)
0	1	0	reset
1	0	1	set
1	1	–	**forbidden**

$S = R = 1$ forces $Q = \overline{Q} = 0$; releasing both at once gives an unpredictable state.

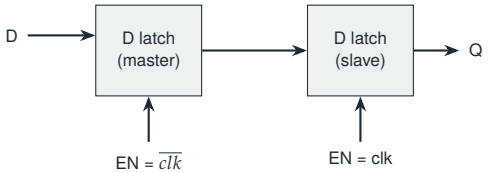
- **NAND version** ($\overline{S}\,\overline{R}$ latch): inputs active low; $\overline{S} = \overline{R} = 0$ forbidden. Classic use: **switch debouncer** with an SPDT switch (the hardware alternative to Chu's debounce FSM).
- **Gated SR latch**: AND S and R with an enable `EN` (or clock); the latch only reacts while `EN`=1.
- **Gated D latch**: make $S = D$, $R = \overline{D}$, so the forbidden input can never happen. While $EN = 1$ the latch is **transparent** (Q follows D); when $EN = 0$ it **holds** the last value. (74373.)

▲ Latch vs. flip-flop

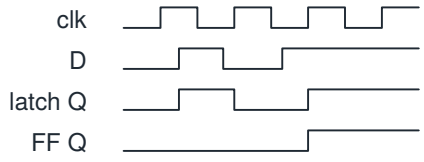
A **latch** is **level-sensitive** (transparent for the whole time EN is active). A **flip-flop** is **edge-triggered** (samples only at the clock edge). Synchronous design uses flip-flops; a latch in an FPGA design is usually the unintended-memory bug of Chu §3.5.4.

D.2 Edge-triggered D flip-flop

Master-slave construction: two D latches in series with opposite enables. While clk=0 the **master** is transparent and follows D; the slave holds. When clk rises the master closes (freezes D) and the **slave** opens and passes it to Q. Net effect: Q takes the value D had **at the rising edge**, $Q^+ = D$.



Master enabled by $\overline{clk}$, slave by clk ⇒ positive-edge D FF



Latch Q follows D while clk=1; the FF only samples D at rising edges, so it never sees the short first pulse

Asynchronous preset/clear force Q to 1/0 immediately, ignoring the clock (7474; Chu's async reset).

D.3 JK and T flip-flops

JK flip-flop: like SR but $J = K = 1$ is useful: it **toggles**.

$$Q^+ = J\overline{Q} + \overline{K}Q$$

In a level-triggered JK with $J = K = 1$ the output would toggle again and again while the clock is high (**race-around**); edge triggering / master-slave solves it.

T flip-flop (toggle): $Q^+ = T \oplus Q$. Make it from a JK with $J = K = T$, or from a D FF with $D = T \oplus Q$. With $T = 1$ it divides the clock frequency by 2: the basis of counters.

J	K	Q^+	action
0	0	Q	hold
0	1	0	reset
1	0	1	set
1	1	$\overline{Q}$	toggle

Type	Characteristic equation	$Q \to Q^+$: 0→0	0→1	1→0	1→1
SR	$Q^+ = S + \overline{R}Q,\ \ SR = 0$	S=0, R=X	S=1, R=0	S=0, R=1	S=X, R=0
D	$Q^+ = D$	D=0	D=1	D=0	D=1
JK	$Q^+ = J\overline{Q} + \overline{K}Q$	J=0, K=X	J=1, K=X	J=X, K=1	J=X, K=0
T	$Q^+ = T \oplus Q$	T=0	T=1	T=1	T=0

Characteristic equations and **excitation tables** (which inputs cause a wanted transition; X = don't care). Excitation tables are used to design counters/FSMs by hand.

D.4 Timing and metastability

- T_{setup}: D stable before the edge; T_{hold}: after the edge; T_{cq}: clock-to-output delay. Maximum clock: $T_{clk} \geq T_{cq} + T_{comb} + T_{setup}$ (Chu §4.1).
- Violating setup/hold ⇒ **metastability**: Q hangs between 0 and 1 for an unpredictable time.
- Inputs that are asynchronous to the clock (buttons, other clock domains) go through a **synchroniser**: two FFs in series, giving the first one a full clock period to settle.

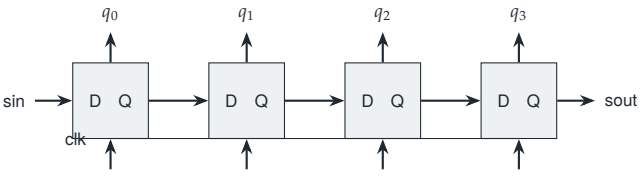
Why one clock (synchronous design)

If every FF uses the same clock edge, all state changes happen at once and the only timing check is the single loop register → logic → register. Ripple clocks, gated clocks and latches break this and make timing unpredictable. This is the principle behind the whole of Chu's book.

D.5 Registers and shift registers

A **register** is n D flip-flops sharing one clock: it stores an n-bit word.

- **Load enable**: a 2:1 mux in front of each D: $D_i = LD \cdot d_i + \overline{LD} \cdot Q_i$ (load or keep). Never gate the clock.
- **Tri-state output**: `OE` connects Q to a shared **bus** only when selected (74173; SAP-1 registers).
- **Register file**: several registers + write decoder + read multiplexers (Chu §4.2.3).



4-bit shift register (SIPO / SISO)

Shift registers move bits one position per clock: **SISO** (serial in, serial out: delay line), **SIPO** (serial-to-parallel: UART receiver), **PISO** (parallel-to-serial: UART transmitter), **PIPO**. A **universal** shift register (74194) can hold, shift left, shift right or load, chosen by a 4:1 mux per bit (Chu §4.3.1). Shifting left multiplies by 2, right divides by 2.

D.6 Counters

Asynchronous (ripple) counter: chain of T flip-flops ($T = 1$); each FF is clocked by the previous FF's output. Very few gates, but each stage adds T_{cq}, the outputs change at different times (glitchy intermediate values), and it violates synchronous design (Chu §4.5.2 warning).

Synchronous counter: all FFs share the clock. Bit i toggles when all lower bits are 1:

$$T_0 = 1,\ T_1 = Q_0,\ T_2 = Q_1Q_0,\ T_3 = Q_2Q_1Q_0$$

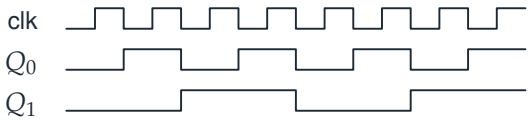
With D FFs this is simply "register + incrementer" (Chu's `r_next <= r_reg + 1`).

- **Mod-m**: clear to 0 when the count reaches $m - 1$ (synchronously!), e.g. decade counter 0–9.
- **Up/down, loadable** counters: add muxes (74193, Chu Table 4.1).
- **Ring counter**: shift register with one circulating 1 (`000001` → `000010` → …); n FFs give n states, already decoded. SAP-1 uses a 6-state ring counter for its T-states.
- **Johnson counter**: shift register with inverted feedback; n FFs give $2n$ states.

▷ Designing a counter by hand: 2-bit up counter with D FFs

Q_1	Q_0	Q_1^+	Q_0^+
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

With D FFs, $D = Q^+$: $D_0 = \overline{Q_0}$, $D_1 = Q_1 \oplus Q_0$.
With JK FFs use the excitation table: $J_0 = K_0 = 1$, $J_1 = K_1 = Q_0$.
Same method for any FSM: state table → state encoding → excitation → K-maps → gates. VHDL synthesis does these steps for you (Chu Ch. 5).



Each bit divides the frequency by 2

⋆ Exam practice

1. Draw a NOR SR latch and explain why $S = R = 1$ is not allowed. How does the D latch avoid it?
2. Explain the difference between a transparent latch and an edge-triggered flip-flop with a timing diagram.
3. Show how a D flip-flop is built from two latches (master-slave).
4. Give the JK characteristic table and equation. Build a T flip-flop from a JK and from a D flip-flop.
5. Design a synchronous mod-3 counter (0,1,2) with D flip-flops. ($D_1 = Q_0$, $D_0 = \overline{Q_1}\,\overline{Q_0}$.)
6. Compare ripple and synchronous counters. Why does Chu forbid using a counter output as a clock?

E

How memory works: RAM and ROM

Memory organisation, address decoding, a RAM built from latches, SRAM and DRAM cells, ROM as a truth table, ROM/PROM/EPROM/EEPROM/Flash, programmable logic and memory expansion

Foundations (links to Chu Ch. 10–11)

E.1 Organisation and signals

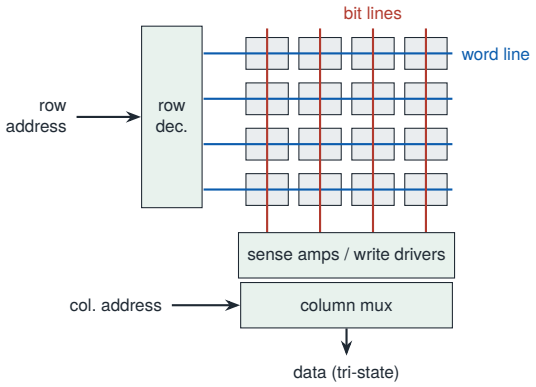
A memory stores 2^n **words** of m bits: "$2^n \times m$". It needs n **address** lines and m **data** lines. Capacity $= 2^n \cdot m$ bits.

▷ Sizes

1K × 8: 10 address lines, 8 data lines, 8192 bits. 256K × 16 (S3 board SRAM): 18 address lines, 16 data lines, 4 Mbit. 16 × 8 (SAP-1): 4 address lines.

Typical control signals (active low): **CS/CE** chip select (enable the chip), **WE** write enable, **OE** output enable (drive the data bus). The data pins are usually a **bidirectional tri-state bus**.

Internally the bits form a 2-D **array**. The **row decoder** turns the address into one active **word line**; the selected cells connect to the vertical **bit lines**; **sense amplifiers** read them and a **column multiplexer** picks the wanted bits. A square array keeps both the decoder and the lines short.

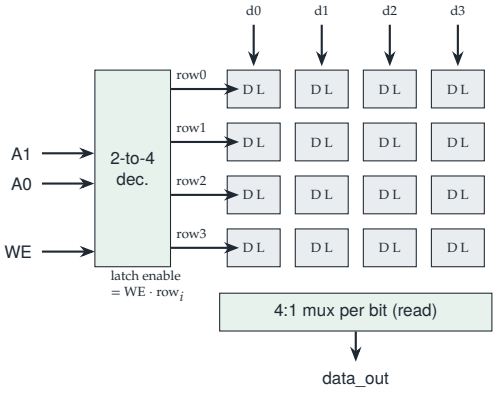


E.2 A small RAM built from latches (how it really works)

A 4 × 4 RAM from what you already know:

- 16 **D latches** (or FFs) in 4 rows of 4 bits.
- A **2-to-4 decoder** on the address selects one row.
- **Write**: $WE \cdot \text{row}_i$ enables the latches of row i; they load `data_in`.
- **Read**: a **4:1 mux** per bit (or tri-state buffers enabled by the row signal and OE) puts row i on `data_out`.

That is exactly Chu's **register file** (§4.2.3) and what synthesis builds from a VHDL array. Real SRAM chips replace each latch by a 6-transistor cell and the mux by bit lines and sense amplifiers, because latches are far too big for millions of bits.



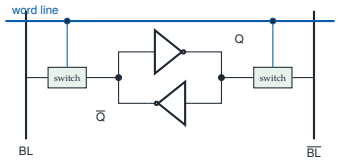
E.3 SRAM and DRAM cells

SRAM cell (6T): two cross-coupled **inverters** (a latch, like the SR latch) store the bit; two **access transistors** connect it to bit line BL and $\overline{BL}$ when the word line is high. **Static**: keeps data as long as power is on, no refresh; fast (ns); 6 transistors per bit, so less dense. Used for caches, FPGA block RAM, the S3 board's IS61LV25616 chips.

Asynchronous SRAM has no clock: address in, data

out after t_{AA}; writes are latched on the rising edge of WE (Chu Ch. 10). **Synchronous** SRAM registers address/data on a clock (FPGA block RAM: read data appears one clock later, Chu §11.4). **DRAM cell (1T1C)**: one transistor + one **capacitor**; the bit is a charge. Very dense and cheap (main memory), but:

- the charge leaks ⇒ every row must be **refreshed** (typically every 64 ms);
- reading is **destructive** (the sense amplifier rewrites the value);
- slower, multiplexed row/column addresses (RAS/-CAS); SDRAM/DDR add a clocked interface.



6T SRAM cell: cross-coupled inverters (4 transistors) + 2 access transistors

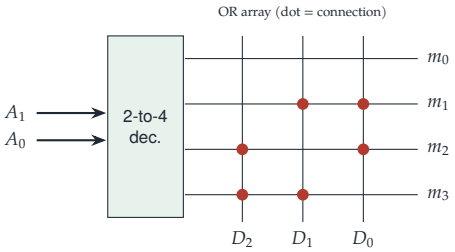
E.4 ROM: memory that is really a truth table

A **ROM** ($2^n \times m$) is **combinational**: the address is the input, the stored word is the output. Internally it is an **n-to-2^n decoder** (all minterms) followed by an **OR array** whose programmed connections choose which minterms feed each output. So a ROM stores a **truth table** and can implement any m functions of n variables (sum of minterms, Part 0.A).

▷ A 4×3 ROM storing a small function

Address $A_1A_0 \rightarrow$ data $D_2D_1D_0$: 00→000, 01→011, 10→101, 11→110.
$D_0 = m_1 + m_2 = A_1 \oplus A_0$, $D_1 = m_1 + m_3 = A_0$, $D_2 = m_2 + m_3 = A_1$.

Uses: look-up tables (sine, 7-segment patterns: Chu Listing 11.5), character **font ROM** (Chu Ch. 13), **program memory** (PicoBlaze instruction ROM), **microcode** / control ROM of a CPU (SAP-2/3), boot code (BIOS). An FPGA **LUT** is a tiny RAM used as a ROM.



Type	Programming / erasing
Mask ROM	data fixed at manufacture; cheapest in volume
PROM	one-time programmable (blow fuses)
EPROM	floating-gate transistor; erase with UV light through a window
EEPROM	electrically erasable, byte by byte; slow writes
Flash	EEPROM erased in blocks; dense; USB/SSD, FPGA configuration PROM (S3 board XCF02S)

Volatile or not; RAM or ROM

Volatile (lost without power): SRAM, DRAM, FPGA configuration. **Non-volatile**: ROM family and Flash. This is why an FPGA must be **reloaded at every power-up** from a Flash PROM or over JTAG (Chu §2.6.4). "RAM" means random *access* (any address, same time), not "writable"; ROMs are random access too.

E.5 Programmable logic: from ROM to FPGA

- **ROM**: fixed AND array (full decoder), programmable OR array.
- **PLA**: programmable AND *and* OR arrays (only the needed product terms).
- **PAL/GAL**: programmable AND, fixed OR, plus output flip-flops.
- **CPLD**: many PAL-like blocks + interconnect; non-volatile.
- **FPGA**: thousands of small LUTs (tiny RAMs) + flip-flops + routing + RAM/multiplier/clock blocks; SRAM-configured (Chu Ch. 2).

Memory expansion

Wider words: chips side by side share address and control; each supplies part of the data bits (two $1K \times 4$ $\rightarrow 1K \times 8$). **More words**: the lower address bits go to all chips; the upper bits go to a **decoder** that drives the **chip selects**; data lines are shared (tri-state).

▷ 4K × 8 from 1K × 4 chips

Width: 2 chips per row. Depth: 4 rows. Total **8 chips**. Address: 12 bits: A9..A0 to every chip, A11..A10 into a 2-to-4 decoder driving the four row chip-selects.

⋆ Exam practice

1. How many address and data lines does a $64K \times 16$ memory have? What is its capacity in bytes? (*16, 16, 128 KB.*)
2. Draw a 4×2 RAM from D latches, a decoder and multiplexers; explain a write and a read.
3. Compare SRAM and DRAM cells: structure, speed, density, refresh.
4. Show that a ROM is decoder + OR array, and program a 8×2 ROM as a full adder (S, C_{out} from A, B, C_{in}).
5. Design a $2K \times 8$ memory from $1K \times 4$ chips (number of chips, decoding).
6. Why must an SRAM-based FPGA be configured after every power-up?

F

A simple CPU: SAP-1

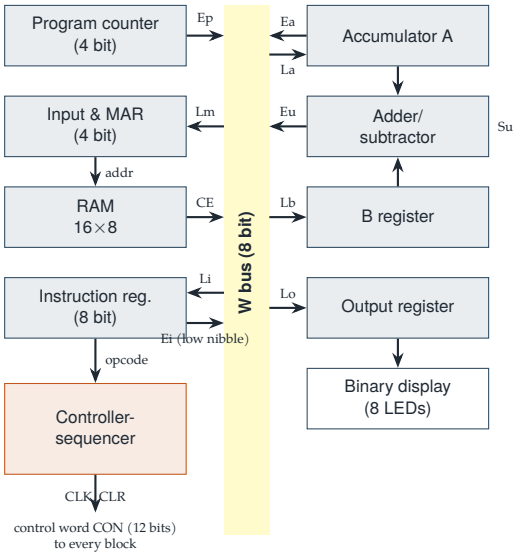
Malvino's Simple-As-Possible computer: architecture on one 8-bit bus, instruction set, fetch and execute T-states, control words, the controller-sequencer, a traced program, and how it maps to an FSMD and to PicoBlaze

Foundations (links to Chu Ch. 6 and 14)

F.1 What SAP-1 is

SAP-1 (Simple-As-Possible, from Malvino & Brown, *Digital Computer Electronics*) is the smallest useful stored-program computer, built from the blocks of the previous chapters (74xx registers, a 74189 RAM, an adder-subtractor, a ring counter and a decoder). It is also the classic breadboard computer. Every idea in it reappears in PicoBlaze and in any real CPU. Chapter G builds it again from individual gates, next to complete, tested VHDL.

- **von Neumann**: program and data in the same memory.
- **One 8-bit bus** (the **W bus**); every register that can put data on it has a **tri-state** output; only one at a time drives it.
- **16 × 8 RAM** ⇒ 4-bit addresses; 8-bit instructions = 4-bit **opcode** + 4-bit **address** operand.
- 5 instructions, no jumps, no flags: a fixed straight-line program.



SAP-1 architecture (after Malvino). Left: program side; right: data side.

F.2 The blocks and their control signals

Block	Function	Control
Program counter (PC)	4-bit counter; address of the next instruction; cleared to 0 by CLR	`Cp (count), Ep (put on bus)`
Input & MAR	memory address register: latches the address from the bus (switches allow manual programming)	`Lm' (load, active low)`
RAM 16×8	program + data; puts the addressed word on the bus	`CE' (enable output)`
Instruction register (IR)	holds the instruction: upper nibble (opcode) to the controller, lower nibble (address) back to the bus	`Li' (load), Ei' (enable lower nibble)`
Controller-sequencer	ring counter (T1..T6) + instruction decoder + control matrix; outputs the control word	`CLK, CLR`
Accumulator A	working register; always feeds the adder-subtractor	`La' (load), Ea (output to bus)`
Adder-subtractor	$A + B$ or $A - B$ (two's complement, Part 0.C)	`Su (subtract), Eu (output)`
B register	second operand	`Lb' (load)`
Output register + display	shows the result on LEDs	`Lo' (load)`

Control word (12 bits, primes = active low): $\text{CON} = C_p\, E_p\, \overline{L_m}\, \overline{CE}\;\; \overline{L_i}\, \overline{E_i}\, \overline{L_a}\, E_a\;\; S_u\, E_u\, \overline{L_b}\, \overline{L_o}$

F.3 Instruction set

Mnemonic	Opcode	Operation
`LDA addr`	`0000`	A ← RAM[addr]
`ADD addr`	`0001`	B ← RAM[addr]; A ← A + B
`SUB addr`	`0010`	B ← RAM[addr]; A ← A − B
`OUT`	`1110`	OUT ← A
`HLT`	`1111`	stop the clock

LDA/ADD/SUB are **memory-reference** instructions (lower nibble = address); OUT and HLT use no operand. Since there are no jumps, the program runs from address 0 until HLT.

Fetch-decode-execute

Every instruction takes one **machine cycle** of **6 T-states** (clock cycles) from a 6-bit **ring counter**: **T1–T3 fetch** (same for all instructions), **T4–T6 execute** (depend on the opcode). The ring counter changes on the falling clock edge, registers load on the rising edge, so control signals are stable when data is loaded.

F.4 T-states and control words

Instruction	T-state	Active signals / RT operation	CON (hex)
fetch (all)	T1 address	Ep, Lm': MAR ← PC	`5E3`
	T2 increment	Cp: PC ← PC + 1	`BE3`
	T3 memory	CE', Li': IR ← RAM[MAR]	`263`
LDA	T4	Ei', Lm': MAR ← IR(3:0)	`1A3`
	T5	CE', La': A ← RAM[MAR]	`2C3`
	T6	no operation	`3E3`
ADD	T4	Ei', Lm': MAR ← IR(3:0)	`1A3`
	T5	CE', Lb': B ← RAM[MAR]	`2E1`
	T6	Eu, La': A ← A + B	`3C7`
SUB	T4, T5	as ADD	`1A3, 2E1`
	T6	Su, Eu, La': A ← A − B	`3CF`
OUT	T4	Ea, Lo': OUT ← A (T5, T6: nop)	`3F2`
HLT	T4	stops the clock (decoder output HLT gates CLK off)	`--`

▷ Decoding a control word: T1 = 5E3

`5E3` = `0101 1110 0011` = Cp=0, **Ep=1**, **Lm'=0**, CE'=1 | Li'=1, Ei'=1, La'=1, Ea=0 | Su=0, Eu=0, Lb'=1, Lo'=1. Only the PC drives the bus and only the MAR loads: **MAR** ← **PC**. Inactive = 0 for active-high signals, 1 for active-low ones; so the "idle" word is `3E3`.

F.5 The controller-sequencer (hardwired control)

Three parts:

1. **Ring counter**: T1..T6, one active at a time (Part 0.D).
2. **Instruction decoder**: decodes the IR opcode into LDA, ADD, SUB, OUT, HLT lines (a 4-to-16 decoder, Part 0.C).
3. **Control matrix**: AND-OR logic combining T-states and decoded instructions into each control bit:

$$C_p = T_2 \qquad E_p = T_1 \qquad \overline{L_i} = \overline{T_3}$$
$$\overline{L_m} = \overline{T_1 + T_4(\text{LDA} + \text{ADD} + \text{SUB})}$$
$$\overline{CE} = \overline{T_3 + T_5(\text{LDA} + \text{ADD} + \text{SUB})}$$
$$\overline{E_i} = \overline{T_4(\text{LDA} + \text{ADD} + \text{SUB})}$$
$$\overline{L_a} = \overline{T_5\,\text{LDA} + T_6(\text{ADD} + \text{SUB})}$$
$$\overline{L_b} = \overline{T_5(\text{ADD} + \text{SUB})}$$
$$E_u = T_6(\text{ADD} + \text{SUB}) \qquad S_u = T_6\,\text{SUB}$$
$$E_a = T_4\,\text{OUT} \qquad \overline{L_o} = \overline{T_4\,\text{OUT}}$$

Hardwired vs. microprogrammed control

Hardwired (above): control matrix of gates; fast, but hard to change. **Microprogrammed** (SAP-2/3, most CISC CPUs): store the control words in a **control ROM**; address = (opcode, T-state); the ROM output *is* CON. Adding an instruction means adding ROM words. (A ROM is just a truth table, Part 0.E.)

SAP-1 is an FSMD

Controller = **FSM** (states T1..T6 + opcode); registers, bus and adder = **data path**; each T-state is an **RT operation** such as MAR ← PC. This is exactly Chu's Ch. 6 structure, and PicoBlaze (Ch. 14) is the same idea with more registers, flags, jumps and 2 clocks per instruction instead of 6.

F.6 Tracing a program

Addr	Contents	Binary
0H	LDA 9H	0000 1001
1H	ADD AH	0001 1010
2H	ADD BH	0001 1011
3H	SUB CH	0010 1100
4H	OUT	1110 xxxx
5H	HLT	1111 xxxx
6H--8H	(unused)	
9H	10H (16)	0001 0000
AH	14H (20)	0001 0100
BH	18H (24)	0001 1000
CH	20H (32)	0010 0000

▷ Result

A = 10H → +14H = 24H → +18H = 3CH → −20H = **1CH** (28): 16 + 20 + 24 − 32 = 28. OUT shows `0001 1100` on the LEDs, then HLT.

Detail of ADD AH (instruction at address 1): T1 MAR←1; T2 PC←2; T3 IR←`0001 1010`; T4 MAR←AH; T5 B←14H; T6 A←10H+14H=24H.

Timing: 5 instructions × 6 T-states, plus T1–T4 of HLT = 34 clock cycles.

F.7 SAP-1 in VHDL (Chu's FSMD style)

Core of a SAP-1 in one FSMD (implicit data path)

```
type t_state is (t1, t2, t3, t4, t5, t6, halt);
type ram_t is array (0 to 15) of
     std_logic_vector(7 downto 0);
signal ram: ram_t := (0=>x"09", 1=>x"1A", 2=>x"1B",
       3=>x"2C", 4=>x"E0", 5=>x"F0", 9=>x"10",
       10=>x"14", 11=>x"18", 12=>x"20", others=>x"00");
signal pc_reg, mar_reg: unsigned(3 downto 0);
signal ir_reg, a_reg, b_reg, out_reg:
       std_logic_vector(7 downto 0);
...
case state_reg is
  when t1 => mar_next <= pc_reg;          state_next <=
     t2;
  when t2 => pc_next <= pc_reg + 1;       state_next <=
     t3;
  when t3 => ir_next <= ram(to_integer(mar_reg));
             state_next <= t4;
  when t4 =>
    state_next <= t5;
    case ir_reg(7 downto 4) is
      when "0000"|"0001"|"0010" =>          -- LDA/ADD/
     SUB
        mar_next <= unsigned(ir_reg(3 downto 0));
      when "1110" => out_next <= a_reg;     -- OUT
      when "1111" => state_next <= halt;    -- HLT
      when others => null;
    end case;
  when t5 =>
    state_next <= t6;
    if ir_reg(7 downto 4)="0000" then
      a_next <= ram(to_integer(mar_reg));   -- LDA
    elsif ir_reg(7 downto 4)="0001" or
          ir_reg(7 downto 4)="0010" then
      b_next <= ram(to_integer(mar_reg));   -- ADD/SUB
    end if;
  when t6 =>
    state_next <= t1;
    if ir_reg(7 downto 4)="0001" then
      a_next <= std_logic_vector(unsigned(a_reg)+
     unsigned(b_reg));
    elsif ir_reg(7 downto 4)="0010" then
      a_next <= std_logic_vector(unsigned(a_reg)-
     unsigned(b_reg));
    end if;
  when halt => null;
end case;
```

Notes

- The shared W bus becomes muxes: in an FPGA there are no internal tri-states (Chu §3.2.3), so "who drives the bus" is simply which value goes into each `_next`.
- The delayed store (Chu §6.1.3) is why MAR is loaded in T4 but read only in T5.
- Easy extensions (typical exam/lab tasks): JMP, JZ with a zero flag, an input port, a 256-byte memory (SAP-2 style), or 2 T-states per instruction by merging steps.

	SAP-1	PicoBlaze	RISC-V RV32I core
Data width	8	8	32
Registers	A, B	16	32 (x0 = 0)
Memory	16 B shared	1K instr. ROM + 64 B RAM	separate/unified, 32-bit addr.
Instructions	5	57	≈40 base
Clocks/instr.	6	2	1–5
Control	hardwired ring counter	hardwired	hardwired decoder
Jumps/flags	none	Z, C, call/return	branch compare, no flags

⋆ Exam practice

1. Draw the SAP-1 block diagram and explain the role of the W bus and tri-state outputs.
2. List the fetch T-states with their control signals and RT operations. Why is fetch identical for all instructions?
3. Give the T-states of SUB and decode the control word `3CF`.
4. Trace a program computing $5 + 7 - 3$ (write memory contents in hex and the value of A after each instruction).
5. Write the control-matrix equations for $\overline{L_m}$ and E_u.
6. Hardwired vs. microprogrammed control: how would you add a JMP instruction to each?
7. Explain how SAP-1 is an FSMD (control path vs. data path) and how PicoBlaze improves on it.

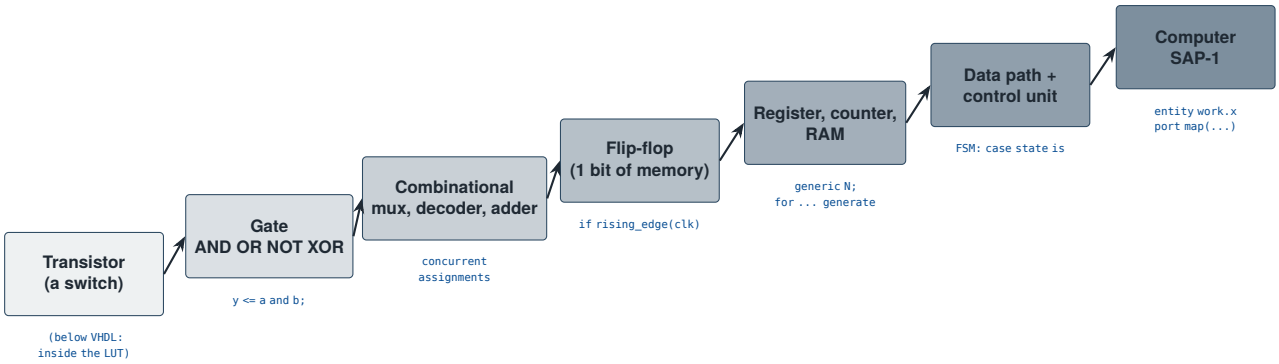
G From logic gates to a computer

SAP-1 seen three ways: as logic gates, as a block diagram, and as VHDL. Every block built from gates next to its VHDL twin, the whole control unit in gates, one bit of the data path, the top level, one instruction traced through all three views, and the same machine as a single FSMD

Foundations (links to Chu Ch. 3, 4, 6)

G.1 The ladder: from a gate to a computer

Chapter F described SAP-1 as blocks. This chapter goes all the way down and back up: every block is drawn **in gates**, shown as a **block**, and written in **VHDL**. Nothing new is needed beyond Chapters A–E. A computer is the same few ideas stacked on top of each other.



Each level is built from the one before it. Blue: the VHDL construct that describes that level.

Level	Built from	The one new idea	Where
Gate	transistors	a Boolean function of voltages: 1 = high, 0 = low	Ch. B
Combinational block	gates	**the output depends only on the inputs now**: choose (mux), select (decoder), compute (adder)	Ch. C, §G.2, §G.5
Flip-flop	gates + **feedback**	**memory**: the output depends on the past and changes only at the clock edge	Ch. D
Register, counter, RAM	flip-flops, muxes, gates	N identical bit cells side by side; a counter feeds its own output back through adding logic	§G.3–§G.6
Data path	registers + ALU + **bus**	all blocks share one set of wires; **control signals** decide who talks and who listens	§G.8
Control unit	ring counter + decoder + gates	a **state machine** that outputs the control signals, one step per clock	§G.7
Computer	data path + control + memory	**stored program**: the instruction is data that the control unit reads and obeys	§G.9

Three views of one machine

- **Gates**: what physically exists. Exact, but thousands of symbols.
- **Blocks**: each block hides its gates behind a name and its ports. This is how you *think* about a computer.
- **VHDL**: a text version of either picture. A *structural* description wires blocks together (§G.9); a *behavioural* one only says what happens each clock (§G.11). The synthesis tool turns both into gates; in an FPGA, into LUTs and flip-flops (Chu §2.2).

The VHDL in this chapter is real and tested

Every listing comes from a complete file in the `vhdl/` folder that comes with this summary (one entity per file). Both versions of the computer, the gate-style structural one and the one-process FSMD, were simulated side by side with GHDL on the program from Ch. F: both show `OUT = 1CH` (28) and halt. Both also synthesise. To run it: `ghdl -a --std=08 *.vhd`, then `ghdl -r --std=08 tb_sap1`.

See it built: Ben Eater's breadboard computer

Ben Eater is an engineer and teacher who publishes free electronics videos on YouTube. In his best-known series he builds an 8-bit computer based on SAP-1 on breadboards, chip by chip: the clock, the registers, the ALU, the RAM, the program counter and the control logic, with LEDs on the wires so you can watch the bits move. It is the physical version of this chapter.

Channel: `youtube.com/@BenEater`

Differences from this summary: his control unit is *microcoded* (control words stored in EEPROMs, see Ch. F) instead of hardwired gates, and he adds instructions such as LDI, STA, JMP, JC and JZ. Some signal names differ. The ideas are the same.

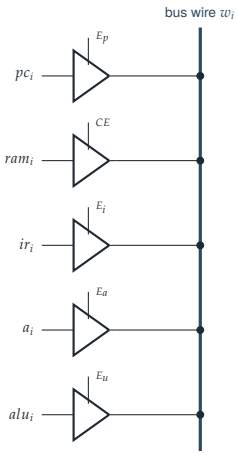
G.2 The W bus: tri-state wires or an AND-OR multiplexer

On the breadboard, every source has a **tri-state** output (Ch. B). When its enable is 0 the output is **Z** (disconnected), so all sources can share the same eight bus wires. The control unit must enable **exactly one** source per T-state, otherwise two outputs fight (**bus contention**).

An FPGA has **no internal tri-states** (Chu §3.2.3), so the bus becomes a **multiplexer**. The simplest form is **AND-OR**: AND each source with its enable, then OR everything. A disabled source gives 0, and $x + 0 = x$, so the one enabled source passes through:

$$w_i = E_p\, pc_i + CE\, ram_i + E_i\, ir_i + E_a\, a_i + E_u\, alu_i$$

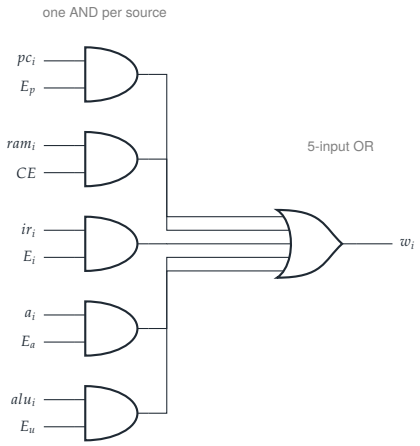
Here CE and E_i mean "active", i.e. $\overline{CE} = 0$ and $\overline{E_i} = 0$.



Breadboard version: tri-state buffers on one shared wire. The enable enters each buffer from the top.

▲ Tri-state VHDL inside an FPGA

`w <= a when ea='1' else (others=>'Z');` is legal VHDL and simulates fine, but an FPGA has tri-state buffers only on its I/O pins. Inside the chip the tool must rewrite it as a mux, or it gives an error. Write the mux yourself.



Bus bit i as AND-OR gates (FPGA version). The control unit makes sure at most one enable is 1.

sap1.vhd: the W bus as AND-OR, 8 bits at once

```
-- the W bus: AND-OR mux of all bus sources
w <= (x8(ep)       and ("0000" & pc))
  or (x8(not ce_n) and ram_q)
  or (x8(not ei_n) and ("0000" & ir(3 downto 0)))
  or (x8(ea)       and a)
  or (x8(eu)       and alu);
```

`x8(e)` copies one enable bit to all 8 positions, so `x8(ep) and pc` ANDs every bit of the source with the enable. The 4-bit sources (PC, IR operand) are padded with `"0000"`.

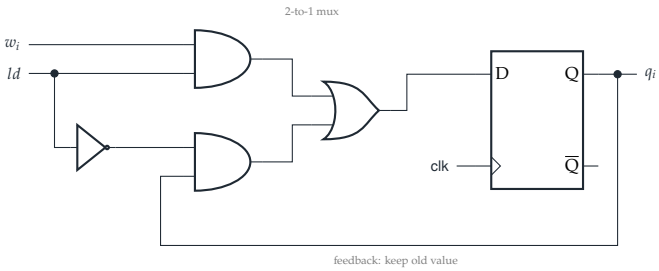
G.3 The register: one bit cell, eight times

A, B, OUT, IR (8 bits) and MAR (4 bits) are all the same circuit. Each bit is a **D flip-flop** with a **2-to-1 mux** in front of its D input:

$$q_i^+ = ld \cdot w_i \; + \; \overline{ld} \cdot q_i$$

where $ld = 1$ when the load signal ($\overline{L_a}$, $\overline{L_b}$, ...) is 0. When $ld = 1$ the flip-flop takes the bus value at the clock edge. Otherwise the mux feeds q_i back, so the bit **keeps** its value. That feedback loop is what "a register holds a value" means in gates.

On the breadboard: two **74LS173** chips (4-bit D register with load enable and tri-state outputs) per 8-bit register. Their load input is active low, which is why SAP-1 has $\overline{L_a}$, $\overline{L_b}$, ...



One register bit: AND-OR mux + D flip-flop.

vhdl/reg_n.vhd: register, written as its gates

```
-- n-bit register with active-low load (like La', Lb').
-- Each bit is a D flip-flop whose D input comes from a
-- 2-to-1 mux: load ? bus : keep the old value.
library ieee;
use ieee.std_logic_1164.all;

entity reg_n is
  generic(N : natural := 8);
  port(clk, clr : in  std_logic;
       en     : in  std_logic;   -- clock enable
       load_n : in  std_logic;
       d      : in  std_logic_vector(N-1 downto 0);
       q      : out std_logic_vector(N-1 downto 0));
end reg_n;

architecture gates of reg_n is
  signal q_reg, q_next : std_logic_vector(N-1 downto 0);
  signal ld : std_logic;
begin
  ld <= (not load_n) and en;
  -- the mux, one AND-OR per bit, written as gates
  mux: for i in 0 to N-1 generate
    q_next(i) <= (ld and d(i))
              or ((not ld) and q_reg(i));
  end generate;
  -- the D flip-flops
  process(clk, clr)
  begin
    if clr = '1' then
      q_reg <= (others => '0');
    elsif rising_edge(clk) then
      q_reg <= q_next;
    end if;
  end process;
  q <= q_reg;
end gates;
```

Reading the code against the drawing

- `ld <= (not load_n) and en;` is the inverter on the load line. `en` is the global run/halt clock enable (§G.7).
- The `generate` loop builds one AND-OR mux per bit: `(ld and d(i))` is the top AND, `((not ld) and q_reg(i))` the bottom AND, `or` the OR gate.
- The clocked `process` is the row of D flip-flops. `clr` is the asynchronous CLR of the 74LS173.

Normal VHDL writes the mux as `if ld='1' then q_reg <= d; end if;` inside the clocked process. The synthesiser builds exactly the same AND-OR mux, or uses the flip-flop's own clock-enable pin.

Register	N	Load signal
MAR	4	$\overline{L_m}$
IR	8	$\overline{L_i}$
A	8	$\overline{L_a}$
B	8	$\overline{L_b}$
OUT	8	$\overline{L_o}$

One file, five registers: the width is a `generic`, set with `generic map(4)` or `generic map(8)` (§G.9).

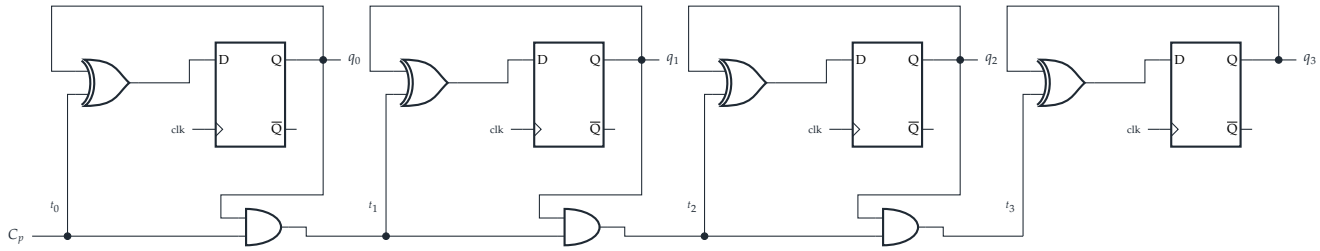
G.4 The program counter: a counter built from T flip-flops

Counting up in binary has a simple rule: **bit i flips when all lower bits are 1** (0111 → 1000). So each bit is a **T flip-flop** (a D flip-flop with an XOR feeding $q \oplus t$ back, Ch. D), and the toggle signals come from an **AND chain**:

$$t_0 = C_p, \qquad t_{i+1} = t_i \cdot q_i, \qquad q_i^+ = q_i \oplus t_i$$

With $C_p = 0$ every $t_i = 0$ and the PC holds. With $C_p = 1$ it adds one. CLR resets all bits to 0, so programs start at address 0. The PC drives the bus (through E_p) but never loads from it; that is what JMP would add (§G.12).

On the breadboard: one **74LS161** (4-bit synchronous counter) plus a 74LS245 tri-state buffer for the bus.



4-bit synchronous counter. Each stage: XOR + D flip-flop = T flip-flop ($q^+ = q \oplus t$). The AND gates along the bottom make $t_{i+1} = t_i\, q_i$.

vhdl/pc4.vhd: program counter, as gates

```
-- 4-bit program counter, synchronous binary counter.
-- Bit i toggles when cp='1' and all lower bits are 1:
--   t(i) = cp and q(0) and ... and q(i-1)
--   q+   = q xor t
library ieee;
use ieee.std_logic_1164.all;

entity pc4 is
  port(clk, clr, en : in  std_logic;
       cp : in  std_logic;                -- count
       q  : out std_logic_vector(3 downto 0));
end pc4;

architecture gates of pc4 is
  signal q_reg, t : std_logic_vector(3 downto 0);
begin
  t(0) <= cp and en;
  t(1) <= t(0) and q_reg(0);            -- AND chain
  t(2) <= t(1) and q_reg(1);
  t(3) <= t(2) and q_reg(2);
  process(clk, clr)
  begin
    if clr = '1' then
      q_reg <= (others => '0');
    elsif rising_edge(clk) then
      q_reg <= q_reg xor t;              -- T flip-flops
    end if;
  end process;
  q <= q_reg;
end gates;
```

- `t(1) <= t(0) and q_reg(0);` … are the three AND gates of the drawing.
- `q_reg <= q_reg xor t;` is the four XOR gates; the clocked process is the four D flip-flops.
- `t(0) <= cp and en;`: with `en` (run) = 0 the counter cannot count, even if `cp` = 1.

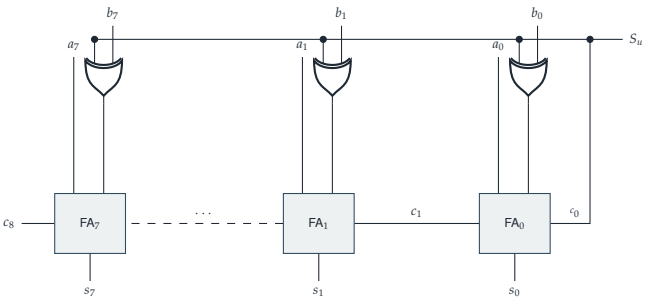
Normal VHDL: `if cp='1' then q <= q + 1;` with `unsigned` (Chu §4.3). The synthesiser builds the same logic, usually on the FPGA's fast carry chain.

G.5 The adder-subtractor: XOR gates plus a ripple of full adders

Ch. C built the full adder from two half adders. Eight in a chain make an 8-bit adder. For **subtraction** use two's complement: $A - B = A + \overline{B} + 1$.

- An XOR gate is a **controlled inverter**: $b_i \oplus 0 = b_i$ and $b_i \oplus 1 = \overline{b_i}$.
- So pass every b_i through an XOR with S_u, and connect S_u to the **carry in** c_0 as well. That adds the $+1$.
- The adder runs all the time (it is combinational). E_u only decides when its result goes onto the bus.

On the breadboard: two **74LS283** (4-bit adder) and two **74LS86** (quad XOR).



8-bit adder-subtractor: an XOR on each b_i; S_u also enters as c_0.

vhdl/full_adder.vhd: one full adder

```
-- Full adder written as its gates:
-- two half adders (XOR + AND) and an OR.
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
  port(a, b, cin : in  std_logic;
       s, cout   : out std_logic);
end full_adder;

architecture gates of full_adder is
  signal p, g1, g2 : std_logic;
begin
  p    <= a xor b;        -- half adder 1: sum
  g1   <= a and b;        -- half adder 1: carry
  s    <= p xor cin;      -- half adder 2: sum
  g2   <= p and cin;      -- half adder 2: carry
  cout <= g1 or g2;       -- OR of the two carries
end gates;
```

vhdl/addsub8.vhd: eight of them with for ... generate

```
-- 8-bit adder-subtractor:
--   su='0': s = A + B      su='1': s = A - B
-- XOR gates invert B when su='1', and su is also the
-- carry in, so A + not(B) + 1 = A - B.
library ieee;
use ieee.std_logic_1164.all;

entity addsub8 is
  port(a, b : in  std_logic_vector(7 downto 0);
       su   : in  std_logic;
       s    : out std_logic_vector(7 downto 0));
end addsub8;

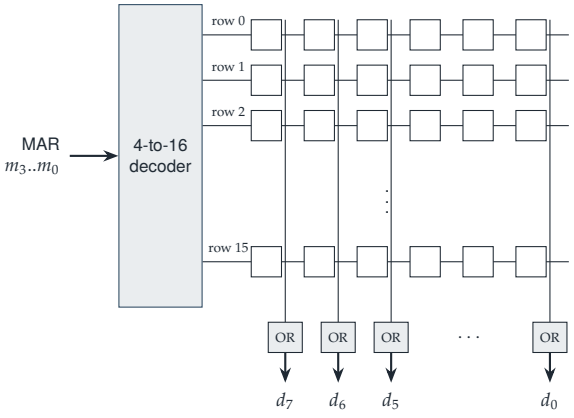
architecture ripple of addsub8 is
  signal bx : std_logic_vector(7 downto 0);
  signal c  : std_logic_vector(8 downto 0);
begin
  c(0) <= su;
  stage: for i in 0 to 7 generate
    bx(i) <= b(i) xor su;     -- controlled inverter
    fa: entity work.full_adder
      port map(a => a(i), b => bx(i), cin => c(i),
               s => s(i), cout => c(i+1));
  end generate;
  -- c(8) = carry out; SAP-1 ignores it (no flags)
end ripple;
```

`for ... generate` is VHDL's "copy this circuit N times": it makes 8 XOR gates and 8 `full_adder` instances, wired by index, exactly the chain in the drawing. Normal VHDL: `s <= a + b when su='0' else a - b;`

▷ Gate-level check: 3CH − 20H with $S_u = 1$

B = `0010 0000` $\xrightarrow{\text{XOR with 1}}$ `1101 1111`. Then $A + \overline{B} + c_0$ = `0011 1100` + `1101 1111` + 1 = `1 0001 1100`. Drop c_8: `0001 1100` = **1CH** (28). This is the last step of the Ch. F program.

G.6 The memory: a decoder, a grid of cells and an OR per column



RAM 16×8: the decoder turns on one row (word line); each cell on that row passes its bit to its column; the column OR gives $d_7..d_0$.

Read path in gates (Ch. E): the **decoder** is 16 AND gates, one per address, so exactly one **word line** is 1. Each cell is a latch whose output is ANDed with its word line. Each **column** ORs its 16 cells:

$$d_j = \sum_{r=0}^{15} \text{row}_r \cdot \text{cell}_{r,j}$$

That is a 16-input AND-OR mux per data bit, the same structure as the bus.
On the breadboard: two **74189** chips (16×4 RAM, inverted tri-state outputs) and DIP switches for typing in the program.

vhdl/ram16x8.vhd: memory with the program preloaded

```
-- 16 x 8 RAM, program preloaded (the "switches").
-- Read is combinational, like the 74189 chip:
-- address in, data out after the access time.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ram16x8 is
  port(addr : in  std_logic_vector(3 downto 0);
       dout : out std_logic_vector(7 downto 0));
end ram16x8;

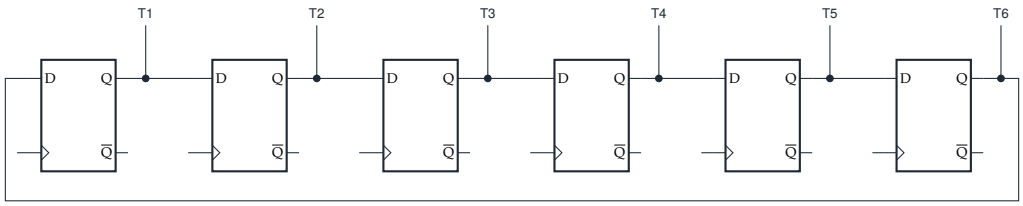
architecture rtl of ram16x8 is
  type ram_t is array (0 to 15) of
       std_logic_vector(7 downto 0);
  constant RAM : ram_t := (
    0  => x"09",  -- LDA 9H
    1  => x"1A",  -- ADD AH
    2  => x"1B",  -- ADD BH
    3  => x"2C",  -- SUB CH
    4  => x"E0",  -- OUT
    5  => x"F0",  -- HLT
    9  => x"10",  -- 16
    10 => x"14",  -- 20
    11 => x"18",  -- 24
    12 => x"20",  -- 32
    others => x"00");
begin
  dout <= RAM(to_integer(unsigned(addr)));
end rtl;
```

- The array *is* the grid; `RAM(to_integer(...))` *is* the decoder plus the column ORs. Reading is combinational, like the 74189.
- SAP-1 never writes memory while running, so the contents are a `constant` (the switches). The synthesiser reports **"found ROM"** and builds it from LUTs.
- To make it writable, add a clocked write:
  `if rising_edge(clk) then`
  `~~if we='1' then ram(a) <= din;`
  (Chu §11.3).

G.7 The control unit in gates

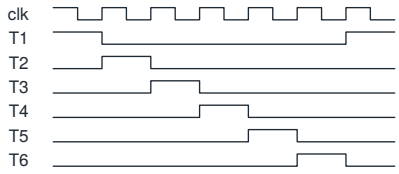
The control unit is what turns the blocks into a computer. It has three parts, and each is small in gates.

1. Ring counter: which step are we on?



T6 feeds back to T1. CLR presets the first flip-flop to 1 and clears the others. All clock inputs share one clock.

Six D flip-flops in a loop. A single 1 circulates: T1, T2, ..., T6, T1, ...



One machine cycle = 6 clocks; each T_k is 1 for one clock.

A ring counter is a **one-hot** FSM (Chu §5.3.2): one flip-flop per state, no state decoder needed, because each T_k wire already *means* "we are in step k".

Why Malvino clocks it on the falling edge

In the chip version the ring counter changes on the **falling** edge, so the control signals are stable half a cycle before the registers load on the **rising** edge. In an FPGA everything uses the rising edge: the control signals are combinational from the current state and settle well within one clock period. That is the normal synchronous rule (Chu §4.1).

vhdl/ring6.vhd: ring counter

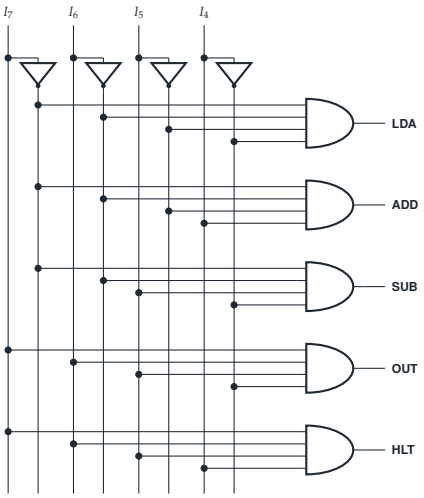
```
-- 6-bit ring counter: T1..T6, exactly one bit is 1.
-- Six D flip-flops in a loop; CLR presets T1.
library ieee;
use ieee.std_logic_1164.all;

entity ring6 is
  port(clk, clr, en : in  std_logic;
       t : out std_logic_vector(1 to 6));
end ring6;

architecture gates of ring6 is
  signal r : std_logic_vector(1 to 6);
begin
  process(clk, clr)
  begin
    if clr = '1' then
      r <= "100000";                    -- T1 active
    elsif rising_edge(clk) then
      if en = '1' then
        r <= r(6) & r(1 to 5);          -- rotate right
      end if;
    end if;
  end process;
  t <= r;
end gates;
```

`r(6) & r(1 to 5)` is the wiring in the drawing: each flip-flop takes the output of the one before it, and the first takes the last.

2. Instruction decoder: which instruction is it?



Opcode decoder. Each input bit I_k has a plain rail and an inverted rail $\overline{I_k}$. A dot is a connection: a dot on the plain rail needs a 1, on the inverted rail a 0.

The upper nibble of IR, $I_7..I_4$, is the **opcode**. Each instruction gets one 4-input AND gate that is 1 only for its own bit pattern, so at most one decoder output is 1. Example:

$$\text{ADD} = 0001 = \overline{I_7}\,\overline{I_6}\,\overline{I_5}\,I_4$$

Two helper ORs save gates in the control matrix:

$$\text{MEM} = \text{LDA} + \text{ADD} + \text{SUB}$$

(the instruction reads RAM), and

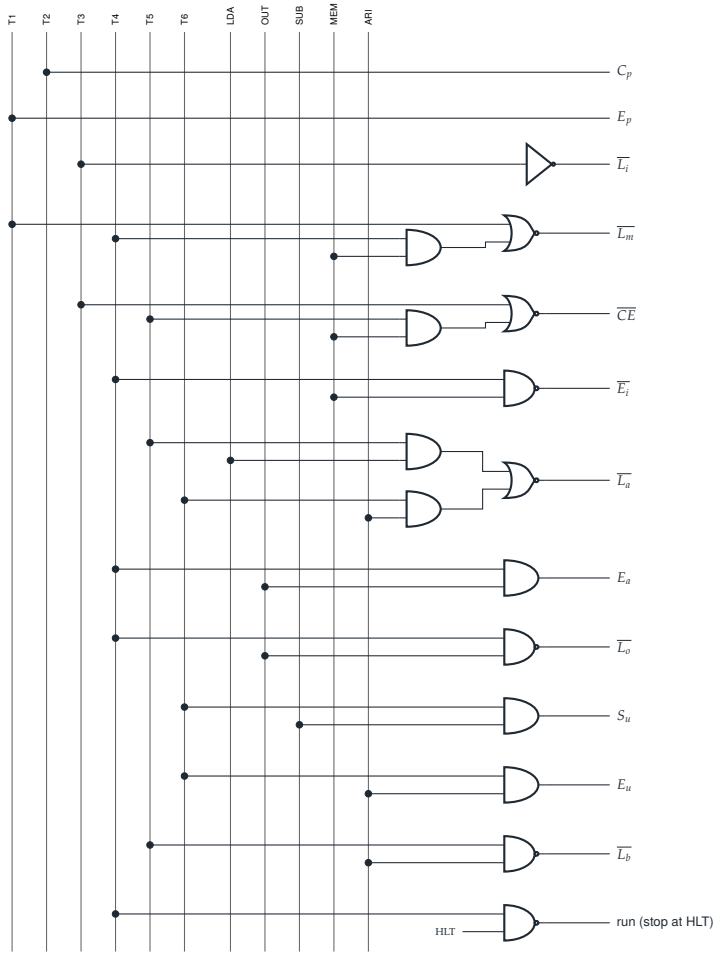
$$\text{ARI} = \text{ADD} + \text{SUB}$$

(the instruction uses B and the ALU).

vhdl/control.vhd: the decoder (n = the inverted rails)

```
-- instruction decoder (one AND gate per opcode)
n <= not op;          -- the inverted rails
lda  <= n(3) and n(2)  and n(1)  and n(0);  -- 0000
add  <= n(3) and n(2)  and n(1)  and op(0); -- 0001
sub  <= n(3) and n(2)  and op(1) and n(0);  -- 0010
outp <= op(3) and op(2) and op(1) and n(0); -- 1110
hlt  <= op(3) and op(2) and op(1) and op(0);-- 1111
mem  <= lda or add or sub;      -- uses RAM
ari  <= add or sub;             -- uses B and the ALU
```

3. Control matrix: which signals now?



The complete control matrix in gates. Vertical lines are the ring counter and decoder outputs (MEM and ARI from the two helper ORs); a dot is a connection. Active-low outputs come from NOR and NAND gates. ("run" is simplified here: the real circuit also has a halt flip-flop, §G.9.)

Each control signal is a sum of products "**this step** AND **this instruction**", read straight off the T-state table in Ch. F. For example $\overline{L_m}$ is active in T1 (every fetch) and in T4 of memory instructions, so $\overline{L_m} = \overline{T_1 + T_4 \cdot \mathrm{MEM}}$: one AND gate and one NOR gate. The active-low signals come straight out of NOR and NAND gates, so no extra inverters are needed.

▲ Stopping: never gate the clock in an FPGA

Malvino stops SAP-1 by ANDing CLK with $\overline{\mathrm{HLT}}$. In an FPGA, a gate on the clock causes clock skew and glitches (Chu §4.1.3). Instead keep the clock running and use a **clock enable**: `run` goes to the `en` input of every register, the PC and the ring counter, so nothing changes any more. A halt flip-flop keeps `run` at 0 (§G.9).

Hardwired control is just an FSM output decoder

Ring counter = state register (one-hot). Decoder + matrix = the **output logic** of a Moore/Mealy machine (Chu Ch. 5): each output is a function of (state, opcode). A microprogrammed controller replaces the matrix with a ROM addressed by (opcode, T-state).

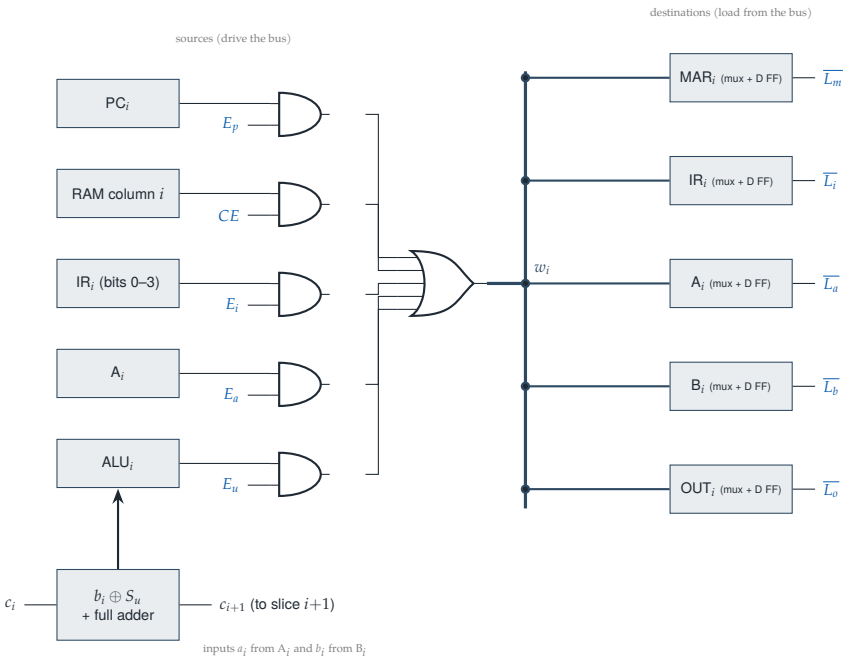
vhdl/control.vhd: the control matrix

```
-- control matrix (Malvino's equations)
cp   <= t(2);
ep   <= t(1);
lm_n <= not (t(1) or (t(4) and mem));
ce_n <= not (t(3) or (t(5) and mem));
li_n <= not t(3);
ei_n <= not (t(4) and mem);
la_n <= not ((t(5) and lda) or (t(6) and ari));
ea   <= t(4) and outp;
su   <= t(6) and sub;
eu   <= t(6) and ari;
lb_n <= not (t(5) and ari);
lo_n <= not (t(4) and outp);
```

Line by line the VHDL *is* the drawing: `not (a or b)` is a NOR gate, `not (a and b)` a NAND gate, `t(4) and outp` an AND gate. The whole control unit is about 25 gates.

G.8 One bit of the whole data path

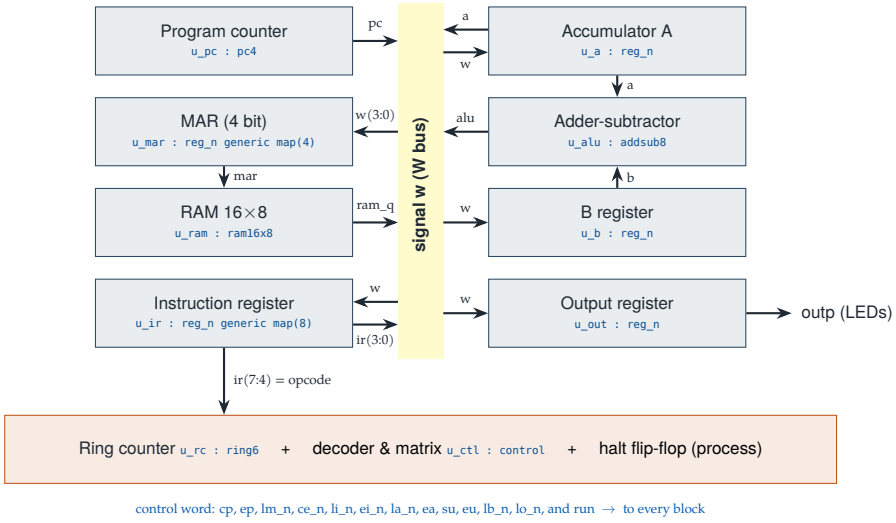
Put the pieces together for **one bit position** i. The 8-bit data path is **8 copies of this slice** side by side (bits 0–3 also contain a PC bit, a MAR bit and an IR operand bit). Only the adder's carry wire connects neighbouring slices. The control signals (blue) are shared: one $\overline{L_a}$ wire goes to all 8 A flip-flops.



Bit slice i of SAP-1: five sources → AND-OR bus → five registers. A and IR appear on both sides because they can drive the bus and load from it.

Read it as one clock cycle, left to right: the control unit enables **one source**; its bit passes the AND-OR bus onto w_i; at the rising edge **the register whose load is active** copies w_i. Every instruction is a short sequence of these transfers. **Size of the whole computer**: 5 registers ($4 + 8 + 8 + 8 + 8 = 36$ bits), PC 4, ring counter 6, halt 1: **47 flip-flops**. Plus 8 × (XOR + full adder), 8 bus muxes, the 16-row memory decoder and about 25 control gates. A few hundred gates: a tiny fraction of even the smallest FPGA.

G.9 The top level: the block diagram written as VHDL



SAP-1 block diagram with the VHDL instance names. Each box is one `entity work.x port map(...)` statement in `sap1.vhd`; each arrow is a `signal`.

vhdl/sap1.vhd: the whole computer, structurally (1/2)

```
-- SAP-1 top level: wires the blocks to one 8-bit W bus.
-- The bus is an AND-OR multiplexer: each source is
-- ANDed with its enable and the results are ORed.
-- (It replaces the tri-state bus of the chip version.)
library ieee;
use ieee.std_logic_1164.all;

entity sap1 is
  port(clk, clr : in  std_logic;
       outp     : out std_logic_vector(7 downto 0);
       halted   : out std_logic);
end sap1;

architecture structural of sap1 is
  signal w : std_logic_vector(7 downto 0);  -- W bus
  signal pc, mar : std_logic_vector(3 downto 0);
  signal ram_q, ir, a, b, alu :
         std_logic_vector(7 downto 0);
  signal t : std_logic_vector(1 to 6);
  signal cp, ep, lm_n, ce_n, li_n, ei_n, la_n, ea,
         su, eu, lb_n, lo_n, hlt : std_logic;
  signal run, halt_reg : std_logic;

  -- repeat one enable bit 8 times, for AND-ing a byte
  function x8(e : std_logic) return std_logic_vector is
  begin
    return (7 downto 0 => e);
  end function;
begin
  -- HLT: stop with a clock ENABLE, never gate the clock
  process(clk, clr)
  begin
    if clr = '1' then
      halt_reg <= '0';
    elsif rising_edge(clk) then
      if t(4) = '1' and hlt = '1' then
        halt_reg <= '1';
      end if;
    end if;
  end process;
  run    <= not (halt_reg or (t(4) and hlt));
  halted <= halt_reg;
```

vhdl/sap1.vhd (2/2): the bus and the boxes

```
  -- the W bus: AND-OR mux of all bus sources
  w <= (x8(ep)       and ("0000" & pc))
    or (x8(not ce_n) and ram_q)
    or (x8(not ei_n) and ("0000" & ir(3 downto 0)))
    or (x8(ea)       and a)
    or (x8(eu)       and alu);

  u_pc : entity work.pc4
    port map(clk, clr, run, cp, pc);
  u_mar: entity work.reg_n generic map(4)
    port map(clk, clr, run, lm_n, w(3 downto 0), mar);
  u_ram: entity work.ram16x8
    port map(mar, ram_q);
  u_ir : entity work.reg_n generic map(8)
    port map(clk, clr, run, li_n, w, ir);
  u_a  : entity work.reg_n generic map(8)
    port map(clk, clr, run, la_n, w, a);
  u_b  : entity work.reg_n generic map(8)
    port map(clk, clr, run, lb_n, w, b);
  u_alu: entity work.addsub8
    port map(a, b, su, alu);
  u_out: entity work.reg_n generic map(8)
    port map(clk, clr, run, lo_n, w, outp);
  u_rc : entity work.ring6
    port map(clk, clr, run, t);
  u_ctl: entity work.control
    port map(ir(7 downto 4), t, hlt, cp, ep, lm_n, ce_n,
             li_n, ei_n, la_n, ea, su, eu, lb_n, lo_n);
end structural;
```

How to read it

- Each `signal` is a **wire** or bundle of wires in the block diagram: `w` is the bus, `t` the 6 ring-counter lines, `cp` … `lo_n` the control word.
- Each `entity work.x port map(...)` is **one box**. These use *positional* mapping: signals listed in the same order as the entity's ports. `generic map(4)` makes the 4-bit MAR from the same file as the 8-bit registers.
- The bus and the halt process are the only logic written in the top file. Everything else is inside the boxes.
- `run` is the global clock enable. It drops to 0 in T4 of HLT and `halt_reg` keeps it there. The clock keeps running.

Block	Chips	VHDL file
Register	74LS173 ×2	`reg_n.vhd`
PC	74LS161	`pc4.vhd`
ALU	74LS283 ×2, 74LS86 ×2	`addsub8.vhd`
RAM	74189 ×2	`ram16x8.vhd`
Bus drivers	tri-state outputs, 74LS245	AND-OR in `sap1.vhd`
Control	flip-flops, AND/-NAND/NOR chips	`ring6.vhd`, `control.vhd`

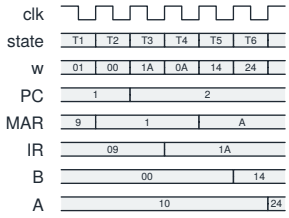
Structural VHDL = the block diagram as text

If you can draw the boxes and wires, you can write the top level: one signal per wire, one instance per box. If you can read the top level, you can draw the diagram. Exam questions go in both directions.

G.10 One instruction through all three views

Take `ADD AH` at address 1 of the Ch. F program. LDA has just finished, so A = 10H and PC = 1. Follow its 6 clocks:

T	Gates that are active	Bus w	Loads
1	E_p; NOR gives $\overline{L_m} = 0$	PC = 01	MAR
2	C_p	00 (no source)	PC+1
3	$\overline{CE} = 0$, $\overline{L_i} = 0$	RAM[1] = 1A	IR
4	ADD, MEM; $\overline{E_i} = \overline{L_m} = 0$	IR(3:0) = 0A	MAR
5	MEM, ARI; $\overline{CE} = \overline{L_b} = 0$	RAM[A] = 14	B
6	ARI; $E_u = 1$, $\overline{L_a} = 0$	ALU = 24	A



Each register changes at the rising edge that *ends* the step that loads it (hex).

T5 in gates: the ring counter makes T5 = 1 and the decoder makes ADD = 1, so MEM = ARI = 1. The NOR for $\overline{CE}$ outputs 0, which ANDs RAM onto the bus. The NAND for $\overline{L_b}$ outputs 0, which switches B's eight muxes to "load". At the rising edge all 8 B flip-flops copy 14H. Every other register's mux is on "keep".

The same 6 steps in the FSMD of §G.11

```
when t1 => mar <= pc;          -- Ep, Lm'
when t2 => pc  <= pc + 1;      -- Cp
when t3 => ir  <= RAM(...mar); -- CE', Li'
when t4 => mar <= ir(3 downto 0); -- Ei', Lm'
when t5 => b   <= RAM(...mar); -- CE', Lb'
when t6 => a   <= a + b;       -- Eu, La'
```

Each RT operation names a **source** (who drives the bus) and a **destination** (who loads). Those are exactly the active control signals in the comments.

G.11 The same computer as one FSMD

This is the **behavioural** version: no bus, no control signals, no gates; only the RT operation of each state, as in Chu Ch. 6. It has the same ports as `sap1.vhd` and gives the same result (the testbench checks both at once).

	Structural	FSMD
Describes	the hardware: blocks, wires, gates	what happens each clock
Bus	explicit AND-OR mux	invisible; the tool builds muxes
Control	ring counter, decoder, matrix	`case state` and `case op`
Size	9 files	1 file, ≈65 lines
New instruction	new gates, edit `control.vhd`	one more `when` line
Good for	understanding, exam drawings, reuse	writing real designs quickly

GHDL output: `gates: OUT = 1C, fsmd: OUT = 1C` and `PASS`. The program takes $5 \times 6 + 4 = 34$ clocks to reach T4 of HLT.

vhdl/tb_sap1.vhd: testbench (core)

```
dut1: entity work.sap1
  port map(clk, clr, o1, h1);   -- gate style
dut2: entity work.sap1_fsmd
  port map(clk, clr, o2, h2);   -- one FSMD

process
  variable cycles : natural := 0;
begin
  wait until clr = '0';
  while h1 = '0' loop
    wait until rising_edge(clk);
    cycles := cycles + 1;
  end loop;
  wait for T;
  report "gates: OUT = " & to_hstring(o1)
       & ", fsmd: OUT = " & to_hstring(o2)
       & ", cycles = " & integer'image(cycles);
  assert o1 = x"1C" and o2 = x"1C"
    report "WRONG RESULT" severity failure;
  report "PASS" severity note;
  std.env.finish;
```

vhdl/sap1_fsmd.vhd: SAP-1 in one process (1/2)

```
-- The same SAP-1 as ONE behavioural FSMD (Chu Ch.6).
-- The synthesiser builds the gates of the other files
-- from this description by itself.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity sap1_fsmd is
  port(clk, clr : in  std_logic;
       outp     : out std_logic_vector(7 downto 0);
       halted   : out std_logic);
end sap1_fsmd;

architecture fsmd of sap1_fsmd is
  type state_t is (t1, t2, t3, t4, t5, t6, halt);
  type ram_t is array (0 to 15) of unsigned(7 downto 0);
  constant RAM : ram_t := (0 => x"09", 1 => x"1A",
    2 => x"1B", 3 => x"2C", 4 => x"E0", 5 => x"F0",
    9 => x"10", 10 => x"14", 11 => x"18", 12 => x"20",
    others => x"00");
  signal state : state_t;
  signal pc, mar : unsigned(3 downto 0);
  signal ir, a, b, o : unsigned(7 downto 0);
  alias op : unsigned(3 downto 0) is ir(7 downto 4);
begin
  process(clk, clr)
  begin
    if clr = '1' then
      state <= t1; pc <= (others => '0');
      mar <= (others => '0'); ir <= (others => '0');
      a <= (others => '0'); b <= (others => '0');
      o <= (others => '0');
    elsif rising_edge(clk) then
      case state is
        when t1 => mar <= pc;          state <= t2;
        when t2 => pc  <= pc + 1;      state <= t3;
        when t3 => ir  <= RAM(to_integer(mar));
                   state <= t4;
```

vhdl/sap1_fsmd.vhd (2/2)

```
        when t4 =>
          state <= t5;
          case op is
            when "0000" | "0001" | "0010" => -- memory
              mar <= ir(3 downto 0);
            when "1110" => o <= a;              -- OUT
            when "1111" => state <= halt;       -- HLT
            when others => null;
          end case;
        when t5 =>
          state <= t6;
          if op = "0000" then
            a <= RAM(to_integer(mar));   -- LDA
          elsif op = "0001" or op = "0010" then
            b <= RAM(to_integer(mar));   -- ADD/SUB
          end if;
        when t6 =>
          state <= t1;
          if op = "0001" then a <= a + b;       -- ADD
          elsif op = "0010" then a <= a - b;    -- SUB
          end if;
        when halt => null;
      end case;
    end if;
  end process;
  outp   <= std_logic_vector(o);
  halted <= '1' when state = halt else '0';
end fsmd;
```

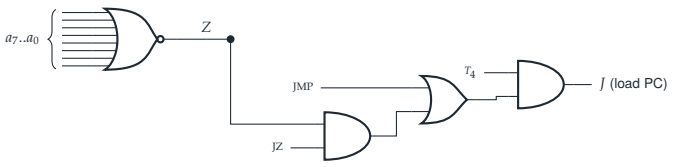
G.12 Extending it: JMP and JZ in gates and in VHDL

JMP addr (say opcode `0110`): in T4, copy IR(3:0) into the PC.

- **Gates**: a 6th decoder AND for `0110`. The PC needs a **load** input: a 2-to-1 mux in front of each T flip-flop (the register cell of §G.3). A new control signal $J = T_4 \cdot \text{JMP}$ loads it, and $\overline{E_i} = \overline{T_4(\text{MEM} + \text{JMP})}$ so that IR drives the bus.
- **FSMD**: one line in T4,
  `when "0110" => pc <= ir(3 downto 0);`

JZ addr (say `0111`): jump only if A = 0.

- **Gates**: a **zero flag** is an 8-input **NOR** of $a_7..a_0$ (1 only when all bits are 0). Then $J = T_4(\text{JMP} + \text{JZ} \cdot Z)$.
- **FSMD**: `when "0111" =>`
  `~~if a = 0 then pc <= ir(3 downto 0); end if;`



Zero flag and jump logic: $Z = \overline{a_7 + \cdots + a_0}$, $J = T_4(\text{JMP} + \text{JZ} \cdot Z)$

Why jumps make it a real computer

With a conditional jump a program can **loop** and **decide**. Given enough memory, SAP-1 could then compute anything a bigger computer can. That step, from straight-line code to branches, is the step from SAP-1 to SAP-2, to PicoBlaze (Ch. 14) and to every real CPU.

⋆ Exam practice

1. Draw one bit of a register with load enable using AND, OR, NOT and a D flip-flop. Write the same register in VHDL in two ways (gate equations and `if load`).
2. Explain why an FPGA design replaces the tri-state W bus with a multiplexer. Draw the AND-OR bus

for one bit with 5 sources.

3. Design a 4-bit synchronous counter from T flip-flops (XOR + D FF). Give the equation for t_3.
4. Show with XOR gates and full adders how one circuit both adds and subtracts. Compute 3CH − 20H at bit level.
5. Draw the SAP-1 opcode decoder for LDA, ADD, SUB, OUT, HLT. How many AND gates would a full 4-to-16 decoder need?
6. From the T-state table, derive the gate equations for $\overline{L_m}$, $\overline{CE}$ and $\overline{L_a}$ and draw them.
7. Given the block diagram, write the structural top-level VHDL (signals and port maps). Given `sap1.vhd`, draw the block diagram.
8. Trace `SUB CH` through T1–T6: active gates, bus value, which register loads, and the VHDL RT operation.
9. Add JZ: draw the zero-flag and jump logic, and change both the control matrix and the FSMD.
10. Why must the HLT instruction not gate the clock in an FPGA? What is used instead?

Part I

Basic digital circuits

VHDL, FPGA flow, combinational & sequential design, FSM, FSMD

01 Gate-level combinational circuits

VHDL program skeleton: library, entity, architecture, concurrent statements, structural description and a first testbench

Book Chapter 1, pp. 1–10

1.1 VHDL in one paragraph

VHDL = *VHSIC Hardware Description Language* (VHSIC = very high-speed integrated circuit). Sponsored by the US DoD, standardised as **IEEE 1076** (VHDL-87, the book uses **VHDL-93**). It can model a system at many levels; the book uses only the *synthesizable* subset. VHDL is **case-insensitive** and **free-format**; identifiers use letters, digits and _ and must start with a letter; comments start with `--`. The book writes constants in UPPERCASE.

1.2 The skeleton of every design [§1.2]

Listing 1.1: 1-bit equality comparator

```
library ieee;
use ieee.std_logic_1164.all;
entity eq1 is
   port(
      i0, i1: in  std_logic;
      eq:     out std_logic
   );
end eq1;

architecture sop_arch of eq1 is
   signal p0, p1: std_logic;
begin
   -- sum of two product terms
   eq <= p0 or p1;
   -- product terms
   p0 <= (not i0) and (not i1);
   p1 <= i0 and i1;
end sop_arch;
```

The three parts

- **library/use**: makes the `std_logic` type from `ieee.std_logic_1164` visible.
- **entity**: the I/O "black box". Port format `name: mode type;` where mode is `in`, `out` or `inout` (bidirectional).
- **architecture**: the body. Optional declarations (internal `signals`, constants) before `begin`; concurrent statements after it. An entity may have several architectures (e.g. `sop_arch`, `struc_arch`).

$eq = i_0 \cdot i_1 + i_0' \cdot i_1'$ (sum of products).

Concurrent statements = circuit parts running in parallel

Each concurrent signal assignment is a small circuit: the left side is its output, the expression its function and inputs. When any input changes, it re-evaluates (after a propagation delay). **Order does not matter**; connections are implied by signal names. This is the big difference from C, where statements run sequentially.

1.3 Data types and operators [§1.2.4]

Item	What to remember
`std_logic`	9 values. Synthesizable: `'0'`, `'1'`, `'Z'` (high impedance). Simulation only: `'U'` (uninitialised) and `'X'` (unknown, e.g. two drivers fighting). `'-'`, `'H'`, `'L'`, `'W'` not used.
`std_logic_vector`	Array of `std_logic`. Declare `a: in std_logic_vector(7 downto 0);` MSB on the left. `a(7 downto 4)` is a slice, `a(1)` one element. Avoid ascending `(0 to 7)`.
Logical operators	`not, and, or, xor` (bitwise on vectors). `and`, `or`, `xor` have the **same precedence**, so use parentheses: `(a and b) or (c and d)`.
Strong typing	Every object has a type; only operations defined for that type are legal (important in Ch. 3).

1.4 Structural description [§1.3]

Build a larger circuit from existing components by **component instantiation**. The code is a text version of a schematic.

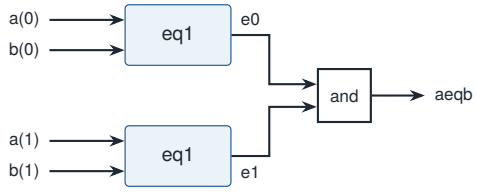


Figure 1.2: 2-bit comparator from two 1-bit comparators

Listing 1.3 (VHDL-93 direct entity instantiation)

```
architecture struc_arch of eq2 is
   signal e0, e1: std_logic;
begin
   eq_bit0_unit: entity work.eq1(sop_arch)
      port map(i0=>a(0), i1=>b(0), eq=>e0);
   eq_bit1_unit: entity work.eq1(sop_arch)
      port map(i0=>a(1), i1=>b(1), eq=>e1);
   aeqb <= e0 and e1;
end struc_arch;
```

- Syntax: `label: entity lib.entity_name(arch_name) port map(formal=>actual, ...);` **formal** = port of the component, **actual** = signal in the enclosing architecture. `work` is the default library. If `(arch_name)` is omitted, the last compiled architecture is used.
- **VHDL-87 style**: first `component eq1 port(...); end component;` in the declaration part, then `eq_bit0_unit: eq1 port map(...)`. Both are concurrent statements.

1.5 Testbench [§1.4]

A testbench is VHDL that mimics a lab bench: **UUT** (unit under test) + **test-vector generator** + (optional) **monitor**. It has an *empty entity* (no ports).

Listing 1.5 / 2.3 (shortened)

```
entity eq2_testbench is
end eq2_testbench;
architecture tb_arch of eq2_testbench is
   signal test_in0, test_in1: std_logic_vector(1 downto
     0);
   signal test_out: std_logic;
begin
   uut: entity work.eq2(struc_arch)
      port map(a=>test_in0, b=>test_in1, aeqb=>test_out)
     ;
   process   -- no sensitivity list
   begin
      test_in0 <= "00"; test_in1 <= "00";
      wait for 200 ns;
      test_in0 <= "01"; test_in1 <= "00";
      wait for 200 ns;
      -- ... more vectors ...
      assert false
         report "Simulation Completed"
         severity failure;   -- stops simulation
   end process;
end tb_arch;
```

What to remember

- The `process` is a special construct whose statements run **sequentially**; `wait for` holds the vector for a time.
- No monitor here: we read the waveform in the simulator ("virtual logic analyzer").
- `assert false report ... severity failure` is a trick to end the simulation.
- `wait for` is **not synthesizable**; it is only for testbenches.

⋆ Exam practice

1. Write the gate-level VHDL for a 2-to-4 decoder with enable, using only logical operators. [§1.6]
2. Explain the difference between a *formal* and an *actual* signal in a port map.
3. Why can the three concurrent statements in Listing 1.1 be written in any order?
4. Which `std_logic` values can be synthesised, and which only appear in simulation? What does `'X'` mean?

02 FPGA devices and EDA software

LUT-based logic cells, Spartan-3 resources, the S3 board, and the development flow from HDL to bit file

Book Chapter 2, pp. 11–33

2.1 What an FPGA is [§2.2.1]

A **field-programmable gate array** is a 2-D array of **generic logic cells** and **programmable switches**. A design is implemented by configuring each cell's function and each switch's connection, then downloading the configuration "in the field" (not in a fab).

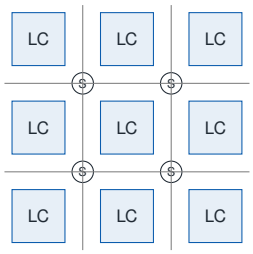


Fig. 2.1: logic cells (LC) + programmable switches (S)

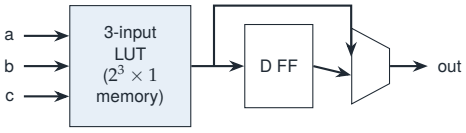


Fig. 2.2: LUT-based logic cell; output taken directly or registered

LUT = look-up table

An n-input LUT is a small $2^n \times 1$ memory. Writing its content (the truth table) lets it implement **any** n-input combinational function (e.g. $a \oplus b \oplus c$ = truth table 01101001). The D FF after it makes sequential circuits possible. **Macro cells** are hard blocks designed at transistor level: memory blocks, multipliers, clock managers, I/O blocks, even processor cores.

2.2 Xilinx Spartan-3 [§2.2.2]

- **Logic cell (LC)** = 4-input LUT + D FF + carry logic (arithmetic) + mux logic (wide muxes). The LUT can also be a 16×1 **distributed RAM** or a **16-bit shift register**.
- **Slice** = 2 LCs; **CLB** (configurable logic block) = 4 slices = 8 LCs.
- **Macro blocks**: 18×18 combinational **multiplier**; 18K-bit synchronous **block RAM**; **DCM** (digital clock manager; uses a digital delay-locked loop to reduce skew and shift phase/frequency); **IOB** (input/output block; supports many I/O standards, holds the only tri-state buffers).

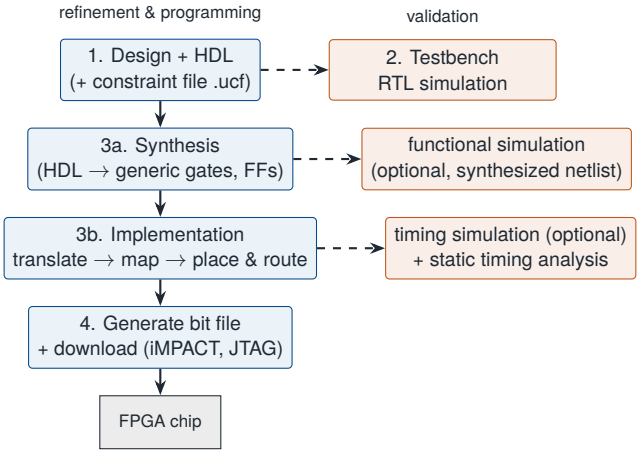
Device	LCs	BRAMs / mult.	BRAM bits
XC3S50	1,728	4	72K
XC3S200	**4,320**	**12**	**216K**
XC3S400	8,064	16	288K
XC3S1000	17,280	24	432K
XC3S5000	74,880	104	1,872K

Table 2.1 (excerpt). The S3 board uses the **XC3S200** (FT256, speed −4).

2.3 The Digilent S3 board [§2.3]

Resource	Used in
50 MHz crystal oscillator (20 ns period)	every design (system clock)
8 slide switches, 4 push buttons, 8 LEDs	testing circuits; buttons need debouncing (Ch. 5–6)
4-digit 7-segment display, **active low**, time-multiplexed	Ch. 3.7, 4.5.1
RS-232 port + voltage-level converter	UART, Ch. 7
PS2 port (3.3 or 5 V jumper)	keyboard/mouse, Ch. 8–9
VGA port (1 bit per colour = 8 colours)	Ch. 12–13
Two 256K × 16 asynchronous SRAMs (IS61LV25616AL, 10 ns)	Ch. 10
2M-bit platform flash PROM (XCF02S), JTAG port, three 40-pin expansion headers	configuration, I/O

2.4 Development flow [§2.4]



The steps in words (exam favourite)

- **Synthesis** (logic synthesis): HDL constructs → generic gate-level netlist (gates, FFs).
- **Translate**: merge all design files into one netlist. **Map** (technology mapping): generic gates → the FPGA's LCs and IOBs. **Place & route**: choose physical locations and wire routes. **Static timing analysis** at the end gives max delay / max clock rate.
- **Programming file** (`.bit`) is generated from the final netlist and shifted serially into the FPGA.
- **RTL simulation** checks the HDL; **functional simulation** (after synthesis) checks synthesis; **timing simulation** (after implementation) uses real delays. With good coding practice, RTL simulation + static timing analysis are enough.

2.5 Tools (ISE WebPack 8.2, ModelSim XE) [§2.5–2.7]

- **ISE Project Navigator**: sources, processes, transcript and workplace windows; "auto make" runs earlier steps automatically. Synthesis tool: **XST**.
- ISE's auto-generated `STD_LOGIC_ARITH` and `STD_LOGIC_UNSIGNED` clauses are not IEEE standard: replace with `use ieee.numeric_std.all;`
- **Design summary**: slices, FFs, LUTs and whether timing constraints are met.
- **iMPACT** downloads the `.bit` file via **boundary scan (JTAG)**; the other device in the chain (PROM) is bypassed.
- **ModelSim**: create project, compile, simulate, view waveform.

User constraint file (.ucf): pin assignment

```
# 4 slide switches
NET "a<0>" LOC = "F12";  # switch 0
NET "a<1>" LOC = "G12";  # switch 1
NET "b<0>" LOC = "H14";  # switch 2
NET "b<1>" LOC = "H13";  # switch 3
NET "aeqb" LOC = "K12";  # led 0
# timing constraint (Ch. 4)
NET "clk" TNM_NET = "clk";
TIMESPEC "TS_clk" = PERIOD "clk" 20 ns HIGH 50 %;
```

Pins are fixed by the board wiring, so top-level port names must be mapped to the right pins. "Allow unmatched LOC constraints" lets one template UCF serve every design.

⋆ Exam practice

1. Show how a 3-input LUT implements $y = a \oplus b \oplus c$ (give the 8 memory bits).
2. Describe synthesis, translate, map, place-and-route and bit-file generation in one sentence each.
3. What is the difference between RTL, functional and timing simulation? Which does the book say you normally need?
4. Name the four Spartan-3 macro blocks and what each is for.

03 RT-level combinational circuits

Relational and arithmetic operators, numeric_std and type conversion, routing constructs, avoiding latches, generics and four design examples

Book Chapter 3, pp. 35–70

3.1 Operators and the `numeric_std` package [§3.2]

- **Relational**: `= /= < <= > >=` return `boolean`; synthesised as **comparators**. The book only uses the result inside routing constructs (`when`, `if`).
- **Arithmetic** on `integer/natural` is only used for **constants and array bounds**, never for synthesised signals.
- **IEEE `numeric_std`** adds `unsigned` and `signed` (arrays of `std_logic` interpreted as numbers) and overloads `+ - *` and relational ops on them (also mixed with `natural/integer`).
- `*` infers Spartan-3's 18 × 18 multipliers (XC3S200 has 12); use with care.
- **Avoid** `std_logic_arith`, `std_logic_unsigned`, `std_logic_signed`: not IEEE standard.

From a	To	Function / cast
`unsigned,signed`	`slv`	`std_logic_vector(a)`
`signed,slv`	`unsigned`	`unsigned(a)`
`unsigned,slv`	`signed`	`signed(a)`
`unsigned,signed`	`integer`	`to_integer(a)`
`natural`	`unsigned`	`to_unsigned(a,size)`
`integer`	`signed`	`to_signed(a,size)`

Table 3.3 (`slv` = `std_logic_vector`). `slv` is *not* a number: it cannot be converted directly to/from integer.

Type conversion: legal and illegal (s = slv(3 downto 0), u = unsigned(3 downto 0))

```
u1 <= s1;                    -- NOT ok: type mismatch
u1 <= unsigned(s1);          -- ok: type casting
u2 <= to_unsigned(5,4);      -- ok: conversion function
s3 <= std_logic_vector(to_unsigned(5,4));      -- ok: two conversions
u4 <= u2 + u1;  u5 <= u2 + 1;                  -- ok: unsigned+unsigned, unsigned+natural
s5 <= s2 + s1;                                  -- NOT ok: + undefined for slv
s5 <= std_logic_vector(unsigned(s2) + unsigned(s1));   -- ok
```

Data-type rules of the book (§3.2.4)

`std_logic/std_logic_vector` for ports and non-arithmetic internal signals · `'Z'` only to infer a tri-state buffer · `unsigned/signed` for signals with arithmetic · casting/conversion functions between them · `integer` only for constants and bounds · boolean results only inside routing constructs · user-defined 2-D array type for storage (register file, §4.2.3) · user-defined enumerated type for FSM states (Ch. 5).

Concatenation, shifting and tri-state [§3.2.3]

& is just wiring: fixed-amount shifts

```
-- a: std_logic_vector(7 downto 0)
rot <= a(2 downto 0) & a(7 downto 3); -- rotate right 3
shl <= "000" & a(7 downto 3);         -- logical shift
     right 3
sha <= a(7)&a(7)&a(7) & a(7 downto 3);-- arithmetic
     shift right 3
```

Built-in shift functions sometimes cannot be synthesised; a variable shift amount needs a routing circuit (barrel shifter, §3.7.3).

Tri-state buffer and bidirectional pin

```
y <= a_in when oe='1' else 'Z';
-- bi: inout std_logic
bi <= sig_out when dir='1' else 'Z';
sig_in <= bi;
```

Xilinx-specific

Spartan-3 has tri-state buffers **only in the IOBs**, so `'Z'` can only be used on ports mapped to physical pins.

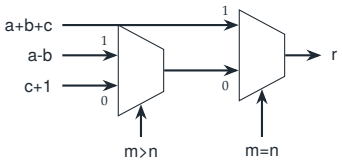
3.2 Routing: concurrent assignment statements [§3.3]

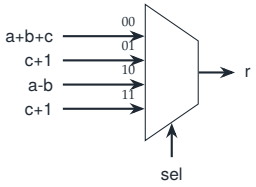
Conditional signal assignment (`when-else`) ⇒ **priority** routing network: a *cascade of 2-to-1 muxes*. Conditions evaluated in order; delay grows with the number of clauses.

```
r <= a+b+c when m=n else
     a-b   when m>n else
     c+1;
```

Selected signal assignment (`with-select`) ⇒ one 2^n-**to-1 multiplexer**. Choices must be **mutually exclusive** and **all-inclusive**; `when others` is always needed because `std_logic` has unsynthesisable values (`'X'`, `'U'`...). Use `|` to list several values.

```
with sel select
  r <= a+b+c when "00",
       a-b   when "10",
       c+1   when others;
```





In both cases **all value expressions are computed in parallel**; the conditions only drive the mux select lines.

3.3 Processes, `if` and `case` [§3.4–3.5]

A `process` is one concurrent statement whose body is **sequential statements**. The book uses processes for only two things: **routing with `if`/`case`** and **templates for memory elements** (Ch. 4). For combinational logic, the **sensitivity list must contain all inputs**.

Multiple assignments: inside a process only the *last* assignment to a signal takes effect (`c<=a and b; c<=a or b;` = `c<=a or b`). Outside a process the same two lines would create two drivers tied together = **design error**.

- `if/elsif/else` ≡ `when-else` (priority network), when each branch holds one assignment.
- `case ... when others` ≡ `with-select` (multiplexer).
- `if/case` are more flexible: any number of statements per branch, nesting. Example: one comparator `if a>b then large<=a; small<=b; else ...` instead of two `>` in two `when-else` statements. Max of three values is clearer as a nested `if`.

▲ Unintended memory (latch) §3.5.4

VHDL says a signal **keeps its previous value** if it is not assigned in a process. Synthesis then infers a latch or feedback loop. Three rules for combinational processes: (**1**) include **all input signals** in the sensitivity list; (**2**) always include the **`else`** branch; (**3**) **assign every output in every branch**.

Wrong: breaks all three rules

```
process(a)              -- b missing
begin
  if (a > b) then       -- eq not assigned
    gt <= '1';
  elsif (a = b) then    -- gt not assigned
    eq <= '1';
  end if;               -- no else
end process;
```

Fixed with default values at the top

```
process(a,b)
begin
  gt <= '0';  eq <= '0';  -- defaults
  if (a > b) then
    gt <= '1';
  elsif (a = b) then
    eq <= '1';
  end if;
end process;
```

3.4 Constants and generics [§3.6]

`constant N: integer := 4;` replaces "hard literals"; evaluated at pre-processing, costs no hardware. A **generic** passes a parameter into an entity (like a constant that can be set per instance) ⇒ **scalable, reusable** code. Set it with `generic map`; if omitted, the default is used.

Adder with carry-out, extended by one bit

```
a_ext   <= unsigned('0' & a);
b_ext   <= unsigned('0' & b);
sum_ext <= a_ext + b_ext;
sum  <= std_logic_vector(sum_ext(N-1 downto 0));
cout <= sum_ext(N);
```

Listing 3.11: generic adder + instantiation

```
entity gen_add_w_carry is
  generic(N: integer := 4);
  port(a, b: in  std_logic_vector(N-1 downto 0);
       cout: out std_logic;
       sum:  out std_logic_vector(N-1 downto 0));
end gen_add_w_carry;
...
adder_8_unit: entity work.gen_add_w_carry
   generic map(N=>8)
   port map(a=>a8, b=>b8, cout=>c8, sum=>sum8);
```

3.5 Design examples [§3.7]

Circuit	Idea to remember
Hex-to-7-segment decoder	One `with hex select` listing 16 patterns for `sseg(6 downto 0)` (segments a..g), `sseg(7)` = decimal point. Segments are **active low** ('0' = lit): "0" → `"0000001"`. The 4 displays share pins via the `disp_mux` time-multiplexing module (Ch. 4.5.1).
Sign-magnitude adder	MSB = sign, rest = magnitude (3=`0011`, −3=`1011`). Stage 1: **sort** operands by magnitude into `max/min`. Stage 2: if signs equal, **add** magnitudes; else **subtract** min from max. Result sign = sign of `max`.
Barrel shifter (8-bit rotate right by `amt`)	Version 1: `with amt select` listing 8 cases (wide mux, grows badly). Version 2: $\log_2 n$ **stages**; stage k rotates by 2^k or passes through, controlled by `amt(k)`; total = $m_2 2^2 + m_1 2^1 + m_0 2^0$.
Simplified floating-point adder	13-bit format: sign s, 4-bit exponent e, 8-bit significand f; value $(-1)^s \times .f \times 2^e$; must be **normalised** (MSB of $f = 1$) or zero; no rounding (truncate). Four stages: **sort** (compare `exp&frac`), **align** (shift small significand right by $e_b - e_s$), **add/sub** (sign-magnitude), **normalise** (count leading zeros like a priority encoder, shift left, handle carry-out by shifting right and incrementing exponent, underflow → 0).

Listing 3.17: multi-stage barrel shifter (rotate right)

```
-- stage 0, shift 0 or 1 bit
s0 <= a(0) & a(7 downto 1) when amt(0)='1' else a;
-- stage 1, shift 0 or 2 bits
s1 <= s0(1 downto 0) & s0(7 downto 2) when amt(1)='1' else s0;
-- stage 2, shift 0 or 4 bits
y  <= s1(3 downto 0) & s1(7 downto 4) when amt(2)='1' else s1;
```

⋆ Exam practice

1. Write a 4-request priority encoder using (a) `when-else`, (b) `case`. Sketch the hardware each implies.
2. Find three errors in a combinational process that infers a latch, and rewrite it correctly.
3. Correct: `signal s1,s2: std_logic_vector(7 downto 0); ... s2 <= s1 + 1;`
4. Explain the four steps of the FP adder using $0.1101 \times 2^3 + 0.1000 \times 2^1$ (decimal version in Fig. 3.8).
5. Why does a multi-stage barrel shifter scale better than a single wide mux?

04 Regular sequential circuits

D FF timing, the synchronous design method, register and register-file templates, shift registers, counters, ticks, sequential testbenches, LED multiplexing, stopwatch and FIFO

Book Chapter 4, pp. 71–110

4.1 D flip-flop and timing [§4.1]

A **sequential circuit** has memory (internal state); its output depends on input *and* state. In the **synchronous design methodology** all storage elements are driven by **one global clock** and sample data on the same edge. It lets us separate storage from logic and is the basis of synthesis, verification and test. **Every design in the book follows it.**

FF type	clk	reset / en	q^*
plain D FF	↑	–	d
async reset	–	reset=1	0
	↑	reset=0	d
sync enable	↑	en=0	q (hold)
	↑	en=1	d

Timing parameters

- T_{cq}: clock-to-q delay.
- T_{setup} / T_{hold}: d must be stable this long *before/after* the sampling edge, otherwise the FF may go **metastable**.

$$T_{clock} = T_{cq} + T_{comb} + T_{setup}$$

$$f_{max} = \frac{1}{T_{clock}} = \frac{1}{T_{cq} + T_{comb} + T_{setup}}$$

T_{comb} = worst propagation delay of the next-state logic. With the 50 MHz board clock, f_{max} must exceed 50 MHz ($T_{clock} < 20$ ns). Set it in the UCF with `TIMESPEC ... PERIOD "clk" 20 ns.`

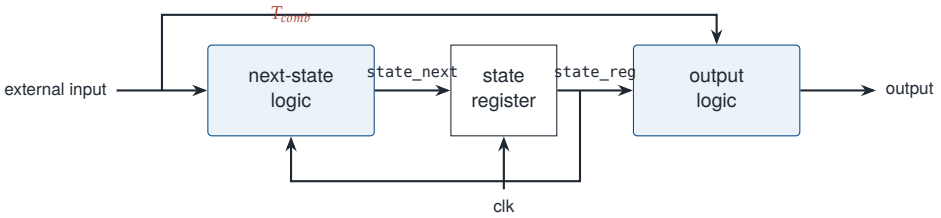


Figure 4.2: every synchronous system. Timing analysis is only needed on the single loop through the register.

Three kinds of sequential circuit (the plan of Ch. 4–6)

Regular sequential circuit: next-state logic is a regular structure (incrementer, shifter), e.g. counters, shift registers (Ch. 4). **FSM**: "random" next-state logic without a repetitive pattern (Ch. 5). **FSMD**: FSM (control path) + regular circuits (data path), used to implement **register-transfer (RT) methodology** algorithms (Ch. 6).

4.2 HDL templates for FF and register [§4.2]

Listing 4.2: D FF with asynchronous reset

```
process(clk, reset)
begin
   if (reset='1') then
      q <= '0';
   elsif (clk'event and clk='1') then
      q <= d;
   end if;
end process;
```

`clk'event and clk='1'` = rising edge (change + new value 1). Only `clk` (and `reset`) are in the sensitivity list: d is sampled only at the edge. **Asynchronous reset** breaks the synchronous method and is used *only* for power-on initialisation.

Listing 4.4: two-segment style, FF with sync enable

```
-- D FF (memory)
process(clk, reset)
begin
   if (reset='1') then
      q_reg <= '0';
   elsif (clk'event and clk='1') then
      q_reg <= q_next;
   end if;
end process;
-- next-state logic (combinational)
q_next <= d when en='1' else q_reg;
-- output logic
q <= q_reg;
```

The book's coding style: separate memory from logic

Suffix _reg = register output, _next = register input. Once the register is isolated, what remains is **pure combinational logic** coded with Ch. 3 techniques. Longer code, but you can see the circuit. **Why a sync enable?** Instead of a derived 1 MHz clock for a slow subsystem, generate an **enable tick** that is high 1 cycle every 50 cycles. Everything stays in one clock domain; this also replaces **gated clocks**.

Register file [§4.2.3]

Listing 4.6: 2^W-by-B register file

```
architecture arch of reg_file is
  type reg_file_type is array (2**W-1 downto 0) of
       std_logic_vector(B-1 downto 0);
  signal array_reg: reg_file_type;
begin
  process(clk, reset)
  begin
    if (reset='1') then
      array_reg <= (others=>(others=>'0'));
    elsif (clk'event and clk='1') then
      if wr_en='1' then
        array_reg(to_integer(unsigned(w_addr))) <=
     w_data;
      end if;
    end if;
  end process;
  -- read port
  r_data <= array_reg(to_integer(unsigned(r_addr)));
end arch;
```

- One write port (`w_addr`, `w_data`, `wr_en`), one or more read ports. Two generics: `W` address bits (2^W words), `B` bits per word.
- No built-in 2-D array in `std_logic_1164`: define an **array-of-arrays type**.
- Indexing with a signal infers a **decoder** (write) and a **multiplexer** (read).
- `(others=>'0')` sets all bits to 0.

Xilinx-specific

Each Spartan-3 LC has a D FF with async reset and sync enable. Using LCs only for storage is wasteful; for bigger storage use **distributed RAM** or **block RAM** (Ch. 11).

4.3 Simple regular circuits [§4.3]

Free-running shift register (shift right)

```
r_next <= s_in & r_reg(N-1 downto 1);
s_out  <= r_reg(0);
```

Next-state logic is pure wiring ⇒ smallest T_{comb}, highest possible f_{max}.

Universal shift register (ctrl)

```
with ctrl select
  r_next <= r_reg                          when "00", --
     no op
            r_reg(N-2 downto 0) & d(0)     when "01", --
     shift left
            d(N-1) & r_reg(N-1 downto 1)   when "10", --
     shift right
            d                              when others;
     --load
```

Does parallel-to-serial (load then shift) and serial-to-parallel (shift then read).

Listing 4.9: free-running binary counter

```
signal r_reg, r_next: unsigned(N-1 downto 0);
...
r_next <= r_reg + 1;     -- wraps automatically
q <= std_logic_vector(r_reg);
max_tick <= '1' when r_reg=(2**N-1) else '0';
```

Listing 4.11: mod-m counter (generics M and N)

```
-- N = ceil(log2 M) bits
r_next <= (others=>'0') when r_reg=(M-1) else
          r_reg + 1;
max_tick <= '1' when r_reg=(M-1) else '0';
```

Tick

A **tick** is a signal asserted for exactly **one clock cycle** (suffix `_tick`). It is used to drive the **enable** of another synchronous block. It is the book's standard way to run "slow" logic without a second clock.

syn_clr	load	en	up	q^*	operation
1	–	–	–	0	synchronous clear
0	1	–	–	d	parallel load
0	0	1	1	$q+1$	count up
0	0	1	0	$q-1$	count down
0	0	0	–	q	pause

Table 4.1: universal binary counter (priority top to bottom)

`reset` (asynchronous) is only for system initialisation; `syn_clr` is sampled at the clock edge and is fine for normal synchronous operation. Code it with one `when-else` following the table order.

4.4 Testbench for sequential circuits [§4.4]

Clock, reset and stimulus (Listing 4.12)

```
constant T: time := 20 ns;  -- 50 MHz
...
-- clock: runs forever
process
begin
   clk <= '0';  wait for T/2;
   clk <= '1';  wait for T/2;
end process;
-- reset for the first half cycle
reset <= '1', '0' after T/2;
-- stimulus
process
begin
   syn_clr <= '0'; load <= '0'; en <= '0'; up <= '1';
   d <= (others=>'0');
   wait until falling_edge(clk);
   wait until falling_edge(clk);
   load <= '1'; d <= "011";
   wait until falling_edge(clk);
   load <= '0';
   ...
   assert false report "Simulation Completed" severity
     failure;
end process;
```

- Clock process has no sensitivity list and repeats forever.
- `reset <= '1', '0' after T/2;` emulates power-on reset. Without it, signals start at `'U'`.
- **Change inputs at the falling edge** (`wait until falling_edge(clk)`) so they are stable around the rising edge (setup/hold satisfied). Each such line = one clock cycle.
- Other forms: `wait for 10*T;`, `wait until q="100";`, `for i in 1 to 10 loop wait until falling_edge(clk); end loop;`
- **Simulation tip** [§4.5.1]: large prescaler counters (18-bit) waste simulation time; make the width a constant/generic and use a small value (e.g. 4) in the testbench.

4.5 Case study 1: LED time-multiplexing [§4.5.1]

The four 7-segment digits share the 8 segment lines (a–g, dp); each digit has its own **active-low enable** `an`. Enable one digit at a time, fast enough that the eye sees all four lit. **Pins: 32 → 12** (8 segments + 4 enables).

- Refresh around **1000 Hz**: fast enough to fool the eye, slow enough for LEDs to switch fully.
- Use an **18-bit counter**; its two MSBs select the digit and drive a 2-to-4 decoder for `an`. Each digit switches at $50\text{M}/2^{16} \approx 800$ Hz.
- Hex version: mux the four 4-bit `hex` values first, then use **one** hex-to-7-seg decoder (mux width 8→5 bits).

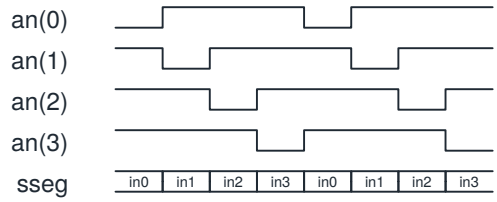


Figure 4.6 (simplified): the enables rotate

Core of disp_mux

```
constant N: integer := 18;
q_next <= q_reg + 1;
sel <= std_logic_vector(q_reg(N-1 downto N-2));
process(sel, in0, in1, in2, in3)
begin
  case sel is
    when "00" => an <= "1110"; sseg <= in0;
    when "01" => an <= "1101"; sseg <= in1;
    when "10" => an <= "1011"; sseg <= in2;
    when others => an <= "0111"; sseg <= in3;
  end case;
end process;
```

4.6 Case study 2: stopwatch 00.0–99.9 s [§4.5.2]

A 3-digit **BCD counter** (each decimal digit in 4 bits, 139_{10} = `0001 0011 1001`) with `clr` (sync clear) and `go` (enable).

- Prescaler: **mod-5,000,000** counter gives a tick every 0.1 s at 50 MHz ($5 \cdot 10^6 \times 20$ ns $= 0.1$ s).
- **Design I**: cascade of three decade (mod-10) counters. The 10-s digit is enabled only when the 0.1-s tick is high *and* the two lower digits are 9.
- **Design II**: one nested `if` ("reaches .9, 9.9, 99.9").

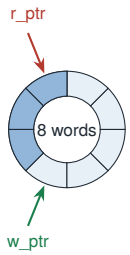
▲ Do not use a counter output as the next stage's clock

A ripple ("hooked") clock looks simpler but violates the synchronous principle: multiple clock domains, skew, glitches and timing analysis that no longer works. **Use one clock and enable ticks.** The book calls the alternative "a very poor design practice".

4.7 Case study 3: FIFO buffer [§4.5.3]

A **FIFO** is "elastic" storage between two subsystems. `wr` writes `w_data`; the head is *always* visible on `r_data`, and `rd` actually **removes** the head. Status: `full`, `empty`.

Circular-queue implementation: register file + controller with a **write pointer** (head) and a **read pointer** (tail), each advancing by one per operation. **Problem**: `r_ptr = w_ptr` both when full and when empty. **Solution**: two status FFs, `full_reg` (init 0) and `empty_reg` (init 1), updated each cycle from `wr & rd`.



Controller next-state logic (Listing 4.20)

```
wr_op <= wr & rd;
case wr_op is
 when "00" =>                        -- no op
 when "01" =>                        -- read
   if (empty_reg /= '1') then
     r_ptr_next <= r_ptr_succ;
     full_next  <= '0';
     if (r_ptr_succ = w_ptr_reg) then
       empty_next <= '1';
     end if;
   end if;
 when "10" =>                        -- write
   if (full_reg /= '1') then
     w_ptr_next <= w_ptr_succ;
     empty_next <= '0';
     if (w_ptr_succ = r_ptr_reg) then
       full_next <= '1';
     end if;
   end if;
 when others =>                      -- write and read
   w_ptr_next <= w_ptr_succ;
   r_ptr_next <= r_ptr_succ;
end case;
wr_en <= wr and (not full_reg);    -- write only if not
     full
```

⋆ Exam practice

1. $T_{cq} = 1$ ns, $T_{setup} = 0.5$ ns, $T_{comb} = 6$ ns. Find f_{max}. Can it run at 50 MHz? ($1/7.5\,ns = 133\,MHz$: *yes.*)
2. Write the complete VHDL for a mod-10 counter with a `max_tick`, using the two-segment style.

3. Why is a synchronous enable tick preferred over a derived clock for a slow subsystem?
4. How does the FIFO distinguish full from empty when both pointers are equal? Trace 8 writes then 1 read for a 4-word FIFO.
5. Design a circuit that produces a 1-cycle tick every 1 ms from the 50 MHz clock. (*mod-50,000 counter, 16 bits.*)

05 Finite state machines (FSM)

Moore vs. Mealy outputs, state diagrams and ASM charts, the FSM code template, edge detectors and the FSM debouncer

Book Chapter 5, pp. 107–130

5.1 Moore and Mealy [§5.1.1]

An **FSM** moves among a finite number of internal states; transitions depend on current state and input, with no simple repetitive pattern ("random logic"). Its main use: the **controller** of a larger system (the control path of an FSMD). Block diagram = state register + next-state logic + output logic (Fig. 5.1).

- **Moore output**: function of **state only**.
- **Mealy output**: function of **state and input**.

A complex FSM can have both.

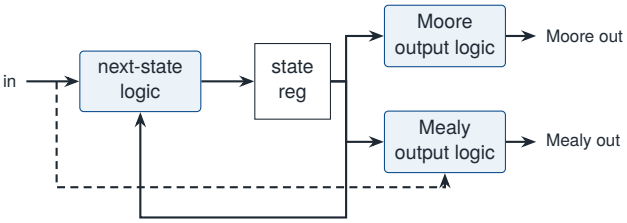
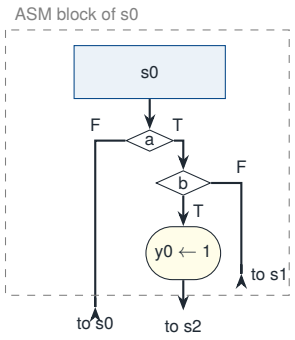


Fig. 5.1: the input also feeds Mealy output logic (dashed)

5.2 State diagram and ASM chart [§5.1.2]

State diagram: circles = states, arcs labelled with logic expressions. **Moore outputs inside the circle**; **Mealy outputs on the arc** (after the condition). Only asserted outputs are written; otherwise the default (unasserted) value holds. Compact; good for simple FSMs.

ASM chart (algorithmic state machine): network of **ASM blocks**. Each block = one **state box** (rectangle; Moore outputs; one exit), optional **decision boxes** (diamond; T/F exits) and **conditional output boxes** (rounded; Mealy outputs). Flowchart-like; better for complex conditions. Both carry the same information and convert into each other.



Part of Fig. 5.3(b): Mealy `y0` asserted in s0 when `a=b=1`

5.3 FSM code template [§5.2]

Listing 5.1 (structure)

```
architecture two_seg_arch of fsm_eg is
  type eg_state_type is (s0, s1, s2);
  signal state_reg, state_next: eg_state_type;
begin
  -- state register
  process(clk, reset)
  begin
    if (reset='1') then
      state_reg <= s0;
    elsif (clk'event and clk='1') then
      state_reg <= state_next;
    end if;
  end process;
```

```
  -- next-state logic
  process(state_reg, a, b)
  begin
    case state_reg is
      when s0 =>
        if a='1' then
          if b='1' then state_next <= s2;
          else          state_next <= s1;
          end if;
        else            state_next <= s0;
        end if;
      when s1 =>
        if (a='1') then state_next <= s0;
        else            state_next <= s1;
        end if;
```

```
      when s2 =>        state_next <= s0;
    end case;
  end process;
  -- Moore output logic
  y1 <= '1' when (state_reg=s1) or (state_reg=s2)
        else '0';
  -- Mealy output logic
  y0 <= '1' when (state_reg=s0) and (a='1') and (b='1')
        else '0';
end two_seg_arch;
```

How to write any FSM

1. **Enumerated type** for symbolic states: `type t is (s0,s1,s2);`. Synthesis does the **state assignment** (binary, one-hot...) automatically.
2. **State register** process (with async reset to the initial state).
3. **Next-state logic**: a `case state_reg is` that follows the diagram/ASM chart arc by arc.
4. **Moore** outputs from `state_reg` only; **Mealy** outputs from `state_reg` and inputs.

Alternative (Listing 5.2): merge next-state and output logic in **one** combinational process; put **default values** for all outputs and `state_next <= state_reg` at the top to avoid latches. Once the ASM chart is right, coding is almost mechanical. (ISE's StateCAD can draw diagrams and generate HDL.)

▲ Common FSM errors

Missing state/input in the sensitivity list; an output not assigned in some state (latch); forgetting `when others` if the type is not fully covered; using `state_next` where `state_reg` is meant.

5.4 Example: rising-edge detector [§5.3.1]

Generates a one-cycle **tick** when a slow input changes from 0 to 1.

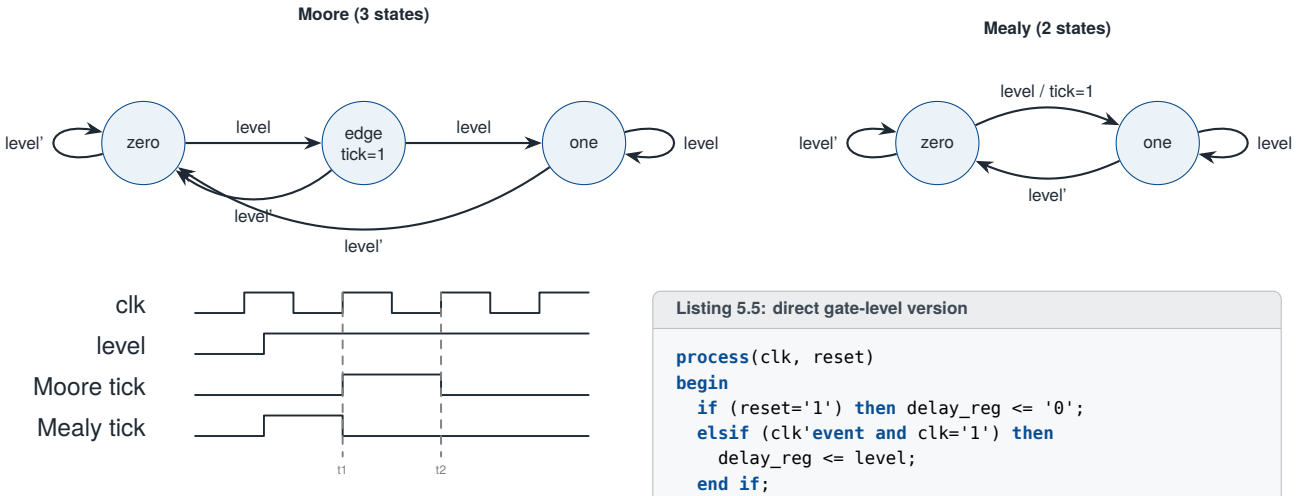


Fig. 5.5 (simplified): Mealy tick is usable at t1, Moore at t2

Listing 5.5: direct gate-level version

```
process(clk, reset)
begin
  if (reset='1') then delay_reg <= '0';
  elsif (clk'event and clk='1') then
    delay_reg <= level;
  end if;
end process;
tick <= (not delay_reg) and level;
```

Tick = "now 1 and previous 0". It is the **same circuit** as the Mealy FSM with states zero=0, one=1.

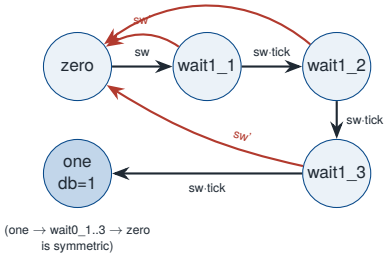
Moore vs. Mealy: the subtle differences (exam classic)

Mealy needs fewer states and **responds faster** (tick in the *same* cycle as the input change, so the output is sampled one clock earlier). But its output **width varies** with input timing and **input glitches pass straight through** to the output. **Moore** output is exactly one full clock cycle, glitch-free with respect to the input, but **one cycle later**. If the receiver is a synchronous circuit on the same clock, width and glitches do not matter as long as the output is stable around the edge, so Mealy is often preferred.

5.5 Example: debouncing circuit (FSM version) [§5.3.2]

Mechanical switches **bounce** when pressed or released; bounces usually settle within **20 ms**. Scheme: a free-running **10 ms timer** gives `m_tick`; the FSM changes the debounced output only after the input has been stable for three ticks.

- States: `zero`, `wait1_1`, `wait1_2`, `wait1_3`, `one`, `wait0_1`, `wait0_2`, `wait0_3`.
- In `zero`: `sw`=1 → `wait1_1`. In each `wait1_k`: if `sw`=0 go back to `zero`; if `m_tick` go to the next wait state; after `wait1_3` go to `one`. Mirror image for 1→0.
- Because the timer is free-running, the first tick can come at any time: actual stable time is **between 20 and 30 ms**, not exact. (Fixed with an FSMD in Ch. 6.)



Testing circuit (§5.3.3): push button → (a) edge detector directly and (b) debouncer + edge detector, each driving an 8-bit counter shown on the 7-segment display. Pressing a few times shows the raw counter counting bounces as extra edges.

⋆ Exam practice

1. Draw Moore and Mealy state diagrams for a *dual-edge* detector (tick on 0→1 and 1→0). [§5.5.1]
2. Convert the Mealy edge detector to an ASM chart and write its VHDL with the template.
3. Explain why the FSM debouncer's delay is 20–30 ms and not exactly 20 ms.
4. Parking lot: sensors a, b; car enters when the sequence is 00→10→11→01→00. Draw the FSM that asserts `enter` for one cycle. [§5.5.3]
5. Give two advantages and two disadvantages of Mealy outputs.

06 FSM with data path (FSMD)

Register-transfer methodology, ASMD charts, the delayed-store trap, explicit vs. implicit data path, and the five classic FSMD designs

Book Chapter 6, pp. 131–160

6.1 RT operations and the ASMD chart [§6.1]

An **FSMD** = **control path** (an FSM that reads commands and status, and generates control signals) + **data path** (regular sequential circuits that do the data manipulation). It implements the **register-transfer (RT) methodology**: an algorithm is a sequence of data transfers and operations among registers.

RT operation: $r_{dest} \leftarrow f(r_{src1}, r_{src2}, \dots)$. Sources feed a combinational $f(\cdot)$; the result is stored in the destination at the **next rising clock edge**. Examples: `r1 <- 1`, `r <- r`, `r0 <- r1`, `n <- n-1`, `y <- a xor b xor c`, `s <- a*a + b*b`.

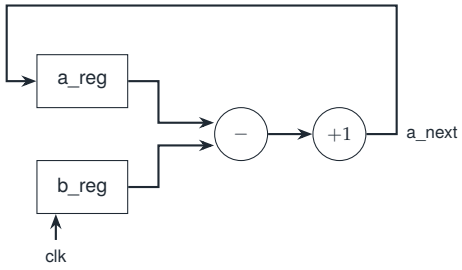


Fig. 6.1: $a \leftarrow a - b + 1$; a_reg changes only at the next edge

ASMD chart = ASM chart with RT operations written in the state boxes and conditional output boxes. The **default RT operation** $r \leftarrow r$ (keep value) is not drawn. In hardware, each destination register gets a **multiplexer** whose select is driven by the FSM state (Fig. 6.2): all RT operations in one ASMD block happen **in parallel** in the same clock cycle.

▲ The delayed store (§6.1.3): the most-tested subtlety of this chapter

An RT operation inside an ASMD block is only **stored when the FSMD leaves the block** (at the next edge). If a state box does `r <- r-1` and a decision box in the same block tests `r=0`, the test sees the **old** value of `r`. If you need the new value, test `r_next=0` (the combinational output), written with the "immediate" notation `r_next := r-1`.

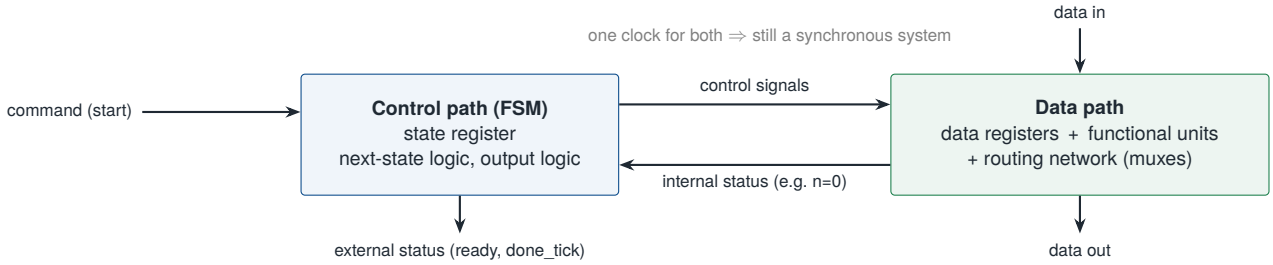


Figure 6.5: block diagram of an FSMD

6.2 Example that shows the method: FSMD debouncer [§6.2]

The Ch. 5 debouncer is *not* RT methodology: FSM and timer run independently, so the wait is 20–30 ms. Here the FSM **controls** the timer: on the 0→1 input change it loads a 21-bit register `q` with `"1...1"` and decrements it every cycle while `sw`=1. Wait time = $2^{21} \times 20$ ns $\approx$ **42 ms**, exact. Outputs: `db_level` (debounced level) and `db_tick` (one-cycle tick at the 0→1 transition, so no edge detector is needed after it).

Two ways to code an FSMD (§6.2.2–6.2.4)

Explicit data path: identify the components (here a custom 21-bit down-counter with `q_load`, `q_dec` and status `q_zero`) and code them separately; the FSM only produces control signals. **Implicit data path**: write the RT operations directly in the FSM's `case` (`q_next <= q_reg - 1;`); synthesis builds the data path. Simpler and follows the ASMD chart, **but less control**: e.g. three * in three states may give **three multipliers**; explicit code can **share one** multiplier through a mux. The two styles can be mixed.

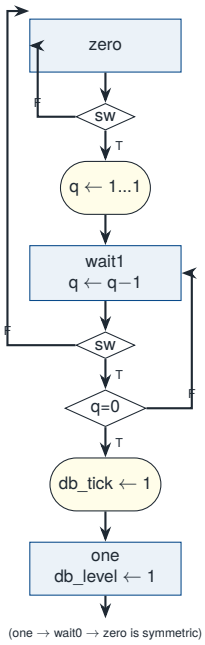


Fig. 6.6 (left half): ASMD chart of the FSMD debouncer

6.3 Design examples [§6.3]

All share one interface convention: input `start` (command), outputs `ready` (idle, can take a new input) and `done_tick` (one-cycle pulse when finished), because an FSMD is usually part of a bigger system. States are typically `idle`, `op` (iterate), `done`.

Fibonacci number [§6.3.1]

$fib(i) = 0,\ 1,\ fib(i-1) + fib(i-2)$ for $i = 0, 1, > 1$. Registers `t0`, `t1` (last two values) and index `n`.

- `idle`: `ready`=1; on `start`: `t0<-0`, `t1<-1`, `n<-i`, go to `op`.
- `op`: if `n=0`: `t1<-0`, go `done`; elsif `n=1`: go `done`; else `t1<-t1+t0`, `t0<-t1`, `n<-n-1`.
- `done`: `done_tick`=1, back to `idle`. Result `f = t1`.

Because all three RT operations in `op` are stored **simultaneously**, `t0` gets the *old* `t1`, exactly as required (no temporary needed). Output must be wide enough: fib grows fast (the book uses 20 bits for $i \leq 31$).

Core of Listing 6.4 (implicit data path)

```
when op =>
   if n_reg=0 then
      t1_next <= (others=>'0');
      state_next <= done;
   elsif n_reg=1 then
      state_next <= done;
   else
      t1_next <= t1_reg + t0_reg;
      t0_next <= t1_reg;
      n_next  <= n_reg - 1;
   end if;
when done =>
   done_tick  <= '1';
   state_next <= idle;
```

Division by the long-division algorithm [§6.3.2]

`/` cannot be synthesised, so build it. Registers: `d` (divisor), `rh:rl` (double-width dividend, `rh` starts at 0, `rl` = dividend). Each iteration: **shift `rh:rl` left** one bit (equivalent to shifting the divisor right); if `rh >= d` then `rh <- rh - d` and the new quotient bit (shifted into the LSB of `rl`) is 1, else 0. After N iterations **`rl` = quotient, `rh` = remainder**. States: `idle`, `op`, `last` (the last step must not shift the remainder), `done`. Compare-and-subtract is coded as a separate segment.

▷ 13 ÷ 4 with 4-bit operands

step	rh	rl	action
init	0000	1101	
1	0001	1010	1<4: q bit 0
2	0011	0100	3<4: q bit 0
3	0110→0010	1001	6≥4: sub, q bit 1
4	0101→0001	0011	5≥4: sub, q bit 1

Quotient `rl`=0011=3, remainder `rh`=1. ✓

Binary-to-BCD conversion ("shift and add 3") [§6.3.3]

A BCD shift register is split into 4-bit digits. Shifting left doubles the value; a digit > 9 after the shift needs correction (-10 and carry, i.e. $+6$). It is cheaper to **correct before shifting**:

1. For each BCD digit: **if digit > 4, add 3**.
2. Shift the whole BCD register left 1 bit, shifting in the next binary bit (MSB first).
3. Repeat for all input bits.

The book's circuit: 13-bit input, 4 BCD digits, an FSMD iterates 13 times with four separate adjustment circuits (combinational, one per digit).

▷ $243 = 1111\,0011_2$

after bit	hundreds	tens	ones	
7 (1)	0000	0000	0001	
6 (1)	0000	0000	0011	
5 (1)	0000	0000	0111	
4 (1)	0000	0001	0101 (7>4: +3=10)	Result
3 (0)	0000	0011	0000 (5>4: +3=8)	
2 (0)	0000	0110	0000	
1 (1)	0001	0010	0001 (6>4: +3=9)	
0 (1)	0010	0100	0011	
243. ✓				

Period counter and accurate low-frequency counter [§6.3.4–6.3.5]

Period counter: count clock cycles between two rising edges of the input (edge detector gives `edge` tick): period $= N \times \frac{1}{f}$. The book measures in **ms**: register `t` counts 0..49,999 (1 ms at 20 ns) and wraps; register `p` increments each time `t` wraps. States `idle`, `waite` (wait for first edge), `count`, `done`. Constant `CLK_MS_COUNT = 50000`. **Low-frequency counter** (1–10 Hz): counting pulses for 1 s is inaccurate at low frequency (cannot tell 2.123 from 2.567 Hz). Instead **measure the period and take the reciprocal**: $f = 10^6/p_{[\mu s]}$ or $1000/p_{[ms]}$. Built from three existing FSMDs (period counter → divider → bin-to-BCD) sequenced by a **master FSM** using their `start/done_tick` handshakes. Shows **hierarchical design by reuse**.

⋆ Exam practice

1. Draw the ASMD chart of the Fibonacci circuit and explain why `t0 <- t1` gets the old value of `t1`.
2. An ASMD state box contains `c <- c+1` and the decision box below tests `c=10`. What does the circuit actually do? How do you fix it?
3. Convert 157_{10} = `1001 1101` to BCD using shift-and-add-3. (*Answer: 0001 0101 0111.*)
4. Divide $11 \div 3$ with the rh:rl algorithm; show each step.
5. Explicit vs. implicit data path: give one advantage of each, with the multiplier-sharing example.
6. The debounce FSMD should wait 20 ms at 50 MHz. How many bits does `q` need? ($20\,ms/20\,ns = 10^6$ *cycles* $\Rightarrow$ *20 bits.*)

Part II

I/O modules

UART, PS2 keyboard & mouse, external SRAM, block RAM, VGA graphics & text

07 UART

Serial frame, 16× oversampling, baud-rate generator, receiver and transmitter FSMDs, interface buffering, customisation and error checks

Book Chapter 7, pp. 163–182

7.1 Serial transmission [§7.1]

A **UART** (universal asynchronous receiver and transmitter) sends parallel data over a serial line. Used with **RS-232** (EIA standard for electrical, mechanical, functional and procedural characteristics); the S3 board has the voltage-converter chip, so we only design the UART.

- Transmitter = special **shift register**: load parallel, shift out bit by bit at a fixed rate. Receiver: shift in, reassemble.
- Line is **1 when idle**. Frame: **start bit 0**, 6/7/8 data bits (**LSB first**), optional parity, **stop bit(s) 1** (1, 1.5 or 2).
- Parity: **odd** parity bit = 0 when data has an odd number of 1s; **even** parity bit = 0 when data has an even number of 1s.
- **No clock is sent**: both sides must agree on baud rate (2400, 4800, 9600, 19200…), data bits, stop bits and parity in advance.

The book's design: **19,200 baud, 8 data bits, 1 stop bit, no parity** ("8N1").

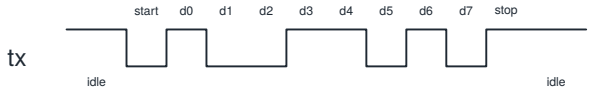


Fig. 7.1 style: one byte in 8N1. Bits d0..d7 = 1,0,0,1,1,0,1,0 sent LSB first, so the byte is `0101 1001` = `0x59` ('Y').

Oversampling procedure (16×) §7.2.1

Sampling rate = **16** × **baud rate**. With N data and M stop bits:

1. Wait until the line goes **0** (start bit); start the tick counter.
2. When the counter reaches **7**: middle of the start bit. Clear counter.
3. When it reaches **15**: middle of the next bit. Sample it, shift into the register, clear counter.
4. Repeat step 3 $N-1$ more times (data), once for parity if used, and M times for the stop bits.

The ticks play the role of a clock: they estimate the **middle of each bit**, off by at most 1/16 of a bit. Price: baud rate must be a small fraction of the system clock.

7.2 Receiving subsystem [§7.2]

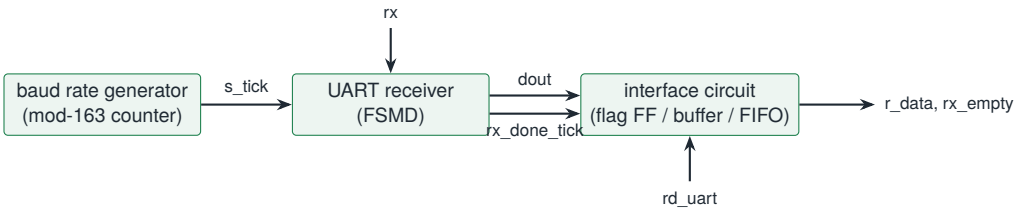


Figure 7.2: receiving subsystem

▷ Baud rate generator

Ticks at 16 × 19,200 = 307,200 per second. From 50 MHz:

$$M = \frac{50 \times 10^6}{16 \times 19{,}200} = 162.8 \approx \mathbf{163}$$

so a **mod-163 counter** (8 bits) with its `max_tick` as `s_tick`. It is an **enable tick**, not a new clock (no new clock domain). Error ≈ 0.15%, far below the tolerance of ±1/16 bit accumulated over a frame.

Receiver ASMD (Fig. 7.3): states `idle`, `start`, `data`, `stop`. Registers: `s` (counts sampling ticks: to 7 in `start`, 15 in `data`, `SB_TICK-1` in `stop`), `n` (number of data bits received), `b` (reassembled byte). The FSMD only advances when `s_tick`=1. Constants `DBIT`=8 and `SB_TICK`=16/24/32 for 1/1.5/2 stop bits. Output `rx_done_tick` for one cycle at the end.

Listing 7.1: UART receiver (next-state logic)

```
case state_reg is
 when idle =>
   if rx='0' then
     state_next <= start;
     s_next <= (others=>'0');
   end if;
 when start =>
   if (s_tick='1') then
     if s_reg=7 then
       state_next <= data;

       s_next <= (others=>'0');
       n_next <= (others=>'0');
     else
       s_next <= s_reg + 1;
     end if;
   end if;
 when data =>
   if (s_tick='1') then
     if s_reg=15 then
       s_next <= (others=>'0');
       b_next <= rx & b_reg(7 downto 1); -- LSB first
       if n_reg=(DBIT-1) then
         state_next <= stop;
       else
         n_next <= n_reg + 1;
       end if;
     else
       s_next <= s_reg + 1;
     end if;
   end if;
 when stop =>
   if (s_tick='1') then
     if s_reg=(SB_TICK-1) then
       state_next <= idle;
       rx_done_tick <= '1';
     else
       s_next <= s_reg + 1;
     end if;
   end if;
end case;
```

7.3 Interface circuit [§7.2.4]

The main system polls the UART. The interface must (1) signal that a new word is available and prevent it being read twice, (2) give buffer space.

Scheme	How	Weakness
Flag FF	`rx_done_tick` sets the flag (`set_flag`); the main system reads data directly from the receiver's shift register and pulses `clr_flag`. `rx_empty` = not flag.	No extra buffer: a new frame overwrites the old word before it is read = **data overrun**.
Flag FF + one-word buffer	On `rx_done_tick` the word is copied to a buffer register and the flag is set.	Overrun only if the next word arrives before the old one is read.
FIFO (Ch. 4.5.3)	`rx_done_tick` → FIFO `wr`; main system reads the head and pulses `rd`; `empty` = no data.	Overrun only when the FIFO is full; size adjustable.

7.4 Transmitter and complete UART [§7.3–7.4]

- The transmitter is a shift register driven by the **same baud generator**; it shifts out one bit every **16 ticks** (internal counter), so no new counter is needed.
- ASMD like the receiver: `idle` → (`tx_start`, load data) `start` → `data` → `stop`; `tx_done_tick` at the end. A 1-bit register `tx_reg` drives the line to **filter glitches**.
- On the transmit side, the *main system* sets the flag / writes the FIFO and the *transmitter* clears it / reads it.
- **Complete UART** (Fig. 7.5): baud generator + receiver + transmitter + two FIFOs. Ports: `rx`, `tx`, `rd_uart`, `wr_uart`, `w_data`, `r_data`, `rx_empty`, `tx_full`.

Verification (loop-back) §7.4.2

PC HyperTerminal (19200, 8 data, 1 stop, no parity, no flow control) → S3 board. Received characters wait in the 4-word receive FIFO; a debounced button tick reads one word, **adds 1**, and writes it to the transmit FIFO. Type "HAL", press the button 3 times, and "IBM" comes back. `r_data` is shown on the LEDs; `tx_full`/`rx_empty` on the 7-segment display.

Xilinx-specific

PicoBlaze's download package includes a UART built from low-level Xilinx primitives: more efficient but not portable.

7.5 Customising and error checking [§7.5]

- **Baud rate**: change the `M` generic of the baud generator.
- **Data bits**: `DBIT` (upper limit of `n`).
- **Parity**: add a `parity` state between `data` and `stop`.
- **Stop bits**: `SB_TICK` = 16, 24, 32.

Errors the receiver can detect: **parity error** (parity mismatch); **frame error** (stop bit sampled as 0); **buffer overrun** (new word ready while flag still set / FIFO full at `rx_done_tick`).

ASCII tips: 7-bit codes. `0x30`–`0x39` = digits 0–9 (lower hex digit = value); upper/lower case letters differ in **one bit** (`0x20`): `A`=`0x41`, `a`=`0x61`. CR = `0x0D`, LF = `0x0A`.

⋆ Exam practice

1. Draw the waveform for sending `0x4B` ("K") in 8N1 and mark where the receiver samples each bit.
2. Compute the baud-generator divisor for 9600 baud with a 50 MHz clock and 16× oversampling. (*325.5 ⇒ 326.*)
3. Why does the receiver count to 7 in the start state but to 15 in the data state?
4. Compare the three receiver interface schemes with respect to data overrun.
5. Modify the receiver ASMD to check the stop bit and output a `frame_err` signal.

08 PS2 keyboard and mouse

PS2 packet and timing, filtered falling-edge receiver, scan codes, last-released-key interface, host-to-device transmission, bidirectional interface and the mouse protocol

Book Chapters 8 and 9, pp. 185–222

8.1 PS2 port and device-to-host protocol [§8.1–8.2]

PS2 (from IBM Personal System/2): two signal wires, **`ps2d`** (serial data) and **`ps2c`** (clock), plus Vcc and ground supplied by the host. Lines are **open-collector**; modern devices work with 3.3 V (the S3 board has a 5 V jumper for older ones).

- **11-bit packet**: start bit (0), **8 data bits (LSB first)**, **odd parity**, stop bit (1). Same format for keyboard and mouse; the meaning of the data differs.
- Unlike a UART, the **clock is sent** on `ps2c` by the device: data is valid at the **falling edge** of `ps2c`.
- `ps2c` period **60–100 μs** (10–16.7 kHz); `ps2d` stable at least 5 μs before and after the falling edge.

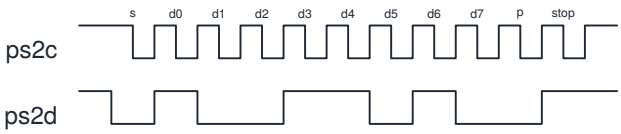


Fig. 8.1 style: data read on each falling edge of ps2c

Receiver design (§8.2.3)

Like a UART receiver but **the falling edge of `ps2c` replaces oversampling**. Because `ps2c` is slow and may be noisy, first **filter** it: an 8-bit shift register samples `ps2c` each system clock; output becomes 1 (0) only after **eight consecutive 1s (0s)**, so glitches shorter than 8 cycles are ignored. Then a normal edge detector gives `fall_edge`. ASMD: `idle` → (falling edge and `rx_en`) `dps` (shift in the remaining 10 bits in one state, counter `n`) → `load` (one extra cycle to finish the shift, assert `rx_done_tick`). No error checking; a robust design checks start/parity/stop bits and adds a **watchdog timer**.

8.2 Keyboard scan codes [§8.3]

The keyboard's embedded microcontroller scans the key matrix and sends codes:

- Key pressed → **make code** (usually 1 byte, e.g. **A = `1C`**; extended keys have 2–4 bytes starting with `E0`, e.g. up-arrow `E0 75`).
- Key held (**typematic**) → make code repeated about every **100 ms** after **0.5 s**.
- Key released → **break code** = **`F0`** + make code (A: `F0 1C`).

There is no separate upper-case code: the **host** must track the shift key.

Action	Byte sequence
press + release A	`1C F0 1C`
hold A, release	`1C 1C ... 1C F0 1C`
Shift+A (upper case)	`12 1C F0 1C F0 12`

Last-released-key circuit (§8.4, Fig. 8.4): PS2 receiver + 4-word FIFO + small FSM. State `wait_brk`: watch packets; when `F0` arrives go to `get_code`; the next packet is the make code of the released key → write it into the FIFO (`got_code_tick`) and return. Assumes one normal key at a time; does not work for extended keys. **Verification**: make code → ASCII (`key2ascii` ROM/case) → UART → HyperTerminal.

8.3 Mouse protocol [§9.2]

A PS2 mouse reports **relative movement** and button status in **3 bytes (3 packets)**, byte 1 first:

bit	7	6	5	4	3	2	1	0
byte 1	y_v	x_v	y_8	x_8	1	m	r	l
byte 2				$x_7 \dots x_0$				
byte 3				$y_7 \dots y_0$				

x, y: **9-bit two's complement** (positive = right/up); x_v, y_v: overflow; l/r/m: buttons. Defaults: resolution **4 counts/mm**, sample rate **100 samples/s**. In **stream mode** the mouse sends data whenever there is movement or button activity.

Initialisation: why the mouse needs bidirectional PS2

After power-up the mouse is **not** streaming. Sequence:

1. Power-on self test: mouse sends `AA` (test passed) then `00` (mouse ID). Usually missed because the FPGA is not configured yet.
2. Host sends **`F4`** (enable stream mode); mouse sends an acknowledge byte. The book writes **`FE`**; in the standard PS2 protocol the acknowledge is `FA` and `FE` means "resend". Use the book's value in the exam unless your lecturer says otherwise.
3. Mouse streams 3-byte packets.

Reset command: `FF` → acknowledge, self test, `AA 00`; stream mode disabled.

8.4 Host-to-device transmission [§9.3]

1. Host pulls `ps2c` low for **at least 100 μs** (inhibits the device; "request to send").
2. Host pulls `ps2d` low (start bit) and **releases** `ps2c` (high-Z).
3. The device now generates the clock; on each falling edge of `ps2c` the host puts the next bit (8 data LSB first, then parity) on `ps2d` and holds it until the next falling edge.
4. Host releases `ps2d`; the device **acknowledges** by pulling `ps2d` low at the last clock.

Mouse interface (§9.5)

An FSMD on top of the bidirectional PS2 circuit, 7 states: `init1` (send `F4`), `init2` (wait for transmission done), `init3` (wait for the acknowledge packet), then `pack1`, `pack2`, `pack3` (assemble 3 bytes) and `done` (`m_done_tick`), looping over the pack states. Outputs: `xm`, `ym` (9-bit), `btnm` (3-bit), `m_done_tick`.

- Both lines need **tri-state buffers**: `ps2c <= ps2c_out when tri_c='1' else 'Z';` (same for `ps2d`).
- Transmitter ASMD: `idle` → (`wr_ps2`: load `par & din` into a shift register, load 13-bit counter) `rts` (`ps2c`=0 for ≈164 μs) → `start` → `data` (9 bits, on filtered falling edges) → `stop` → `idle`. Output `tx_idle`.
- **Bidirectional interface**: `tx_idle` drives the receiver's `rx_en`, so **transmission has priority**; the receiver is only enabled when the transmitter is idle.

Test circuit (§9.5.2): a 10-bit counter tracks the x position (adds the sign-extended `xm` on every `m_done_tick`; left/right button sets it to 0/max); its 3 MSBs light one of 8 LEDs. **Keyboard LEDs** can also be controlled by the host: send `ED` then `0X` with X = `0snc` (scroll, num, caps).

⋆ Exam practice

1. Compare the PS2 and UART frames: who provides the clock, parity, bit order, when data is sampled.
2. Why is `ps2c` filtered before edge detection, and how many system-clock cycles must a glitch last to get through?
3. Write the byte sequence for typing capital "AB" (Shift held) on a PS2 keyboard. (*A=1C, B=32, Shift=12.*)
4. Describe the host-to-device protocol and why tri-state buffers are needed.
5. A mouse packet is `28 05 FE`. Decode dx, dy and the buttons. (*byte1 = 0010 1000: y_8=1, bit 3=1, no buttons; dx=+5, dy=−2.*)

10 External SRAM and memory controller

Asynchronous SRAM signals and timing, the controller as a synchronous wrapper, a safe two-cycle design, faster alternatives and their hazards, DCM/IOB/DDR tricks

Book Chapter 10, pp. 215–244

10.1 The IS61LV25616AL SRAM [§10.1–10.2]

RAM is used for large storage because a RAM cell is much smaller than a FF. An **asynchronous SRAM** has no clock: address, data and control must be applied in a specific order and held stable for given times. A synchronous system therefore talks to it through a **memory controller**, which takes commands synchronously and generates properly timed signals, making memory look synchronous. Performance = accesses completed per unit time.

The S3 board: **two** 256**K** × 16 **chips** (1 MB total). Signals: **ad** (18-bit address), **dio** (16-bit *bidirectional* data), and five **active-low** controls: `ce_n` (chip enable), `we_n` (write), `oe_n` (output enable), `lb_n`/`ub_n` (lower/upper byte). The book uses one chip in 16-bit mode, so `ce_n`, `lb_n`, `ub_n` are always 0.

Operation	we_n	oe_n	dio
output disabled	1	1	Z
read 16-bit word	1	0	data out
write 16-bit word	0	–	data in

Fig. 10.1(c): simplified functional table

Param.	Meaning	ns
t_{RC}	read cycle time	min 10
t_{AA}	address access time	max 10
t_{OHA}	output hold after addr change	min 2
t_{DOE}	`oe_n` access time	max 4
t_{HZOE}	`oe_n` off → high-Z	max 4
t_{LZOE}	`oe_n` on → low-Z	min 0
t_{WC}	write cycle time	min 10
t_{SA} / t_{HA}	address setup / hold	0 / 0
t_{PWE1}	`we_n` pulse width	min 8
t_{SD} / t_{HD}	data setup / hold (to `we_n`↑)	6 / 0

Figs. 10.2–10.3: key timing values

Read and write sequences §10.3.2

Read (`we_n`=1): place address and activate `oe_n`; wait at least t_{AA}; data is valid on `dio`; deactivate `oe_n`. **Write** (`we_n`-controlled): place address and data, activate `we_n`; wait at least t_{PWE1}; **deactivate `we_n`**: data is latched on its **0→1 edge**; then remove data. Although $t_{HD} = 0$, you must make sure `we_n` goes high *before* the data disappears (delay variations).

10.2 Controller structure [§10.3]

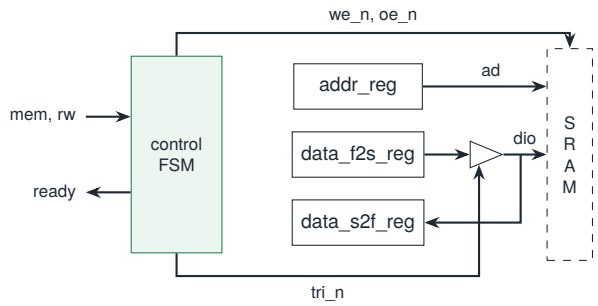


Fig. 10.5 (simplified): data path = address register, two data registers, tri-state buffer; control path = FSM

Main-system side: `mem` (start an operation), `rw` (1 = read, 0 = write), `addr` (18 bit), `data_f2s` (FPGA-to-SRAM), `data_s2f_r` (registered read data), `data_s2f_ur` (unregistered, directly from `dio`, available one cycle earlier), `ready` (can accept a command; operations take several cycles).

Register file vs. SRAM (§10.3.3): a register file has several read ports, reads and writes at the same time, writes in one cycle, and read data needs no clock or control. It is faster and more flexible, but FFs are big, so it only suits small storage.

10.3 The safe design [§10.4]

ASMD (Fig. 10.6): five states `idle`, `rd1`, `rd2`, `wr1`, `wr2`.

- `idle`: `ready`=1; on `mem`=1 sample `addr` (and data for write) into registers; go to `rd1` or `wr1`.
- `rd1`, `rd2`: `oe_n`=0 (40 ns $\gg t_{AA}$ = 10 ns). Leaving `rd2`, store `dio` in `data_s2f_reg`.
- `wr1`: `we_n`=0 and `tri_n`=0 (drive `dio`); 20 ns > t_{PWE1} = 8 ns.
- `wr2`: `we_n`=1 but **`tri_n` still 0**: data is still on the bus at the `we_n` 0→1 edge.

Performance: access = 2 cycles (40 ns); **back-to-back = 3 cycles (60 ns)** because it returns to `idle`.

Glitch-free fast outputs: look-ahead output buffer

Control signals to the SRAM must be glitch-free with small clock-to-output delay. Put a **D FF on each Moore output**; to avoid the one-cycle delay, compute the buffered output from **`state_next`** instead of `state_reg` ("look ahead"). Tri-state buffers are needed on the bidirectional `dio`; the output registers should be placed in the **IOBs** (with suitable slew rate / drive) to minimise pad delay.

Testing: (1) a basic circuit with buttons to load data, write and read one address; (2) a comprehensive tester that writes all 2^{18} locations with a pattern (reversed address bits), reads them back at full rate, counts mismatches, and can inject errors with the switches.

10.4 More aggressive designs and why they are risky [§10.5]

Design	Change	Access / back-to-back	Hazard
Safe (§10.4)	2 states per op, return to idle	2 / 3 cycles	large margins, none
Alternative I	check `mem` in `rd2`/`wr2`, skip idle	2 / 2	read→write turnaround: SRAM and FPGA tri-state buffers switch at the same time ⇒ **bus fighting** (both drive `dio` briefly; large transient current)
Alternative II	one state per op (drop `rd2`, `wr2`)	1 / 2	read: t_{AA} + **two pad delays** (4 to >10 ns each) in 20 ns; write: `we_n` and data removed simultaneously, not guaranteed that `we_n` rises first
Alternative III	I + II combined	1 / 1	all of the above; `we_n` must be low only for **part of a clock cycle** (a Moore output would stay low during back-to-back writes, so no rising edge to latch)

▲ Limitation of FSM control

An FSM cannot produce a control sequence **finer than its clock period**; the aggressive designs depend on delays that RT-level HDL cannot specify, so synthesis results are not reliable. Gating `we_n` with the clock is a bad idea (glitches, delay variation).

Device features that help (§10.5.5)

DCM: multiply/divide/phase-shift the clock; e.g. run the controller at 200 MHz (5 ns) and use more, shorter states. **IOB**: output FFs in the pad, programmable slew rate and drive. **DDR register** + DCM `clk180`: create a clean **half-cycle `we_n`** pulse (load the pulse on `clk`, load '1' on `clk180`).

⋆ Exam practice

1. Draw the ASMD chart of the safe SRAM controller and show that it satisfies t_{AA} and t_{PWE1} at 50 MHz.
2. Why does the safe controller keep `tri_n` active in `wr2`?
3. What is "fighting" on the data bus and when can it happen?
4. Why is a look-ahead output buffer used, and how does it avoid the extra clock cycle?
5. How would you get a 10 ns `we_n` pulse inside a 20 ns clock cycle reliably?

11 Embedded memory: block and distributed RAM

Spartan-3 memory options, three ways to use them, and the XST inference templates for RAM and ROM

Book Chapter 11, pp. 245–260

11.1 Storage options on the S3 board [§11.2]

- **Distributed RAM**: built from LC look-up tables; each LUT = 16 × 1 synchronous RAM; cascade for wider/deeper. Competes with logic. Only for small memories. **Asynchronous read possible.**
- **Block RAM**: dedicated blocks separate from the logic; a fast SRAM with a synchronous, configurable interface. **16K data bits + 2K parity bits** each, organised from 16K × 1 to 512 × 32. **Synchronous read only.**
- Both are already "wrapped" synchronously: **no memory controller needed**; single- or dual-port.

Storage (XC3S200 / S3 board)	Capacity
FFs (registers)	≈4.5K bits
Distributed RAM	30K bits
Block RAM (12 × 16K)	192K bits*
External SRAM (2 × 256K×16)	8M bits

*The book quotes 172K in §11.2 and 192K in §12.5; with parity bits it is 216K (Table 2.1).

Use this ladder to pick: registers for a few words, distributed RAM for small tables, block RAM for FIFOs / look-up tables / video tile memory, external SRAM for full-screen frame buffers.

11.2 Three ways to include a memory module [§11.3]

Method	How	Pros / cons
HDL component instantiation	instantiate the Xilinx primitive (e.g. `RAMB16_S1_S1`) with generics; code from ISE *Edit → Language Templates*	full control; tedious, error-prone, Xilinx-only
Core Generator (Coregen)	wizard generates `.xco` + a `.vhd` wrapper for simulation	convenient; outside the HDL framework, compatibility issues
Behavioural HDL inference (used in the book)	follow the XST templates exactly; synthesis recognises the pattern	readable, simulates without libraries, "semi-portable"; **check the synthesis report** that the right module was inferred

11.3 Inference templates [§11.4]

Write is always synchronous. **Read** can be:

- **asynchronous**: address used directly; data after a short delay ⇒ only **distributed RAM**;
- **synchronous**: address **registered** at the clock edge, then used; data one cycle later ⇒ can use **block RAM**.

Listing 11.2: single-port RAM, synchronous read

```
architecture beh_arch of xilinx_one_port_ram_sync is
  type ram_type is array (0 to 2**ADDR_WIDTH-1)
     of std_logic_vector(DATA_WIDTH-1 downto 0);
  signal ram: ram_type;
  signal addr_reg: std_logic_vector(ADDR_WIDTH-1 downto
     0);
begin
  process(clk)
  begin
    if (clk'event and clk='1') then
      if (we='1') then
        ram(to_integer(unsigned(addr))) <= din;
      end if;
      addr_reg <= addr;
    end if;
  end process;
  dout <= ram(to_integer(unsigned(addr_reg)));
end beh_arch;
```

Asynchronous-read version: no `addr_reg; dout <= ram(to_integer(unsigned(addr)));` **Dual-port RAM**

(book version): port A read/write, port B **read only** (enough for video memory: one write port, one read port). Two addresses; with synchronous read both are registered.

Dual port, sync read (Listing 11.4, core)

```
if (clk'event and clk='1') then
  if (we='1') then
    ram(to_integer(unsigned(addr_a))) <= din_a;
  end if;
  addr_a_reg <= addr_a;
  addr_b_reg <= addr_b;
end if;
...
dout_a <= ram(to_integer(unsigned(addr_a_reg)));
dout_b <= ram(to_integer(unsigned(addr_b_reg)));
```

ROM: a combinational circuit; Spartan-3 has no real ROM. Two templates:

- **Asynchronous ROM** = constant array indexed by the address ⇒ logic cells (another form of `case`); for small tables. Example: hex-to-7-segment table `HEX2LED_ROM`.
- **Synchronous ROM** = RAM template without write, with registered address ⇒ **block RAM**; the clock is "artificial", added just to infer block RAM; for large tables (font ROM).

▷ Synthesis report check

A 4K × 8 single-port RAM with synchronous read ($2^{12} \times 2^3 = 32$K bits) should be reported by XST as a block RAM and use **two** 16K-bit block RAMs.

⋆ Exam practice

1. Why can a RAM with asynchronous read not be mapped to block RAM?
2. Write the VHDL for a 256 × 8 synchronous ROM and explain why a clock is added.
3. Rebuild the FIFO of §4.5.3 with block RAM. What changes in its behaviour? (*Read data appears one cycle later.*)
4. A sine table: 10-bit input, 8-bit output. How can symmetry reduce it to a $2^8 \times 7$ ROM? [§11.6.4]

12 VGA controller I: synchronisation and graphics

CRT scanning, hsync/vsync timing for 640×480, the vga_sync circuit, bit-, tile- and object-mapped pixel generation, animated pong objects

Book Chapter 12, pp. 257–290

12.1 CRT basics and the S3 VGA port [§12.1]

An electron gun sends a beam that hits phosphor dots; brightness follows the analog video voltage (0–0.7 V). Deflection coils, driven by **sawtooth** waveforms, scan the screen **left to right, top to bottom**; the beam then **retraces**. The digital **`hsync`** and **`vsync`** signals control the sawtooth generators. A colour CRT has three beams (R, G, B) per pixel.

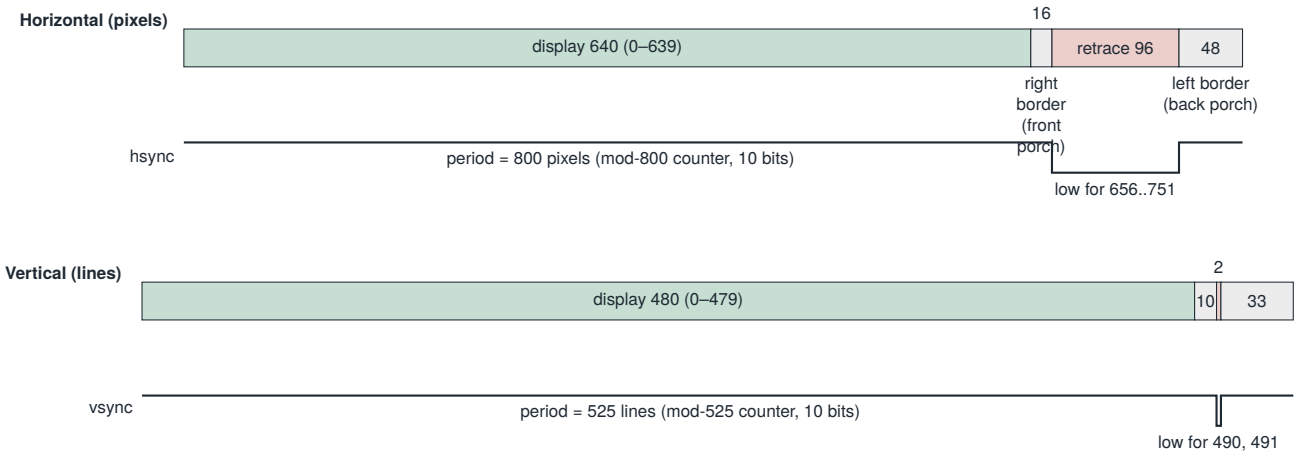
VGA port: 5 active signals: `hsync`, `vsync`, `red`, `green`, `blue` (15-pin D-sub). With N bits per colour a DAC gives 2^N levels and 2^{3N} colours ("$3N$-bit colour"). The S3 board uses **1 bit per colour ⇒ 8 colours**.

R	G	B	colour
0	0	0	black
0	0	1	blue
0	1	0	green
0	1	1	cyan
1	0	0	red
1	0	1	magenta
1	1	0	yellow
1	1	1	white

Table 12.1

Video controller (Fig. 12.3) = **`vga_sync`** (generates `hsync`, `vsync`, `video_on`, `p_tick` and the current coordinates `pixel_x`, `pixel_y`) + **pixel generation circuit** (computes the 3-bit `rgb` for the current pixel from coordinates + external data).

12.2 Synchronisation timing, 640×480 at 25 MHz [§12.2]



- Count from the **start of the display region**, so the counter value *is* the coordinate: `pixel_x` (horizontal counter), `pixel_y` (vertical counter). The y-axis grows **downwards**; (0,0) is top-left, (639,479) bottom-right.
- `h_video_on` when $x < 640$; `v_video_on` when $y < 480$; `video_on` = both. Video must be **black** in borders and retrace.
- Border widths vary between monitor brands (adjust to centre the picture).

▷ Pixel rate

$p = 800$ pixels/line, $l = 525$ lines/screen, $s = 60$ screens/s (at least 30 for motion, ≈60 to avoid flicker):

$$\text{pixel rate} = p \cdot l \cdot s = 800 \cdot 525 \cdot 60$$

$$\approx 25.2\,\text{MHz} \approx 25\,\text{MHz}$$

Line time $= 800/25\text{M} = 32\,\mu\text{s}$; frame $= 525 \times 32\,\mu\text{s} = 16.8\,\text{ms}$.

vga_sync essentials (Listing 12.1)

```
constant HD: integer:=640; -- h. display area
constant HF: integer:=16;  -- h. front porch
constant HB: integer:=48;  -- h. back porch
constant HR: integer:=96;  -- h. retrace
constant VD: integer:=480; -- v. display area
constant VF: integer:=10;  -- v. front porch
constant VB: integer:=33;  -- v. back porch
constant VR: integer:=2;   -- v. retrace
...
-- mod-2 counter: 25 MHz tick from 50 MHz
mod2_next  <= not mod2_reg;
pixel_tick <= '1' when mod2_reg='1' else '0';
-- end of line (799) / end of frame (524)
h_end <= '1' when h_count_reg=(HD+HF+HB+HR-1)
         else '0';
v_end <= '1' when v_count_reg=(VD+VF+VB+VR-1)
         else '0';
-- horizontal counter (mod-800)
process(h_count_reg, h_end, pixel_tick)
begin
  if pixel_tick='1' then
    if h_end='1' then
      h_count_next <= (others=>'0');
    else
      h_count_next <= h_count_reg + 1;
    end if;
  else
    h_count_next <= h_count_reg;
  end if;
end process;
-- vertical counter (mod-525): counts when
-- pixel_tick='1' and h_end='1'
...
-- sync during retrace: 656..751 and 490..491
h_sync_next <= '1' when (h_count_reg>=(HD+HF))
                    and (h_count_reg<=(HD+HF+HR-1))
               else '0';
v_sync_next <= '1' when (v_count_reg>=(VD+VF))
                    and (v_count_reg<=(VD+VF+VR-1))
               else '0';
video_on <= '1' when (h_count_reg<HD)
                and (v_count_reg<VD) else '0';
```

Design points

- System clock 50 MHz = 2× pixel rate: use a **25 MHz enable tick** (mod-2 counter), not a second clock domain. The tick is exported as `p_tick`.
- `h_end`, `v_end` status signals end a line / frame.
- **Output buffers** on `hsync/vsync` remove glitches (1-cycle delay); buffer `rgb` the same way in the top level (`rgb_reg` loaded on `pixel_tick`) so all stay aligned.
- **Polarity**: the text (and the 640×480 standard) says sync is *low* during retrace, but Listing 12.1 drives '1' there; monitors generally accept both. In the exam, draw it low as in Fig. 12.4 and know the region numbers.
- **Test**: connect `rgb` to 3 switches: the whole visible area shows one colour.

12.3 Pixel generation schemes [§12.3]

Scheme	Idea	Memory for 640×480
Bit-mapped	video memory with one word per pixel; address from (`pixel_x`, `pixel_y`); a graphics circuit writes it, a retrieval circuit reads it continuously. Used in modern high-performance controllers.	≈310K pixels: **310K bits** mono, **930K bits** for 3-bit colour. Too big for block RAM ⇒ external SRAM
Tile-mapped	group pixels into tiles (e.g. 8×8); store a tile *code* per tile in **tile memory**, tile patterns in a **pattern ROM**. Text displays (Ch. 13) use it.	$80 \times 60 = 4800$ tiles; 32 patterns ⇒ 5-bit words: 24K bits + $8 \cdot 8 \cdot 32 = 2$K pattern bits ≈ **26K bits**
Object-mapped	only a few objects: each has an object-generation circuit that compares the scan coordinates with its region; no video memory.	none (just registers for positions)

The schemes can be **mixed** (bit-mapped background + object-mapped sprites, or different schemes in different screen areas).

12.4 Object-mapped graphics: the pong screen [§12.4]

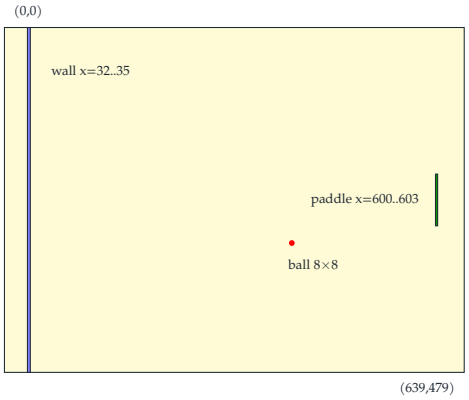


Fig. 12.7: still screen (wall, paddle 72 pixels high, ball)

Object generation circuit: keep the object's coordinates; assert `obj_on` when the scan position is inside its region; supply `obj_rgb`. An **rgb mux** picks the output by priority (= which object is in front); background otherwise (yellow in the book); black when `video_on`=0.

Object on-signals and rgb multiplexing

```
wall_on <= '1' when (WALL_X_L<=pix_x) and (pix_x<=
     WALL_X_R)
          else '0';
bar_on  <= '1' when (BAR_X_L<=pix_x) and (pix_x<=BAR_X_R)
      and
                    (bar_y_t<=pix_y) and (pix_y<=bar_y_b)
          else '0';
process(video_on, wall_on, bar_on, rd_ball_on, ...)
begin
  if video_on='0' then
    graph_rgb <= "000";              -- blank
  else
    if wall_on='1' then    graph_rgb <= wall_rgb;
    elsif bar_on='1' then  graph_rgb <= bar_rgb;
    elsif rd_ball_on='1' then graph_rgb <= ball_rgb;
    else graph_rgb <= "110";         -- yellow background
    end if;
  end if;
end process;
```

Non-rectangular object: round ball from a bit map

Store an 8×8 circle pattern in a small ROM (constant array). If the scan is inside the ball's square: `rom_addr = pix_y(2:0) - ball_y_t(2:0)` (row), `rom_col = pix_x(2:0) - ball_x_l(2:0)` (column); `rd_ball_on = sq_ball_on and rom_bit`.

Animation

Store object boundaries in **registers** and update them **once per frame** using a 60 Hz tick `refr_tick` (asserted when `pix_y`=481, `pix_x`=0, i.e. during vertical retrace). Paddle: `bar_y_reg` ± `BAR_V` while a button is pressed, stopped at screen edges. Ball: position registers + velocity registers `x_delta_reg`, `y_delta_reg` = `BALL_V_P` or `BALL_V_N`; reverse the component on hitting the wall, top/bottom or pad-

dle.

12.5 Bit-mapped graphics: dot trace [§12.5]

A 128 × 128 area (16K pixels × 3 bits) fits in **3 block RAMs**. **Address** = concatenation `y(6:0) & x(6:0)` (no address arithmetic; concatenating full 10-bit x and 9-bit y leaves unused "holes"). A `dot_xy` circuit moves a one-pixel dot like the ball and writes the switch colour at its position; the screen shows its trace.

- **Dual-port RAM**: write port = dot position, read port = current scan position (reads continuously for display).
- **Single-port RAM**: must not disturb display reads. Write only during **vertical retrace** (use `refr_tick` as `we`). Easy here because only 60×3 bits/s are written; hard when a large bandwidth is needed.

⋆ Exam practice

1. Draw the hsync and vsync timing with all four regions and the counter values where sync is low.
2. Compute the pixel rate for 800×600 at 72 Hz with totals 1040×666. (≈*49.9 MHz, i.e. SVGA 50 MHz.*)
3. How much memory does a full-screen 640×480 bit-mapped 8-colour display need, and where can it live on the S3 board?
4. Write the VHDL for a paddle object that moves up/down with two buttons, updated once per frame.
5. Compare bit-, tile- and object-mapped pixel generation.

13 VGA controller II: text and the complete pong game

Characters as tiles, the font ROM, two-stage text generation, font scaling, full-screen text with tile memory and pipeline delays, and the top-level pong controller

Book Chapter 13, pp. 291–320

13.1 Characters as tiles and the font ROM [§13.2]

A tile is a "super pixel": its value is the **code of a pattern**, not a colour. For text, the tile code is the **7-bit ASCII** code and the patterns are the **font**.

- Font: **8 columns** × **16 rows** per character (like the early IBM PC), 128 characters (ASCII `00`–`7F`; codes `00`–`1F` hold special graphic symbols).
- **Font ROM** = $2^7 \times 2^4$ rows × 8 bits = $2^{11} \times 8$ = 16K bits = **one block RAM** (use the synchronous ROM template, §11.4.3).
- ROM address (11 bits) = `char_addr` (7 bits, which character) & `row_addr` (4 bits, which row).
- Screen = 640/8 × 480/16 = **80×30 tiles**.

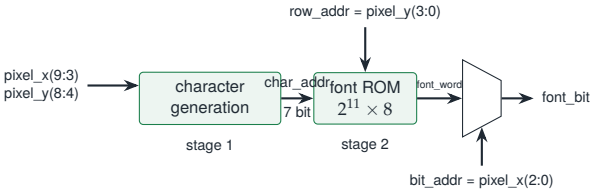


Fig. 13.2: two-stage text generation

Stage 1: upper coordinate bits → which tile → its ASCII code. **Stage 2**: code + lower bits → ROM row → 8-to-1 mux selects the pixel. Font rows are stored `(7 downto 0)` but x grows left to right, so the bit index must be **reversed** (`font_word(not bit_addr)`).

Timing subtlety of the block-RAM ROM

The synchronous font ROM delays its output by **one clock**. Since `pixel_x` only changes every **two** clocks (25 MHz tick at 50 MHz), the right bit is still selected: no fix needed for the simple font display.

Font display circuit (§13.2.4): `char_addr = pixel_y(5:4) & pixel_x(7:3)` shows all 128 patterns as 4 rows of 32, repeated; restrict to the top-left corner.

Font scaling (§13.2.5)

Enlarge each pixel by **dropping LSBs** of the coordinates: for 16 × 32 characters use `row_addr = pixel_y(4:1)`, `bit_addr = pixel_x(3:1)`, `char_addr` from `pixel_y(..:5)` and `pixel_x(..:4)`. Each font pixel becomes 2 × 2 screen pixels. Repeat for 32 × 64. Enlarged fonts look jagged (no new detail).

13.2 Full-screen text display [§13.3]

Now the character generation circuit is a **tile memory** holding the ASCII code of each of the 80 × 30 tiles. Address = concatenated tile coordinates: 7 + 5 = **12 bits**, so a $2^{12} \times 7$ **synchronous dual-port RAM** (write port for the editor, read port for the display). Demo: a cursor (reversed colours) moved right/down by two buttons; a third button writes the 7-bit switch value at the cursor.

▲ Two-cycle pipeline delay

Tile RAM (1 cycle) + font ROM (1 cycle) = **2 cycles**, but `pixel_x` changes every 2 cycles, so when `font_word` arrives `pixel_x` has already moved on and the **wrong bit** would be selected. Fix: pass `pixel_x`/`pixel_y` through two delay registers (`pix_x1_reg`, `pix_x2_reg`) and use the **delayed** x for `bit_addr` (the undelayed values still address the memories). The picture then shifts right by one pixel; correct it by generating `hsync`/`vsync` from the delayed coordinates as well.

13.3 The complete pong game [§13.4]

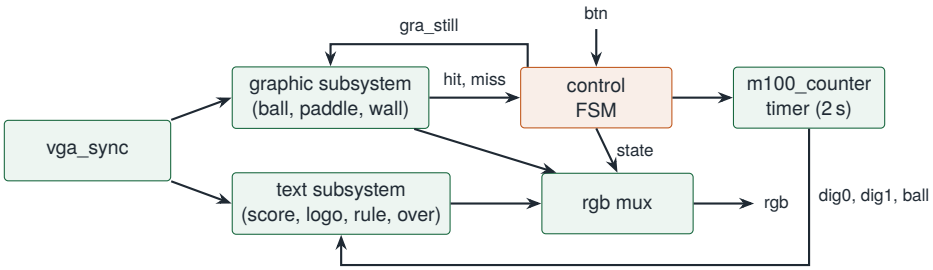


Fig. 13.4 (simplified): top-level structure of the pong game

Rules: 3 balls per game; a paddle hit scores 1 point; after a miss the game pauses and a new ball is given; after 3 misses "Game Over".

- **Text subsystem**: four text objects, each with its own `on` signal and font-ROM address: **score** "Score:DD Ball:D" (16×32 font, top of screen), **logo** "PONG" (64×128), **rule** message (8×16, in a small tile window), **game over** (32×64). Digits: ASCII = `"011" & dig` (`0x30`+digit). Output `text_on(3:0)`.
- **Graphic subsystem changes**: input `gra_still` (centre paddle and ball, no motion); outputs `hit` and `miss` ticks; `graph_on`.
- **Utility counters**: `m100_counter` (two-digit decade counter 00–99 with `d_inc`, `d_clr`); `timer` (2 s from the 60 Hz tick, `timer_start` → `timer_up`).

Control FSM (Fig. 13.6)

- **newgame**: show rule; wait for a button; clear score, balls = 3 → **play**.
- **play**: on `hit`, `d_inc`; on `miss`, start timer, balls −1; if 0 → **over**, else → **newball**.
- **newball**: wait until `timer_up` and a button → **play**.
- **over**: show "Game Over"; after `timer_up` → **newgame**.

rgb mux priority: text first if score (`text_on(3)`), rule in newgame (`text_on(1)`) or game-over in over (`text_on(0)`); else graphics (`graph_on`); else the logo (`text_on(2)`, it is on the *background*, behind the ball); else background colour.

⋆ Exam practice

1. Give the font ROM address for row 5 of the letter 'A' (`0x41`). (*`1000001 0101`.*)
2. Explain why the full-screen text display needs delayed pixel coordinates and the simple font display does not.
3. How many tiles fit on 640×480 with a 16×32 font, and how many address bits does the tile memory need? (*40×15; 6+4 = 10 bits.*)
4. Draw the pong control FSM with its inputs and outputs.

Part III

PicoBlaze microcontroller

Organisation, instruction set, assembly, I/O interface, interrupts

14 PicoBlaze overview and instruction set

From custom FSMD to programmable microcontroller, PicoBlaze organisation, development flow, programming model and all 57 instructions

Book Chapter 14, pp. 323–346

14.1 From FSMD to microcontroller [§14.2]

In an FSMD *everything* is tailored to one application (registers, routing, functional units, FSM) and can finish complex work in one clock cycle. Alternative: **keep the hardware fixed and customise the software**:

- Data path → **register file** (2 read, 1 write port) + **ALU** with a fixed set of functions: RT operation $rd \leftarrow r1\ op\ r2$.
- FSM → **programmable state machine**: state register → **program counter**; per-state output patterns → **instructions** in **instruction memory**; next state is normally PC+1, changed only by jump/call instructions.

Customising the system = writing a new program.

When to use PicoBlaze instead of an FSMD (§14.2.2)

Trade-off: **hardware complexity vs. performance vs. ease of development**. PicoBlaze does **one instruction per 2 clocks**: at 50 MHz, **25 million instructions/s**. Estimate the budget: e.g. a keyboard event every 1 ms leaves 25,000 instructions per event. Use the microcontroller for **non-time-critical "house-keeping" and I/O** tasks; keep time-critical tasks in custom hardware, both in the same FPGA.

14.2 PicoBlaze organisation [§14.3]

Feature	Value
data width / ALU	8-bit, flags **C** (carry) and **Z** (zero)
registers	**16** general-purpose 8-bit (`s0`–`sF`)
data RAM ("scratch pad")	**64 bytes**
instruction width	**18 bits**
program size	**10-bit** address ⇒ **1024** instructions
call/return stack	**31 words**
I/O	**256 input + 256 output ports** (8-bit `port_id`)
speed	**2 clock cycles per instruction**
interrupt	**5 clock cycles** response; vector `3FF`
size	≈**200 logic cells** (< 5% of an XC3S200)

It is a **soft core**: a cell-level HDL description synthesised with the rest of the logic.

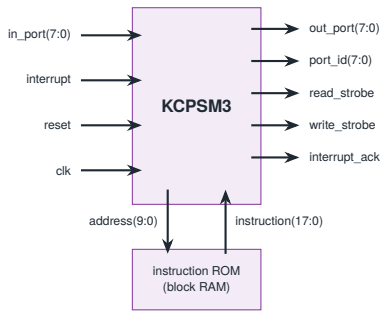
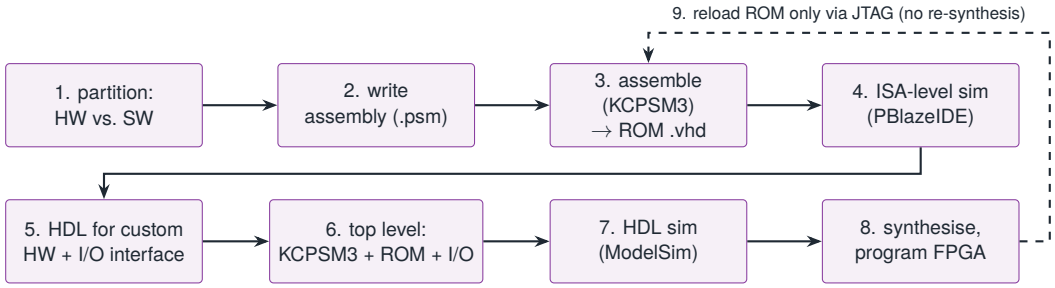


Fig. 14.3: the two top-level HDL modules

KCPSM3 = "constant (K) coded programmable state machine", PicoBlaze's original name. Enhancements over the bare skeleton: data RAM (no direct path to the ALU: *fetch* to a register first), immediate constants, the stack, I/O ports and interrupt logic.

14.3 Development flow [§14.4]



14.4 Programming model and formats [§14.5.1–14.5.2]

Programmer-visible state: 16 registers, 64-byte data RAM, flags **Z**, **C**, **I** (interrupt enable), PC, top-of-stack pointer. Notation: `sX`, `sY` registers; `KK` 8-bit constant or port id; `SS` 6-bit RAM address; `AAA` 10-bit instruction address.

Format	Used by
`op sX, sY`	register-register (`sX` is also destination)
`op sX, KK`	register-constant
`op sX`	shift / rotate
`op AAA`	jump / call
`op`	return, interrupt control

Two assemblers: Xilinx **KCPSM3** and Mediatronix **PBlazeIDE**; PBlazeIDE mnemonics in brackets below.

14.5 Instruction set (57 instructions) [§14.5.3–14.5.9]

Instruction	Operation	Flags
Logical		
`and sX, sY|KK`	sX ← sX and sY/KK	C←0, Z by result
`or sX, sY|KK`	sX ← sX or sY/KK	C←0, Z
`xor sX, sY|KK`	sX ← sX xor sY/KK	C←0, Z
Arithmetic		
`add sX, sY|KK`	sX ← sX + sY/KK	C = carry, Z
`addcy (addc) sX, sY|KK`	sX ← sX + sY/KK + C	C, Z
`sub sX, sY|KK`	sX ← sX − sY/KK	C = borrow, Z
`subcy (subc) sX, sY|KK`	sX ← sX − sY/KK − C	C, Z
Compare and test (registers unchanged)		
`compare (comp) sX, sY|KK`	sX − sY/KK, result discarded	Z=1 if equal; **C=1 if sY/KK > sX**
`test sX, sY|KK`	t ← sX and sY/KK, discarded	Z=1 if t=0; **C = odd parity of t**
Shift and rotate (single register; the bit shifted out goes to C)		
`sl0 / sl1 / slx / sla sX`	shift left; LSB ← 0 / 1 / sX(0) / C	C ← sX(7)
`sr0 / sr1 / srx / sra sX`	shift right; MSB ← 0 / 1 / sX(7) / C	C ← sX(0)
`rl sX / rr sX`	rotate left / right	C ← bit rotated out
Data movement		
`load sX, sY|KK`	sX ← sY / KK	–
`fetch sX, (sY)|SS`	sX ← RAM[sY] / RAM[SS]	–
`store sX, (sY)|SS`	RAM[sY] / RAM[SS] ← sX	–
`input (in) sX, (sY)|KK`	port_id ← sY/KK; sX ← in_port	–
`output (out) sX, (sY)|KK`	port_id ← sY/KK; out_port ← sX	–
Program flow		
`jump AAA; jump c|nc|z|nz, AAA`	PC ← AAA (if condition)	–
`call AAA; call c|nc|z|nz, AAA`	push PC; PC ← AAA (if condition)	–
`return; return c|nc|z|nz`	PC ← pop + 1 (if condition)	–
Interrupt		
`enable / disable interrupt (eint/dint)`	I ← 1 / 0	–
`returni enable|disable`	PC ← pop (**no +1**); restore Z, C; I ← 1/0	Z, C restored

▲ Flag traps

Logical instructions always clear C. **compare** sets C when the *second* operand is larger (i.e. sX < sY): "`compare s0, 10 / jump c, less`" jumps if s0 < 10. **test** puts the *parity* in C. **load**/**fetch**/**input** do **not** change flags. There is no instruction to read instruction memory; constants live inside instructions.

Interrupt in one sentence (§14.5.9)

If I=1 and `interrupt` is asserted, PicoBlaze finishes the current instruction, saves the address of the *next* instruction on the stack, preserves Z and C, clears I and loads **PC = 3FF** (the interrupt vector). `returni` restores the saved address *without* adding 1.

Directives (§14.6): KCPSM3 `ADDRESS 3FF`, `NAMEREG s5, index`, `CONSTANT max, 3F`. PBlazeIDE: `ORG`, `EQU`, `DSIN/DSOUT/DSIO` (I/O ports shown in the simulator), `VHDL "template", "target", "entity"`; hex numbers need `\$`.

⋆ Exam practice

1. s0 = `0x3C`, s1 = `0x5A`. Give s0, Z and C after each of: `and s0,s1`; `add s0,s1`; `compare s0,s1`; `test s0,s1`. (*and: 18, Z0 C0; add: 96, Z0 C0; compare: s0 unchanged, Z0, C1 (5A>3C); test: t=18 has two 1s, Z0, C0.*)
2. How many instructions can PicoBlaze execute in 1 ms at 50 MHz?
3. Explain the difference between `return` and `returni`.
4. Why can a task that must react within 50 ns not be done by PicoBlaze?

15 PicoBlaze assembly programming

Bit and multi-byte manipulation, if/case/for in assembly, subroutines and register use, program structure, assembling, simulating and synthesising

Book Chapter 15, pp. 347–370

15.1 Useful code segments [§15.2]

KCPSM3 conventions: label followed by `:`, comment after `;`, registers `s0`–`sF`, constants in hex (2 digits for data/ports, 3 for addresses).

Bit manipulation with masks: `or` with 1s **sets**, and with 0s **clears**, `xor` with 1s **toggles** ($x \oplus 0 = x$, $x \oplus 1 = x'$). `test` with a one-bit mask **checks** a bit (Z=1 if the bit is 0).

Bit operations on s0

```
or   s0, 20    ; set bit 5          (0010 0000)
and  s0, DF    ; clear bit 5        (1101 1111)
xor  s0, 20    ; toggle bit 5
test s0, 80    ; check MSB
jump nz, msb_set
; extract MSB of s0 into s1 (0 or 1)
load s1, 00
test s0, 80
jump z, done
load s1, 01
done:
```

Multi-byte data: the carry flag links bytes.

24-bit addition x <- x + y (x2:x1:x0, y2:y1:y0)

```
add   x0, y0   ; LSB: no carry in
addcy x1, y1   ; middle byte + carry
addcy x2, y2   ; MSB + carry
; 16-bit shift left of (s1:s0)
sl0 s0         ; bit 7 of s0 -> C
sla s1         ; C -> bit 0 of s1
```

Control structures are built from `compare`/`test` + conditional jumps.

if (s0 = s1) then s3 <- 01 else s2 <- s2 + 1

```
      compare s0, s1
      jump nz, else_branch
      load s3, 01          ; then branch
      jump if_done
else_branch:
      add s2, 01
if_done:
```

case s0 is 1 => ..., 2 => ..., others => ...

```
; no indexed jump, so: chain of if-then-else
      compare s0, 01
      jump z, case_1
      compare s0, 02
      jump z, case_2
      jump case_others
```

for i in 15 downto 0 loop ... end loop

```
      load i, 0F        ; i = NAMEREG alias
loop_body:
      ...               ; body
      sub i, 01
      jump nc, loop_body ; C=1 only when 00 -> FF
```

15.2 Subroutines [§15.3]

`call` saves the PC and jumps; the routine ends with `return`. PicoBlaze saves **only the PC**: **you** must manage registers and data RAM so the caller's state is not destroyed. Every routine needs a **header**: purpose, input/output registers, registers it changes.

Shift-and-add multiplication (`mult_soft`, 8×8→16 bit): loop 8 times; shift the multiplier right so its LSB goes to C; if C=1 add the multiplicand to the upper byte of the product; then shift the 16-bit product right (instead of shifting the multiplicand left). About 60 instructions worst case.

Software multiplication (after Listing 15.1)

```
; s3 = multiplicand, s4 = multiplier
; result s5:s6 (MSB:LSB), uses s2 as counter
mult_soft:
      load s5, 00        ; clear upper product
      load s2, 08        ; 8 iterations
mult_loop:
      sr0  s4            ; LSB of multiplier -> C
      jump nc, shift_prod
      add  s5, s3        ; add multiplicand (C = carry)
shift_prod:
      sra  s5            ; C -> MSB of s5, LSB -> C
      sra  s6            ; C -> MSB of s6
      sub  s2, 01
      jump nz, mult_loop
      return
```

15.3 Program development [§15.4]

1. Derive the pseudo code.
2. Identify tasks; make them subroutines (divide and conquer).
3. **Plan register and data-RAM use** (unique to assembly: no compiler allocates storage). Registers = fast working storage; data RAM = auxiliary storage (must `fetch` to a register to process, then `store` back). Separate **global** storage (whole program) from **local** storage (temporary).
4. Write and test each subroutine.

A microcontroller program is a **polling loop**:

```
call init
forever:
      call task1
      call task2
      ...
      jump forever
```

Demo: $a^2 + b^2$ (§15.4.1)

Read a (upper nibble) and b (lower nibble) of the 8-bit switch, compute $a^2 + b^2$ with `mult_soft`, show the LSBs on the LEDs. Routines: `clr_data_mem` (loop storing 0 in all 64 bytes), `read_switch` (uses `get_upper_nibble` = shift right 4 times, `get_lower_nibble` = `and 0F`), `square`, `write_led`. Global register `sw_in`; 11 bytes of data RAM for a, b, a^2, b^2, sum; aliases `data`, `addr`, `i` for temporaries.

Documentation (§15.4.2): header with program description, **constants** (`CONSTANT`), **data RAM addresses**, **register aliases** (`NAMEREG`), **port ids**, and the **calling hierarchy**. Aliases have no effect on machine code but make it readable and easy to change.

15.4 Tools and synthesis [§15.5–15.6]

- **KCPSM3 assembler** (DOS command line): `kcpsm3 myfile.psm` with templates `ROM_form.vhd/.v/.coe` in the same folder → `myfile.vhd` (instruction ROM as block RAM), plus `.hex` (JTAG reload) and `.fmt` (formatted source).
- **PBlazeIDE**: editor + assembler + **instruction-set simulator** (registers, flags, RAM, PC, stack, emulated I/O ports; step, step over, breakpoints). Import the `.fmt`, add `DSIN/DSOUT` for ports. Different mnemonics (`addc`, `comp`, `in`, `out`...).
- **JTAG reload**: after the I/O hardware is fixed, only the ROM changes; use a ROM template with a JTAG loader and download the new `.hex` without re-synthesising.

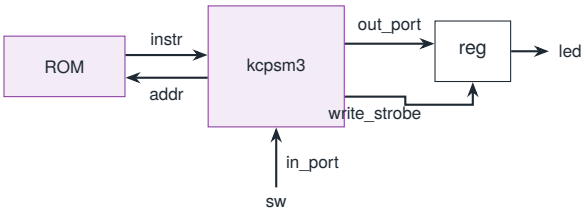


Fig. 15.3: PicoBlaze with one input and one output port

With one output port no decoding is needed: `write_strobe` enables the LED register directly. With one input port, `sw` goes straight to `in_port`.

⋆ Exam practice

1. Write a routine that counts the number of 1s in s0 (result in s1).
2. Write a 16-bit decrement of (s1:s0) and a jump if the result is zero.
3. Why must register usage be documented in a PicoBlaze subroutine header?
4. Write assembly for: `if s0 < 30 then s1 <- s1 + 1.` (*`compare s0,30 / jump nc,skip / add s1,01 / skip:`*)
5. Reverse the bit order of s0 into s1 using shifts and a loop. [§15.8.4]

16 PicoBlaze I/O interface

Output and input instruction timing, decoding and multiplexing, continuous- vs. single-access ports, and the square circuit with switches, LEDs, a hardware multiplier and a UART console

Book Chapter 16, pp. 371–408

16.1 I/O signals and timing [§16.1–16.3]

PicoBlaze has **no built-in peripherals**; you build exactly the I/O the application needs. Signals: `port_id` (8-bit port address), `in_port`, `out_port`, `read_strobe`, `write_strobe` (each strobe asserted in the **second clock cycle** of the instruction). One pair of 8-bit data ports, but `port_id` selects among up to 256 input and 256 output ports.

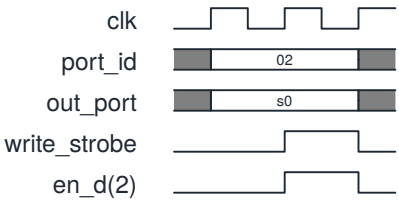


Fig. 16.1: `output s0, 02`; the decoded enable tick stores out_port

Input instruction `input s0, 02`: `port_id`=02 for two cycles; `in_port` is sampled at the end of the instruction, so the external data must be stable then. The input interface is a **multiplexer** controlled by `port_id`.

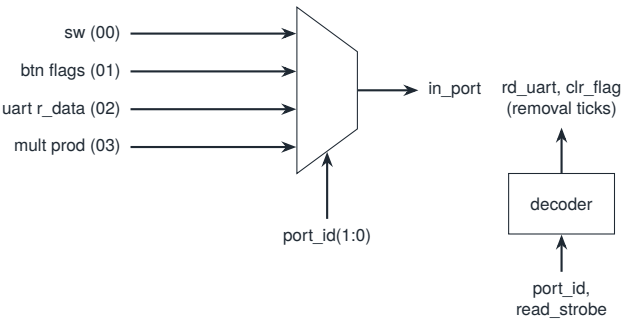
Port type	Meaning and interface
Continuous-access	data always present (switches): **mux only**.
Single-access	data produced by a single event (UART word, button press, FIFO): after reading it must be **removed**, else processed again. Decode `port_id` with **`read_strobe`** to make a one-cycle **removal** tick (e.g. FIFO `rd`, flag-FF clear).

Output interface

Decoding circuit + output registers. In cycle 2, `write_strobe` and a decode of `port_id` give a one-cycle enable tick to the right register, which loads `out_port`.

```
process(port_id, write_strobe)
begin
  en_d <= (others=>'0');
  if write_strobe='1' then
    case port_id(1 downto 0) is
      when "00"   => en_d <= "0001";
      when "01"   => en_d <= "0010";
      when "10"   => en_d <= "0100";
      when others => en_d <= "1000";
    end case;
  end if;
end process;
```

Only the LSBs needed to distinguish ports are decoded (port ids 00–03). With ≤ 8 ports, **one-hot port ids** (01, 02, 04, 08...) make decoding trivial: `en_d(i) <= write_strobe and port_id(i)`. A single port needs no decoding.



Figs. 16.4–16.5 (combined idea): input mux + removal decoder

16.2 Square circuit with switch, buttons and 7-segment display [§16.4]

- **Inputs**: 8-bit switch (continuous); two buttons **s** (set: load *a*, then *b*, alternately) and **c** (clear data RAM), each through a **debouncer + flag FF** (single-access), grouped into one 8-bit input port. A global register `switch_a_b` (0 or 2) is the RAM offset for *a* or *b* and toggles on each s press.
- **Outputs**: four 8-bit output registers feeding the

hardware LED time-multiplexing circuit `disp_mux` (the hardware hides the refresh timing). 17-bit result: 4 hex digits + the leftmost decimal point for the MSB. The 3 lower switch bits select what to display.

Program (Listing 16.1): `init` (clear RAM, `switch_a_b` = 0) then loop `proc_btn`, `square`, `load_led_pttn` (reads switch, fetches values, converts nibbles with `hex_to_led` into LED patterns stored in RAM `led0..led3`), `disp_led` (writes the 4 patterns to output ports).

proc_btn (idea)

```
      input s0, rd_flag_port   ; read + clear flags
      test  s0, 02             ; s button?
      jump  z, chk_c
      ... store switch at a or b, toggle switch_a_b
chk_c: test s0, 01             ; c button?
      jump  z, btn_done
      call  init
btn_done: return
```

16.3 Adding a combinational multiplier and a UART console [§16.5]

Hardware multiplier as a peripheral: PicoBlaze has no coprocessor interface, so the 8×8 multiplier (* in VHDL, inferred into an 18×18 block) is just **two output ports** (operands) and **two input ports** (16-bit product). It finishes within one instruction (2 cycles), so no wait is needed. Replaces the ≈60-instruction `mult_soft`.

mult_hard

```
output s3, mult_src0_port
output s4, mult_src1_port
input  s6, mult_prod0_port   ; LSB
input  s5, mult_prod1_port   ; MSB
return
```

UART console: the Ch. 7 UART with its FIFOs needs no extra circuit; extend the decoders: `out_port` → `w_data`, decoded write tick → `wr_uart`; `r_data` into the input mux, decoded read tick → `rd_uart`; `rx_empty` and `tx_full` grouped with the button flags. Console: prompt `SQ>`; keys `a`/`b` load operands from the switch, `c` clears, `d` dumps the 64-byte data RAM as an 8×8 table; anything else prints `Error`.

tx_one_byte: never overflow the transmit FIFO

```
tx_one_byte:
      input s6, rd_flag_port
      test  s6, 02          ; tx_full bit
      jump  nz, tx_one_byte ; busy-wait until not full
      output data, uart_tx_port
      return
```

⋆ Exam practice

1. Draw the timing of `output s1, 05` and show when the output register loads.
2. Design the decoding circuit for 4 output ports with one-hot ids. Why is it simpler?
3. Why must a button input be a single-access port? What happens if the flag FF is never cleared?
4. Draw the input interface for: switch (port 0), button flags (port 1), UART receive FIFO (port 2), including the removal logic.
5. Compare `mult_soft` and the hardware multiplier port in speed and hardware cost.

17 PicoBlaze interrupts

Polling vs. interrupt, the PicoBlaze interrupt sequence and timing, single and multiple request interfaces, ISR design, and a software-multiplexed LED display driven by a timer interrupt

Book Chapter 17, pp. 409–426

17.1 Interrupt handling [§17.1–17.2]

Normally the program **polls** peripherals (checks status in its loop; the peripheral waits passively). An **interrupt** lets the peripheral start the action: normal execution is suspended and an **interrupt service routine (ISR)** runs. Reserve it for **time-critical** requests.

Sequence (hardware + software):

1. Peripheral asserts `interrupt`; if interrupts are enabled (I=1) PicoBlaze **finishes the current instruction**.
2. It asserts `interrupt_ack`, saves the PC (address of the *next* instruction) on the stack, **preserves Z and C**, clears I, and implicitly executes **`call 3FF`**.
3. Address `3FF` (last instruction word) holds `jump isr`.
4. The ISR does the work and ends with `returni enable` (or `disable`): restore PC, Z, C and set I.

Program skeleton with an ISR

```
        call init
        enable interrupt      ; turn on I flag
forever:
        ...                   ; polling tasks
        jump forever

isr:    ...                   ; service routine
        returni enable

        ADDRESS 3FF           ; interrupt vector
        jump isr
```

Timing (Fig. 17.2)

t1: request asserted during `add s0,s3` (execution continues) → t2: next instruction (`sub s5,01`) aborted → t3: `interrupt_ack`, save address of `sub`, save flags, I=0 → t4: execute `jump isr` at `3FF`. **Up to 5 clock cycles** from request to the first ISR instruction.

17.2 External interrupt interface [§17.3]

A request is like a **single-access port**: after it is accepted it must be cleared, or it would be serviced again. Use a **flag FF**.

Single request: the peripheral's one-cycle `int_request` tick sets the flag; flag output drives `interrupt`; `interrupt_ack` **clears** the flag.

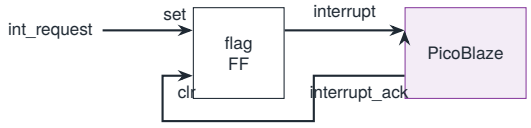


Fig. 17.3

Multiple requests (Fig. 17.4): one flag FF per source; **OR** of the flags drives `interrupt`; the flag outputs also go to the **input mux** so the ISR can read which source(s) requested and serve them in **priority** order. `interrupt_ack` cannot clear the right flag (it does not know which was accepted), so each flag's `clr` is given its own **port id**: the ISR executes a dummy `output` to that port to create the clear tick. An **interrupt controller** (flags + priority encoder giving `req_id`, clearing on `interrupt_ack`) reduces the software overhead [§17.7.5].

17.3 Software considerations [§17.4]

- The polling loop is an implicit **round-robin schedule**; fine if the loop is short enough. Interrupts **give priority** to one or two time-critical requests.
- Interrupts can come at **any time**: the loop timing must allow for interrupt frequency × service time.

▲ Context saving

PicoBlaze saves only PC, Z and C. The ISR must **save and restore every register it uses** (context switching), or use **dedicated registers** reserved for the ISR only. The book recommends: keep interrupts simple, prefer polling, use dedicated ISR registers.

17.4 Design example: LED multiplexing in software [§17.5]

Replace the `disp_mux` hardware: PicoBlaze drives an (4 bits) and `sseg` (8 bits) directly through two output registers and must refresh them at a steady rate.

- **Timer**: mod-500 counter ⇒ tick every $500 \times 20\,\text{ns} = \mathbf{10\,\mu s}$ (0.01 ms). Single flag-FF interrupt interface.
- **ISR**: increment a 16-bit counter in two **dedicated registers** `count_msb:count_lsb` (wraps after $2^{16} \times 10\,\mu s \approx 0.65\,\text{s}$), then `returni enable`.
- **`disp_mux_out`** (replaces `disp_led`): every 2^{10} ticks (≈10 ms) write the next an pattern (`1110`, `1101`, `1011`, `0111`) and its LED pattern; register `led_pos` remembers the current digit.
- `init` adds `enable interrupt` and initialises `led_pos`, `count_msb`, `count_lsb`.

ISR of the example

```
isr:
      add   count_lsb, 01
      addcy count_msb, 00     ; 16-bit increment
      returni enable

      ADDRESS 3FF
      jump isr
```

▷ Timing numbers

Tick period: $500/50\,\text{MHz} = 10\,\mu\text{s}$. Refresh step as stated in the book: $2^{10} \times 10\,\mu s \approx 10\,\text{ms}$ per digit, so each digit returns every $\approx 41\,\text{ms}$ ($\approx 24\,\text{Hz}$). That is slower than the ∼1 kHz recommended in §4.5.1 and may flicker; a step of 2^6 ticks ($\approx 0.64\,\text{ms}$) matches the Ch. 4 design better. Good point to mention if asked to evaluate the design. Instruction budget between ticks: $10\,\mu s/40\,\text{ns} = 250$ instructions; ISR overhead ≈ 5 cycles + 4 instructions.

⋆ Exam practice

1. List everything PicoBlaze does in hardware between the interrupt request and the first ISR instruction.
2. Why is `interrupt_ack` enough to clear a single request but not two?
3. Write an ISR that uses s0 and s1 temporarily, saving them in data RAM. [§17.7.1]
4. Polling vs. interrupt: give one situation where each is the better choice.
5. Design a 4-request priority interrupt controller (block diagram + VHDL for the priority encoder).

Back

Revision tools

Numbers sheet, compare & contrast, template cheat sheet, glossary, practice questions

18 Numbers and formulas sheet

Every number and formula worth memorising, in one place

All chapters

Digital logic foundations (Part 0)

- Two's complement range $-2^{n-1} \ldots 2^{n-1}-1$; $-x = \overline{x}+1$; overflow $V = c_n \oplus c_{n-1}$.
- De Morgan: $\overline{AB} = \overline{A}+\overline{B}$, $\overline{A+B} = \overline{A}\,\overline{B}$; NAND/NOR universal.
- Full adder: $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$; CLA $c_{i+1} = G_i + P_i c_i$.
- TTL: $V_{IH} \geq 2.0$, $V_{IL} \leq 0.8$, $V_{OH} \geq 2.4$, $V_{OL} \leq 0.4$ V; noise margin 0.4 V; fan-out 10.
- Q^+: SR $= S + \overline{R}Q$; D $= D$; JK $= J\overline{Q} + \overline{K}Q$; T $= T \oplus Q$.
- Memory $2^n \times m$: n address, m data lines. ROM = decoder + OR array.
- SAP-1: 16×8 RAM, 5 instr. (LDA 0, ADD 1, SUB 2, OUT E, HLT F), 6 T-states; fetch 5E3, BE3, 263.

Timing and counters

- $T_{clock} = T_{cq} + T_{comb} + T_{setup}$, $f_{max} = 1/T_{clock}$.
- Board clock **50 MHz** ⇔ **20 ns**.
- Mod-M counter needs $N = \lceil \log_2 M \rceil$ bits; tick period $= M \times 20$ ns.
- Divisor for a rate r: $M = 50\,\text{MHz}/r$.
- 1 ms = 50,000 cycles (16 bits); 0.1 s = 5,000,000 (23 bits); 10 ms = 500,000.
- $2^{10} = 1024$, $2^{16} = 65{,}536$, $2^{18} = 262{,}144$, $2^{21} = 2{,}097{,}152$.

Display and debounce

- 7-segment: **active low**; 4 digits share 8 segment lines; pins 32→12.
- LED mux: 18-bit counter, 2 MSBs select; ≈800 Hz per step (target ∼1 kHz).
- Bounces settle within **20 ms**; FSM debouncer: 10 ms timer, 3 ticks ⇒ 20–30 ms; FSMD debouncer: $2^{21} \times 20$ ns ≈ **42 ms**, exact.
- Stopwatch: mod-5,000,000 ⇒ 0.1 s tick.

UART

- Idle 1; start 0; 8 data **LSB first**; stop 1 (8N1).
- Oversampling 16×: count to **7** (middle of start), then **15** per bit; `SB_TICK` = 16/24/32 for 1/1.5/2 stop bits.
- 19,200 baud: $50\text{M}/(16 \cdot 19{,}200) = 162.8$ ⇒ **mod-163**. 9600: 326.
- Errors: parity, frame (stop bit ≠1), buffer overrun.

PS2

- 11 bits: start 0, 8 data LSB first, **odd parity**, stop 1.
- `ps2c` 10–16.7 kHz (60–100 µs); read on **falling edge**; filter = 8 equal samples.
- Make A = `1C`; break = `F0`+make; extended `E0`..; typematic 0.5 s then every 100 ms; Shift = `12`.
- Mouse: 3 bytes, 9-bit 2's-complement dx, dy; `F4` enable stream, `FF` reset, `AA 00` self-test OK; 4 counts/mm, 100 samples/s.
- Host send: `ps2c` low ≥100 µs, then start bit, device clocks.

SRAM (IS61LV25616AL)

- 2 chips × 256K×16 = 1 MB = 8 Mbit; 18-bit address.
- $t_{RC} = t_{WC} = 10$, $t_{AA} = 10$, $t_{PWE1} = 8$, $t_{SD} = 6$, $t_{HD} = 0$, $t_{DOE} = 4$, $t_{HZOE} = 4$ ns.
- Safe controller: 2 cycles per access, **3 cycles** back-to-back.
- Data latched on `we_n` **0**→**1**; controls active low.

FPGA resources (XC3S200)

- LC = 4-input LUT + D FF (+carry, mux); slice = 2 LC; CLB = 4 slices.
- 4,320 LCs; 12 block RAMs (16K data + 2K parity bits each); 12 multipliers 18×18; DCMs.
- Storage: FFs ≈4.5K, distributed RAM 30K, block RAM ≈172–192K, external SRAM 8M bits.

VGA 640×480 @ 60 Hz

- Pixel rate = 800 × 525 × 60 ≈ **25 MHz** (enable tick from 50 MHz).
- H: 640 display, 16 front porch (right), 96 retrace, 48 back porch (left) = **800**; hsync active for 656–751.
- V: 480 display, 10 front (bottom), 2 retrace, 33 back (top) = **525**; vsync for 490–491.
- 1 bit/colour ⇒ 8 colours; `110` = yellow.
- Bit-mapped 310K / 930K bits; tile 8×8 ⇒ 80×60 tiles.
- Font 8×16, 128 chars, ROM $2^{11} \times 8 = 1$ BRAM; text 80×30 tiles; tile RAM $2^{12} \times 7$.
- Animation updated once per frame (60 Hz, at `pixel_y`=481).

PicoBlaze

- 8-bit, 16 registers, 64-byte RAM, 18-bit instructions, 1024 words, 31-level stack, 256+256 ports.
- **2 cycles/instruction** = 25 MIPS at 50 MHz; interrupt ≤5 cycles, vector **3FF**; ≈200 LCs.
- Strobes in cycle 2 of `input/output`.

19

Compare and contrast

The pairs that examiners like to put side by side

Aspect	A	B
Combinational vs. sequential		
Output depends on	present inputs only	inputs and stored state (memory)
Built from	gates, mux, decoder, adder	+ latches / flip-flops, clock
Latch vs. flip-flop		
Sensitive to	level (transparent while enabled)	clock edge only
Use	SRAM cells, (unintended) latches	registers, all synchronous design
TTL vs. CMOS		
Technology	bipolar, 5 V, totem-pole outputs	MOS, 1–6 V, pull-up + pull-down networks
Power / noise margin	static power, 0.4 V margin	power only when switching, larger margin
SRAM vs. DRAM vs. ROM		
Cell	6T latch, fast, no refresh	1T1C charge, dense, refresh, destructive read | ROM: decoder + OR array, non-volatile
Ripple vs. synchronous counter		
Clocking	each FF clocked by previous output	common clock, $T_i = \prod_{j<i} Q_j$
Pros/cons	few gates; slow, glitchy, not synchronous	more gates; fast, clean, analysable
Hardwired vs. microprogrammed control		
How	gates combine T-state + opcode (SAP-1)	control ROM holds control words
Trade-off	fast, hard to change	flexible, easy to add instructions
Conditional (`when-else`, `if`) vs. selected (`with-select`, `case`)		
Hardware	priority network, cascade of 2:1 muxes	one 2^n:1 multiplexer
Delay	grows with number of clauses	grows with width of `sel`
Rules	order matters; final `else`	choices exclusive + all-inclusive; `others`
Concurrent statement vs. process		
Semantics	each statement = parallel circuit; order irrelevant	sequential statements; last assignment wins
Flexibility	one expression per branch, no nesting	many statements per branch, nesting, defaults
Asynchronous reset vs. synchronous clear		
When	anytime, not tied to clock	sampled at the clock edge
Use	power-on initialisation only	normal operation (`syn_clr`)
Moore vs. Mealy output		
Depends on	state only	state and input
Timing	one cycle later, exactly one cycle wide, glitch-free wrt. input	same cycle as input, faster; width varies; input glitches pass
States	usually more	usually fewer
State diagram vs. ASM chart		
Form	nodes + arcs; Moore in node, Mealy on arc	state box, decision box, conditional output box
Best for	simple FSMs (compact)	complex conditions, FSMD (RT ops)
FSM vs. FSMD vs. microcontroller		
Structure	control only	control FSM + custom data path | fixed ALU + register file + program
Speed / cost	fast, small	fastest (parallel ops per cycle), custom | 2 cycles/instruction, general, easy to change
Explicit vs. implicit data path (FSMD code)		
How	data path components coded separately; FSM gives control signals	RT operations written inside the FSM case
Trade-off	more control, resource sharing	shorter, follows ASMD chart; synthesis may duplicate units
FSM debouncer vs. FSMD debouncer		
Timer	free-running 10 ms tick, independent	loaded and controlled by the FSM
Delay	20–30 ms (inexact)	exact ($2^{21} \cdot 20$ ns)
UART vs. PS2		
Clock	none sent; 16× oversampling	device sends `ps2c`; falling edge
Frame	start, 6–8 data, opt. parity, 1–2 stop	start, 8 data, odd parity, stop (fixed)
Direction	separate rx/tx lines	bidirectional open-collector lines (tri-state)
Register file vs. external SRAM vs. block RAM vs. distributed RAM		
Access	reg file: many read ports, read+write same cycle, async read	SRAM: async, needs controller, 2–3 cycles; BRAM: sync read (1 cycle); dist. RAM: async read possible
Size	register file: tiny (FFs); distributed RAM 30K	block RAM ≈190K; SRAM 8M bits
Bit-mapped vs. tile-mapped vs. object-mapped		

20 VHDL template cheat sheet

Write these from memory: they cover almost every code question (after Appendix A of the book)

1. File skeleton

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity name is
   generic(N: integer := 8);
   port(
      clk, reset: in  std_logic;
      d:          in  std_logic_vector(N-1 downto 0);
      q:          out std_logic_vector(N-1 downto 0)
   );
end name;
architecture arch of name is
   signal r_reg, r_next: unsigned(N-1 downto 0);
begin
   ...
end arch;
```

2. Instantiation

```
unit_1: entity work.comp_name(arch)
   generic map(N => 4)
   port map(clk=>clk, reset=>reset, d=>x, q=>open);
```

3. Routing

```
y <= a when s="00" else b when s="01" else c;
with s select
   y <= a when "00", b when "01", c when others;
```

4. Combinational process: defaults first

```
process(a, b, s)
begin
   y <= (others=>'0');       -- default
   case s is
      when "00"   => y <= a;
      when "01"   => y <= b;
      when others => null;
   end case;
end process;
```

5. Register (the only clocked process)

```
process(clk, reset)
begin
   if reset='1' then
      r_reg <= (others=>'0');
   elsif (clk'event and clk='1') then
      r_reg <= r_next;
   end if;
end process;
```

6. Counter / tick generator

```
r_next <= (others=>'0') when r_reg=(M-1) else r_reg+1;
tick   <= '1' when r_reg=(M-1) else '0';
```

7. FSMD template (one next-state process)

```
type state_type is (idle, op, done);
signal state_reg, state_next: state_type;
signal n_reg, n_next: unsigned(3 downto 0);
...
process(clk, reset)            -- state + data regs
begin
   if reset='1' then
      state_reg <= idle; n_reg <= (others=>'0');
   elsif (clk'event and clk='1') then
      state_reg <= state_next; n_reg <= n_next;
   end if;
end process;
process(state_reg, n_reg, start)   -- next-state
begin
   state_next <= state_reg;   -- defaults
   n_next     <= n_reg;
   ready <= '0'; done_tick <= '0';
   case state_reg is
      when idle =>
         ready <= '1';
         if start='1' then
            n_next <= (others=>'1');
            state_next <= op;
         end if;
      when op =>
         n_next <= n_reg - 1;
         if n_reg=0 then       -- old value! (delayed
     store)
            state_next <= done;
         end if;
      when done =>
         done_tick  <= '1';
         state_next <= idle;
   end case;
end process;
```

8. Rising-edge detector

```
delay_next <= level;                -- 1-bit register
tick <= level and (not delay_reg);
```

9. Tri-state / bidirectional

```
pin <= data_out when oe='1' else 'Z';
data_in <= pin;
```

10. Testbench clock and reset

```
process begin
   clk <= '0'; wait for T/2;
   clk <= '1'; wait for T/2;
end process;
reset <= '1', '0' after T/2;
```

21 Glossary

Cover the right column and explain each term in one or two sentences

All chapters

Term	Meaning
Accumulator	CPU register holding one operand and the ALU result (SAP-1 register A).
ALU	Arithmetic logic unit: logic ops, add/subtract, result mux, flags Z, C, N, V.
ASM chart	Flowchart-like FSM description of ASM blocks: state box (Moore outputs), decision box, conditional output box (Mealy outputs).
ASMD chart	ASM chart extended with RT operations; used to specify an FSMD.
Asynchronous SRAM	SRAM without a clock; signals need a specific order and hold times; accessed via a memory controller.
Barrel shifter	Circuit that shifts/rotates by a variable amount; built in $\log_2 n$ stages.
Baud rate	Number of bits per second on a serial line (e.g. 19,200).
BCD	Binary-coded decimal: each decimal digit in 4 bits.
Bit-mapped scheme	Video memory holds one word per pixel.
Block RAM	Dedicated synchronous 18K-bit (16K data) memory block in Spartan-3.
Break code	PS2 keyboard code for a released key: `F0` + make code.
Carry look-ahead	Adder computing carries in parallel from generate $G_i = a_i b_i$ and propagate $P_i = a_i \oplus b_i$.
CLB	Configurable logic block: 4 slices = 8 logic cells.
CMOS	Complementary MOS logic; near-zero static power; used in FPGAs.
Concurrent statement	Statement outside a process; models a circuit part running in parallel.
Constraint file (UCF)	File with pin assignments (`LOC`) and timing constraints (`PERIOD`).
Control path	The FSM part of an FSMD, producing control signals.
Control word	Set of control signals for one clock step (SAP-1: 12 bits per T-state).
Data overrun	A received word overwritten before the system reads it.
Data path	Registers, functional units and routing network of an FSMD.
DCM	Digital clock manager: multiplies/divides/phase-shifts the clock; reduces skew.
DDR register	Double-data-rate output register clocked by two clocks; creates half-cycle pulses.
De Morgan's laws	$\overline{AB} = \overline{A} + \overline{B}$, $\overline{A+B} = \overline{A}\,\overline{B}$.
Debouncing	Filtering the bounces of a mechanical switch (settle within 20 ms).
Decoder	n-to-2^n circuit activating the output selected by the binary input.
Delayed store	RT results are written only when leaving the ASMD block; a decision box sees the old value.
Distributed RAM	RAM built from LUTs (16×1 each); supports asynchronous read.
DRAM	Memory storing bits as charge (1 transistor + capacitor); needs refresh.
Edge detector	Circuit giving a one-cycle tick when an input changes 0→1.
Encoder	Outputs the binary code of the active input; priority encoder picks the highest.
Enumerated type	User type listing symbolic values, e.g. FSM states.
Excitation table	Flip-flop inputs needed for each wanted transition $Q \to Q^+$.
Fetch cycle	T-states that load the next instruction into the IR (SAP-1: T1–T3).
FIFO	First-in-first-out buffer; `wr` writes, `rd` removes the head; `full`, `empty`.
Flag FF	FF set by an event tick and cleared after the event is served.
Font ROM	ROM storing the pixel patterns of characters ($2^{11} \times 8$ for 128 chars of 8×16).
FPGA	Field-programmable gate array: array of logic cells and programmable switches configured in the field.
Frame error	UART stop bit sampled as 0.
FSM	Finite state machine: sequential circuit with "random" next-state logic.
FSMD	FSM with data path; implements RT methodology.
Full adder	Adds A, B, C_{in}: $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$.
Generic	Parameter passed into an entity (e.g. width); set with `generic map`.
Glitch	Short unwanted pulse caused by unequal delays.
Half adder	Adds two bits: $S = A \oplus B$, $C = AB$.
Interrupt	Peripheral-initiated break of normal program flow to run an ISR.
IOB	Input/output block of an FPGA pin: drivers, registers, tri-state buffers.

Term	Meaning
ISR	Interrupt service routine; starts at the vector (PicoBlaze: `3FF`).
JK flip-flop	Flip-flop with hold/reset/set/toggle; $Q^+ = J\overline{Q} + \overline{K}Q$.
Karnaugh map	Gray-coded truth-table grid for grouping 1s to simplify logic.
Latch (unintended)	Memory inferred when a signal is not assigned in all branches of a combinational process.
Logic cell (LC)	4-input LUT + D FF + carry and mux logic.
Look-ahead output buffer	Output FF fed from next-state values to give glitch-free outputs without extra delay.
LUT	Look-up table: $2^n \times 1$ memory that implements any n-input function.
Make code	PS2 keyboard code sent when a key is pressed (A = `1C`).
MAR	Memory address register: holds the address sent to memory.
Mealy output	Output depending on state and input.
Metastability	Unstable FF state when setup/hold times are violated.
Microprogramming	Control unit implemented as a ROM of control words.
Mod-m counter	Counter from 0 to $m-1$ that wraps.
Moore output	Output depending only on the state.
Multiplexer	Selects one of 2^n inputs with n select lines.
Noise margin	Voltage gap between guaranteed output level and required input level.
`numeric_std`	IEEE package with `signed`/`unsigned` and their arithmetic.
Object-mapped scheme	Screen objects generated by circuits comparing scan coordinates with object regions.
Open collector	Output that only pulls low; needs a pull-up; allows wired-AND.
Oversampling	Sampling a serial line faster ($16\times$) than the bit rate to find bit centres.
Pad delay	Delay through an FPGA I/O pad; often larger than internal delays.
Parity bit	Extra bit for error detection (odd/even number of 1s).
PicoBlaze (KCPSM3)	Xilinx 8-bit soft-core microcontroller.
Pixel tick	25 MHz enable tick used by the VGA circuits.
Place and route	Implementation step fixing cell locations and wiring.
Port map	Connection of formal ports of a component to actual signals.
Process	Concurrent statement whose body is sequential; used for routing and memory templates.
Program counter	Register holding the address of the next instruction.
Register file	Array of registers with write port and read port(s).
Retrace	Period in which the CRT beam returns; video must be off.
Ring counter	Shift register circulating a single 1; SAP-1 T-state generator.
RT operation	$r_{dest} \leftarrow f(r_{src}, \ldots)$, stored at the next clock edge.
SAP-1	Malvino's Simple-As-Possible 8-bit computer: 16-byte RAM, 5 instructions, 6 T-states.
Scan code	Code sent by a PS2 keyboard for key activity.
Sensitivity list	Signals that trigger a process; all inputs for combinational logic.
Single-access port	Input port whose data must be removed after reading.
Slice	Two logic cells.
Soft core	Processor delivered as HDL/netlist and synthesised into the FPGA fabric.
SR latch	Cross-coupled NOR (or NAND) gates storing one bit; $S = R = 1$ forbidden.
SRAM	Static RAM: each bit a 6-transistor latch; fast, no refresh.
Stream mode	PS2 mouse mode sending packets on movement (enabled by `F4`).
Strobe	One-cycle signal marking valid I/O data (`read_strobe`, `write_strobe`).
Synchronous design	All FFs on one clock edge; slow parts use enable ticks.
Synthesis	Translating HDL into a gate-level netlist.
T flip-flop	Toggles when $T = 1$: $Q^+ = T \oplus Q$.
T-state	One clock period of an instruction's machine cycle.
Technology mapping	Mapping generic gates to the FPGA's LCs and IOBs.
Testbench	Simulation-only VHDL that applies stimulus to a UUT.
Tick	Signal asserted for exactly one clock cycle.
Tile-mapped scheme	Screen divided into tiles; tile memory stores pattern codes.
Tri-state buffer	Buffer with high-impedance output (`'Z'`); only in IOBs on Spartan-3.
TTL	Transistor-transistor logic (74xx), 5 V bipolar family.
Two's complement	Signed binary: negate by inverting all bits and adding 1.
Typematic	Repeated make code while a key is held.
UART	Universal asynchronous receiver and transmitter.
Video memory	Memory holding the displayed pixel or tile data.
W bus	SAP-1's shared 8-bit bus driven by tri-state register outputs.

22 Mock exam (4 hours)

A full practice paper built from the chapter exercises; use it under exam conditions

Answer keys point to sections

⋆ Part A: short answers (40%, about 6% each)

1. Explain what a LUT is and how a 4-input LUT implements any 4-input function. What else can a Spartan-3 LUT be used for? [§2.2]
2. Explain the three rules that avoid unintended latches, and show a process that breaks one of them. [§3.5.4]
3. Compare Moore and Mealy outputs using the rising-edge detector. Draw both state diagrams and a timing diagram. [§5.3.1]
4. What is the delayed store in an ASMD chart? Give an example and the fix. [§6.1.3]
5. Explain bit-, tile- and object-mapped pixel generation and estimate the memory of each for 640×480. [§12.3]
6. Draw a full adder from gates, then a 4-bit adder-subtractor. Compute $6 - 9$ in 4-bit two's complement and state the flags. [§0.C]
7. List the fetch and ADD T-states of SAP-1 with their RT operations. [§0.F]

⋆ Part B: design problems (60%, about 15% each)

1. **Counter design.** A circuit shows seconds 00–59 on two 7-segment digits using the 50 MHz clock. (a) Block diagram with a prescaler tick and two BCD counters. (b) VHDL for the prescaler and the counters in the two-segment style. (c) Why must the ones digit not clock the tens digit? [§4.5.2]
2. **FSMD.** Design a circuit that counts the 1s in an 8-bit input after `start` and asserts `done_tick`. (a) ASMD chart. (b) VHDL with `ready`/`done_tick`. (c) How many cycles does it take? [§6.3]
3. **UART.** (a) Draw the 8N1 frame for `0x41` and mark the sample points. (b) Compute the baud divisor for 9600 baud. (c) Explain the receiver ASMD states and counters. (d) Design an interface that avoids data overrun. [§7]
4. **PicoBlaze.** A switch (port 00) sets a value; a button (port 01, single-access) requests that the value is doubled and sent to LEDs (port 00, output). (a) Draw the input and output interface with decoding and the flag FF. (b) Write the assembly loop. (c) What changes if the button is served by interrupt? [§16, 17]

Self-check after the mock

For each question note: *Did I draw a diagram? Did I state assumptions (clock, reset, widths)? Did I use the two-segment style? Did I give the timing numbers?* Re-read the chapter of every question below 70%.